
OPTIMIZING POWER GRID TOPOLOGIES WITH REINFORCEMENT LEARNING: A SURVEY OF METHODS AND CHALLENGES

Erica van der Sar
 Department of Mathematics
 Vrije Universiteit
 Amsterdam, The Netherlands
 e.t.van.der.sar@vu.nl

Alessandro Zocca
 Department of Mathematics
 Vrije Universiteit
 Amsterdam, The Netherlands
 a.zocca@vu.nl

Sandjai Bhulai
 Department of Mathematics
 Vrije Universiteit
 Amsterdam, The Netherlands
 s.bhulai@vu.nl

May 16, 2025

ABSTRACT

Power grid operation is becoming increasingly complex due to the rising integration of renewable energy sources and the need for more adaptive control strategies. Reinforcement Learning (RL) has emerged as a promising approach to power network control (PNC), offering the potential to enhance decision-making in dynamic and uncertain environments. The Learning To Run a Power Network (L2RPN) competitions have played a key role in accelerating research by providing standardized benchmarks and problem formulations, leading to rapid advancements in RL-based methods. This survey provides a comprehensive and structured overview of RL applications for power grid topology optimization, categorizing existing techniques, highlighting key design choices, and identifying gaps in current research. Additionally, we present a comparative numerical study evaluating the impact of commonly applied RL-based methods, offering insights into their practical effectiveness. By consolidating existing research and outlining open challenges, this survey aims to provide a foundation for future advancements in RL-driven power grid optimization.

Keywords Power grid reliability · Topology Optimization · Reinforcement Learning · Learning to Run a Power Network

Contents

1 Introduction	3
1.1 Background	3
1.2 Introduction to Power Grids	3
1.2.1 From generation to consumption	4
1.2.2 Topology Optimization	5
1.3 Introduction to Reinforcement Learning	5
1.3.1 Key concepts reinforcement learning	6
1.3.2 RL methods and terminology	7
1.4 Problem Description	8
1.4.1 Reinforcement Learning Framework for Power Grid Management	8
1.4.2 Rules of the game	9
1.5 Research motivation and contributions	10

2 Overview of Challenges and Solutions	10
2.1 Competition and solutions L2RPN 2019 IJCNN - Sandbox	11
2.2 L2RPN 2020 WCCI Competition - A feasibility challenge	11
2.3 L2RPN 2020 NeurIPS - In a sustainable world	12
2.4 L2RPN 2021 ICAPS - Run a power network with trust	12
2.5 L2RPN 2022 WCCI and 2023 TU Delft - Energies of the Future and carbon neutrality	13
2.6 Solutions outside of competitions	13
3 Design choices for RL in Power Grid Control	15
3.1 General RL Techniques	17
3.1.1 Algorithm	17
3.1.2 Action space	17
3.1.3 Observation space	21
3.1.4 Normalization	24
3.1.5 Reward	24
3.1.6 Imitation learning	27
3.1.7 Prioritized replay	27
3.1.8 Exploration strategies	28
3.1.9 Curriculum learning or prioritized training	28
3.1.10 State and action space factorization	28
3.2 Rule Based Techniques	29
3.2.1 Activation threshold	29
3.2.2 Line reconnections and disconnections	30
3.2.3 Revert to reference topology	30
4 Benchmarks, performance metrics and baselines	30
4.1 Benchmarks	30
4.2 Performance metrics	31
4.3 Baselines	32
5 Experimental setup	32
5.1 General setup	33
5.2 Action space experiments	34
5.3 Observation space experiments	35
5.4 Reward function experiments	35
5.5 Curriculum training	35
5.6 Rule-based experiments	36
6 Results, recommendations and guidelines	36
6.1 Baselines	36
6.2 Action spaces	38

6.3	Observation spaces	40
6.4	Reward functions	42
6.5	Curriculum training	44
6.6	Activation threshold	45
6.7	Line switching: Reconnection and disconnection	50
6.8	Revert to reference topology	52
6.9	Rainbow agent	55
6.10	Agent configuration	55
6.11	Evaluating the Rainbow Agent	56
7	Discussion and Future Work	58
8	Conclusion	59
A	Acronyms	64
B	Observation spaces	67
C	Results PPO with different hyperparameters	67

1 Introduction

1.1 Background

Electrical power grids form the backbone of modern society, being responsible for transporting electricity from producers to consumers 24 hours a day, 365 days a year. Operating these grids is a demanding control task that requires continuous monitoring and frequent interventions by skilled experts to maintain network stability, keep power flow within the thermal limits of the equipment, and ensure voltage and frequency levels are met, as highlighted by [1]. The increasing integration of renewable energy sources into the power grid and the rising energy demand add new layers of complexity to this task. With its unpredictable generation patterns, renewable energy makes power flows more variable and less predictable, altering how the network behaves and responds to disturbances. Due to the operational costs of simulation with the current models, operators often rely on their experience to resolve network problems in real-time. Continuing this manual approach may compromise system security or lead to significantly higher costs. According to projections by [2], energy consumption in the Netherlands is expected to grow 30% between 2024 and 2033, with similar trends across Europe ([3]). Meanwhile, the share of controllable energy sources is expected to decrease by approximately 40% due to the shift towards renewable energy. As a result, *Transmission System Operators* (TSOs) will face greater challenges in maintaining network security and reliability, underscoring the need for new grid management strategies.

With its recent advancements and breakthroughs ([4], [5], [6], [7] and [8]), *Reinforcement learning* (RL) has emerged as a promising approach that could assist network operators in decision-making, offering the potential to find cost-effective flexibilities that may currently go unnoticed by human operators. For this reason, in 2019, the French TSO RTE launched the *Learning to Run a Power Network* (L2RPN) challenge introduced in [1], encouraging researchers to use RL for *power network control* (PNC). This challenge seeks to explore how RL can enhance real-time operations and assist with operational planning, ultimately supporting TSOs in navigating the growing complexities of modern power grids. Moreover, the authors of [1] note that RL has already demonstrated potential in a variety of power system problems, reviewed in [9, 10], highlighting its suitability for complex control tasks in this domain.

1.2 Introduction to Power Grids

In this section, we provide a brief background on power grids to facilitate researchers from the RL community stepping into the field of PNC. For a more extensive introduction, see also [11].

1.2.1 From generation to consumption

Electrical power grids are typically divided into two layers, the *transmission grid*, controlled by TSOs, and the *distribution grid*, see Fig. 1. Power is supplied to the network by generators. In the past, these generators were often located close to the consumers, the loads, reducing the transmission distances of the power. However, generation sources such as hydro and nuclear power stations or wind and solar are often situated in geographically remote areas, far from load centers. To reduce resistive losses over these long distances, electricity in the transmission grid is transported at high voltage levels, between 100.000 - 765.000 volts. The voltage in homes in Europe and North America is typically 240 and 120 volts, respectively. Therefore, step-down transformers are used to reduce the voltage for the distribution grid. From the power network's perspective, consumption is aggregated as *load*, and the grid operator's responsibility generally ends at the step-down transformer. To allow for voltage conversion using transformers, the transmission grid operates under *alternating current* (AC). While AC allows for efficient voltage transformation, it also adds complexity compared to direct current (DC) systems, as power flows are governed by nonlinear equations involving both active and reactive power balances.

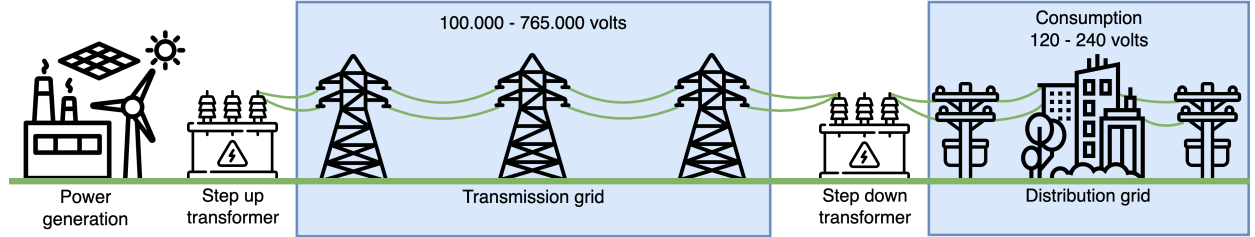


Figure 1: Schematic illustration of power distribution from generation sources to consumers.

A transmission power grid can be represented as a highly interconnected directed graph, with transmission lines represented as edges and *substations* as nodes, see Fig. 2 for a small example. Substations contain the transformers to step up and step down the voltages and connect the generation sources and loads to the grid.

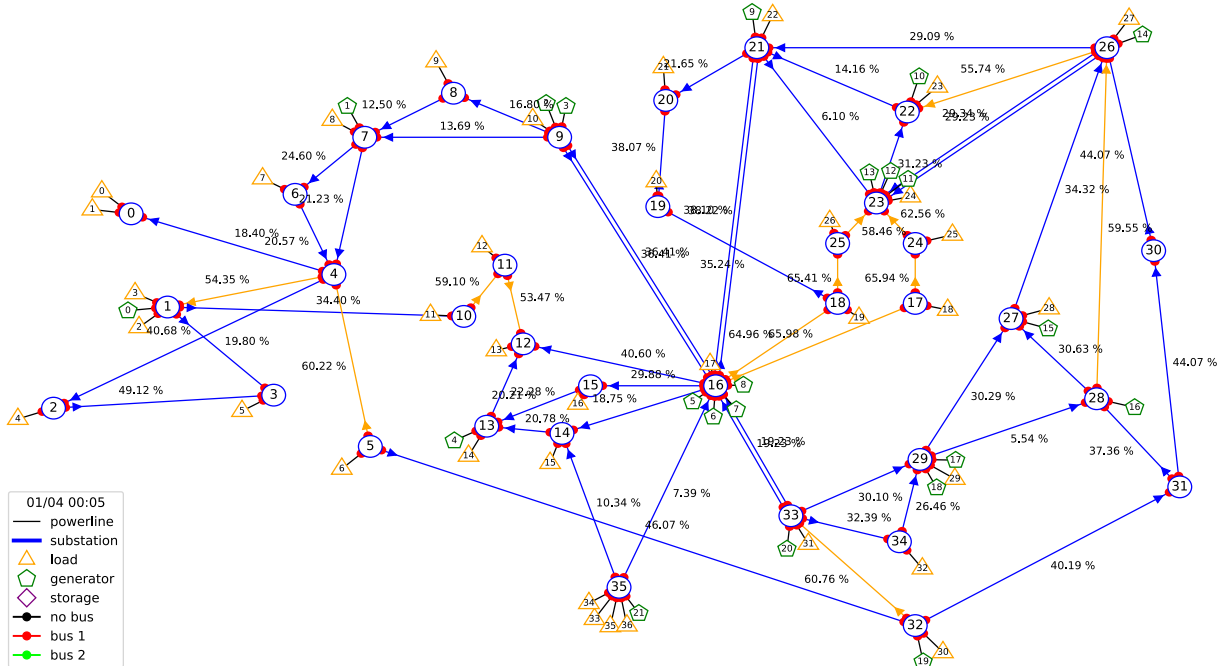


Figure 2: Example of a transmission power grid represented as a directed graph. This example is part of the environment 12rpn_1caps_2021, available in Grid2Op (see Section 1.4).

Human operators continuously monitor and operate the power system from the TSO control centers, intervening when necessary. Operators must always respect three key constraints, described in [11]:

- **Keep the transmission lines within their thermal limits.** Each transmission line has an upper limit to the amount of power that can flow through, determined by its materials' thermal properties. Overloading can lead to overheating, causing lines to sag dangerously close to the ground or damaging critical equipment such as transformers and cables. In such cases, automatic or manual disconnection is often preferable to irreversible damage.
- **Maintain the voltage within a safe range.**
- **Balance generation and load.**

Electricity flows within a power network in the way prescribed by power flow physics. In particular, this means that power cannot be directed from a specific generator to a specific consumer. If a line is disconnected, the power originally flowing on that line redistributes itself across the remaining lines, potentially overloading others. The job of the network operator and the challenge of L2RPN is to efficiently manage the network while avoiding overloads and cascading failures.

Indirectly managing the power flows during congestion by controlling injections, such as redispatching power generation sources, is the most common and well-documented approach in the literature. However, this strategy imposes constraints on producers to be flexible, which comes with additional costs, as mentioned by [12]. TSOs can alleviate congestion also by adjusting the grid topology via two types of topological actions, which, in contrast, have a negligible cost. We distinguish two types of actions: **line switching and bus splitting**. **Line switching consists of remotely disconnecting or reconnecting a transmission line.** Another less explored option is **bus splitting**, modifications performed within a substation (using *node breakers*).

Within each substation, there are multiple *busbars* available to connect the elements. A consistent busbar connection makes a substation a singular node; differing busbar connections split the substation, changing the local network topology. Fig. 4 shows an example of how a bus-splitting action changes the power flows and relieves the congested line in a simulated environment of Grid2Op (the package provided for L2RPN discussed in Section 1.4).

1.2.2 Topology Optimization

The challenges and solutions of L2RPN discussed in this survey paper mostly focus on topology optimization strategies, i.e., those using discrete actions such as line switching and substation reconfigurations. Continuous actions, such as redispatching, are only considered in a few solutions, often as a secondary option.

Topology optimization is a promising tool for relieving congestion, as it exploits existing infrastructure without resorting to costly interventions such as redispatching, load shedding, or demand response, [13, 1]. Despite this potential, topology optimization remains underutilized. Traditionally, the power systems community relies on optimization approaches to solve *optimal power flow* (OPF) problems, including augmented OPF with discrete topological actions, [14, 15, 16]. However, existing optimization techniques struggle to scale up, due to the problem's nonlinear and discrete combinatorial nature, leading to a large and complex optimization space. **Including all topology configuration actions results in a nonconvex mixed-integer nonlinear programming problem, which is NP hard.** Most of the topology optimization research focuses on line switching [17, 18, 19, 20, 21, 22], but neglects the more complex bus-splitting actions, which could offer much more flexibility to the operator [12]. **While bus-splitting is explored in [23, 24, 25, 26], many of these methods still exhibit one or more of the following limitations:**

- **they are computationally expensive,**
- **they are limited to only a small number of substations with bus-splitting options,**
- **they rely on linear (DC) approximation, which neglects reactive power and voltage constraints, and/or**
- **they treat the problem as a static single snapshot, whereas power network control is inherently a sequential decision problem that evolves over time [27].**

These shortcomings highlight the need for new approaches, where RL, a fast-developing field of research specifically designed for sequential decision problems, could play a pivotal role.

1.3 Introduction to Reinforcement Learning

This section provides a brief background on reinforcement learning (RL) for researchers in the power systems community interested in its application to power network control (PNC). For a comprehensive overview of RL, we refer to the book by [28].

1.3.1 Key concepts reinforcement learning

Reinforcement learning is a machine learning technique where an *agent* (the learner and decision maker) **learns from experience** how to make sequential decisions in an environment, **guided by a reward** (feedback).

The sequential decision problem is typically modeled as a Markov Decision Process (MDP), which consists of:

- **State space $\mathcal{S}$** . A state $s_t \in \mathcal{S}$ is the snapshot of the environment observed by the agent at time t , usually represented by a vector or matrix.
- **Action space $\mathcal{A}$** (or $\mathcal{A}_s$ the set of available actions from state s). After observing the state, the agent interacts with the environment at time t by taking an action (decision) $a_t \in \mathcal{A}$ according to its *policy* π .
- **Transition function $p : \mathcal{S} \times \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$** . The quantity $p(s_{t+1} | s_t, a_t)$ describes the probability that action a_t in state s_t leads to state s_{t+1} .
- **Reward function $r : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$** . The quantity $r(s_t, a_t, s_{t+1})$ represents the reward received after transitioning from s_t to s_{t+1} via action a_t . In some settings, this reduces to $r(s_t, a_t)$ or $r(s_t)$.
- **Discount factor $\gamma \in (0, 1)$** , which in a discounted MDP setting, determines the relative importance of future rewards.

Section 1.4 presents the specific MDP setup for the L2RPN problem.

The RL agent interacts with the environment according to a (stochastic) policy $\pi : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$, where $\pi(a_t | s_t)$ defines the probability of choosing the action a_t in state s_t . The goal in RL is for the agent to learn a policy π that maximizes the *expected discounted return*:

$$V_\pi(s) := \mathbb{E}_\pi \left[\sum_{t=0}^T \gamma^t r(s_t, a_t) \mid s_0 = s \right].$$

The function V_π is also referred to as the *state-value function* under policy π . A fundamental property of RL introduced by [29] is the recursive property of the value function, which can be stated as

$$\begin{aligned} V_\pi(s) &= \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} p(s'|s, a) [r(s, a) + \gamma V_\pi(s')] \\ &= \mathbb{E}_{\substack{a \sim \pi \\ s' \sim P}} [r(s, a) + \gamma V_\pi(s')], \end{aligned}$$

where $s' \sim P$ is shorthand for $s' \sim p(\cdot | s, a)$.

Similarly, the *action-value function* (or *Q-value function*) for policy π is defined by

$$Q_\pi(s, a) := \mathbb{E}_\pi \left[\sum_{t=0}^T \gamma^t r(s_t, a_t) \mid s_0 = s, a_0 = a \right] = \mathbb{E}_{s' \sim P} [r(s, a) + \gamma V_\pi(s')].$$

A policy π^* that maximizes the expected return is an *optimal policy*. The optimal state-value function defined by $V^*(s) := \max_\pi V_\pi(s)$, satisfies the *Bellman optimality equation*

$$V^*(s) := \max_{a \in \mathcal{A}} \mathbb{E}_{s' \sim P} [r(s, a) + \gamma V^*(s')].$$

In terms of the *Q-values*, it can be equivalently rephrased as

$$Q^*(s, a) := \mathbb{E}_{s' \sim P} \left[r(s, a) + \gamma \max_{a' \in \mathcal{A}} Q^*(s', a') \right].$$

The general idea of RL is to collect data by interacting with the environment and use this to find an approximation of the value function V or Q and/or to update the policy π .

Deep RL The algorithms applied to the L2RPN problem usually make use of the so-called *Deep RL*. In Deep RL (DRL), the value functions and policies are approximated using a neural network, which is referred to as a parameterized policy and value function, where the parameters, often denoted by θ or ϕ , represent the weights and biases of the neural network. This function approximation enables scalability to large or continuous state/action spaces, in contrast to table-based methods like tabular *Q-learning*.

1.3.2 RL methods and terminology

This survey discusses a variety of (advanced) RL algorithms applied to L2RPN. Describing each algorithm is outside the scope of this survey; here, we provide a few references and highlight the key distinctions between the algorithms discussed in this paper. To help the reader, we categorize the main algorithms used in L2RPN in Fig. 3.

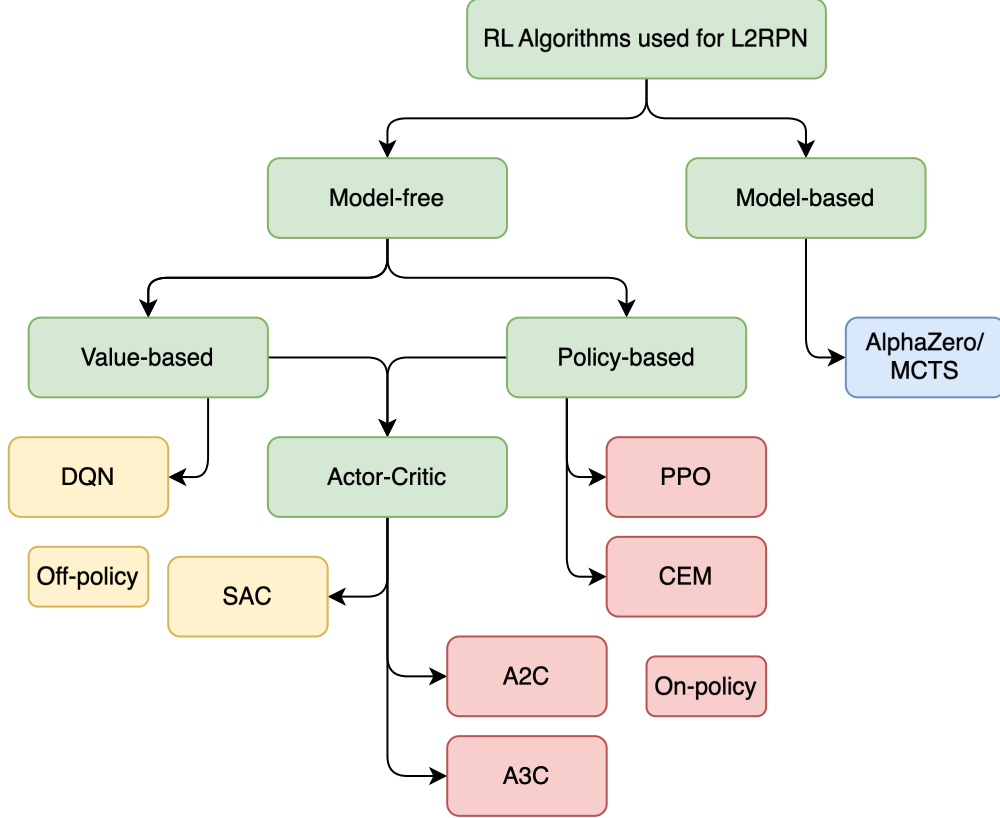


Figure 3: An overview of the algorithms applied to L2RPN categorized by the features discussed in this section.¹

Model-based vs. model-free One of the main distinctions made in RL is between methods that require a model of the environment and methods that do not.

Model-based methods use a model of the environment (either known or learned) to simulate future outcomes and plan ahead. However, an accurate model of the environment is often unavailable or the model needs to be learned, which comes with additional challenges. The most famous example of a model-based approach is *AlphaZero* by [7], also applied in L2RPN.

Model-free methods learn directly from experience without requiring a model. These methods tend to be easier to implement and tune, making them a more popular choice in L2RPN competitions and most other research. Learning is typically done using some variant of *temporal-difference (TD)* updates. A basic TD method update of the value function is

$$V_{k+1}(s_t) \leftarrow V_k(s_t) + \alpha [R_{t+1} + \gamma V_k(s_{t+1}) - V_k(s_t)],$$

where R_{t+1} and s_{t+1} are the observed reward and state at time $t+1$ and $\alpha \in [0, 1]$ is the *learning rate*. The quantity V_k is the old estimate of the value function, while V_{k+1} is the new estimate in iteration k . The quantity $R_{t+1} + \gamma V_k(s_{t+1})$ is the *TD target*.

Value-based vs. policy-based methods Another distinction made between RL methods is whether they are *value-based*, *policy-based*, or a hybrid version of the two, referred to as an *actor-critic* method.

¹This figure is inspired by https://spinningup.openai.com/en/latest/spinningup/rl_intro2.html.

In *value-based* methods, the aim is to learn the values of the states or state-action pairs and then derive a policy from these. A breakthrough in RL was the development of Q -learning, an *off-policy* value-based TD method. In the context of L2RPN, some approaches use a DRL version of Q -learning (DQN), i.e., Double-DQN (DDQN) and Dueling-Double-DQN (D3QN), by [4] and [5].

Policy-based methods learn a policy directly rather than searching for a value function and extracting a policy. This is particularly well-suited for dealing with large or continuous action spaces. In Deep RL, they typically rely on *policy gradient* methods, which optimize the parameterized policy $\pi_\theta(a|s)$ by taking the gradient ascent of J , the expected return of a policy π_θ , with respect to θ , where

$$J(\pi_\theta) := V_{\pi_\theta}(s_0) = \mathbb{E}_{\pi_\theta} [R_\tau]$$

and $\tau = (s_0, a_0, s_1, a_1, \dots)$ is a possible trajectory or episode following policy π_θ . Typically, policy gradient methods are *on-policy* and learn *stochastic* policies, meaning that the policy specifies a probability distribution over the actions. Proximal Policy Optimization (PPO) by [6] and Cross Entropy Method (CEM) by [30] are policy-based methods used in the context of L2RPN. However, CEM uses population-based search and does not rely on policy gradients.

Actor-critic methods combine both paradigms, using a learned value function to reduce the variance of policy gradient updates. Soft Actor-Critic (SAC) and (Asynchronous) Advantage Actor-critic (A2C/A3C) are examples of actor-critic methods applied in L2RPN.

On-policy vs. off-policy *On-policy* methods learn from data generated by the current policy being improved. In contrast, *off-policy* methods can learn from experience generated by a different behavior policy, for example ϵ -greedy with respect to current Q -values, enabling more efficient data reuse and exploration. Using a behavior policy different from the current policy allows for applying techniques such as prioritized experience replay and different exploration strategies, as discussed in Sections 3.1.7 and 3.1.8

1.4 Problem Description

This section describes the platform provided for L2RPN and how the PNC problem is formulated as a Markov Decision Process in Section 1.4.1. Additionally, Section 1.4.2 describes the rules within the L2RPN environment.

1.4.1 Reinforcement Learning Framework for Power Grid Management

To facilitate the L2RPN challenge and further research on this topic, RTE provides an open-source Python package *Grid2Op* ([31]) that simulates realistic power grids. The Grid2Op package enables researchers to develop a sequential decision-making RL model, termed *agent*, capable of controlling the power grid in real-time. The package provides environments with different episodes, each simulating a continuous sequence of power grid states at 5-minute intervals.

Grid2Op provides a typical setup of a *Markov Decision Process* (MDP) defined by a tuple $(\mathcal{S}, \mathcal{A}, p, r)$, where at each time step t an agent observes a state $s_t \in \mathcal{S}$ from the environment and takes an action $a_t \in \mathcal{A}$. The Grid2Op environment computes and returns the next state $s_{t+1} \in \mathcal{S}$ to the agent with probability $p(s_{t+1}|s_t, a_t)$, which is the unknown state transition probability, after which the agent receives an immediate reward $r(s_t, a_t) \in \mathbb{R}$ according to a defined reward function. This agent-environment interaction is visualized in Fig. 4. With this MDP formulation, reinforcement learning (RL) can be used to learn a (stochastic) policy $\pi(a_t|s_t)$ that optimizes the expected discounted reward $\mathbb{E}_\pi[\sum_{t=0}^T \gamma^t r(s_t, a_t)]$, where $\gamma \in (0, 1)$ is the discount factor.

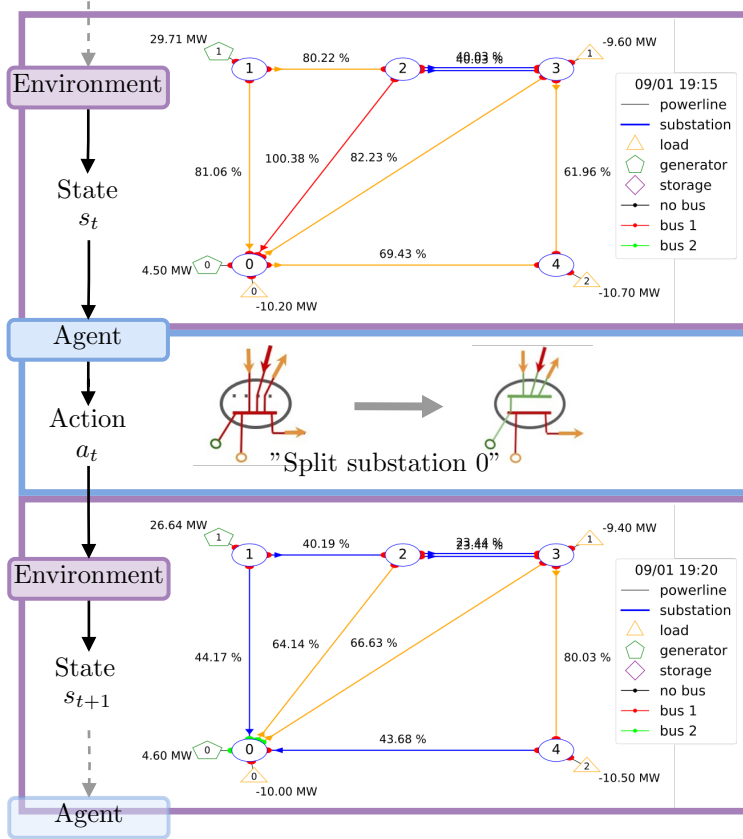


Figure 4: Example of a bus-splitting action by an agent and the effect on the environment. Some elements of substation 0 are assigned to bus 2, which results in a split of substation 0, and a consequent power flow redistribution.

In Grid2Op, the states $s_t \in S$ include all grid information such as the topology of the grid, which includes the connections of the electrical elements, the power flow on the lines, power generation, and load consumption. As mentioned in Section 1.2, the topology of the power grid can be represented as a graph, which we formalize as graph $G = (V, E)$, with nodes $v \in V$, as substations, and edges $e \in E$, as transmission lines, as described in [32]. Each substation connects various *elements* such as line ends, *generators* (producers), *loads* (consumers) and *storage units* (batteries, introduced in [33]).

The total action space $\mathcal{A}$ consists of four types of actions:

1. Bus splitting², shown in Fig. 4 and explained in Section 1.2.1;
2. Line switching (line reconnection/disconnection);
3. Redispatch or curtailment of generator productions;
4. Storage changes (if available).

Bus-splitting and line-switching actions are cost-effective for dynamic response to contingencies, as they indirectly redistribute line flows. These *topological actions* can effectively mitigate contingencies or alleviate line congestion without requiring more expensive generator or load adjustments.

1.4.2 Rules of the game

As discussed in Section 1.2.1 grid operators need to keep the transmission lines within their thermal limit, maintain the voltage within a safe range and balance the generation and load. Similarly, agents in the L2RPN context must

²By default, Grid2Op assigns two busbars per substation and only recently, starting from Grid2Op 1.9.9., it became possible to change this configuration and increase the number of busbars. Therefore, all competitions up to now and the solutions discussed in this paper consider only two busbars per substation.

ensure that no line exceeds its thermal limit. Line loading, denoted by ρ , is defined by the ratio between the amount of power flow passing through and the thermal limit. When a line $e \in E$ is overloaded for too long, i.e. $\rho_e > 1.0$, it will automatically disconnect and a specific number of time steps is needed before this element can be activated again.

Voltage control, and generation-load balance are handled by the environment in the current L2RPN challenge. However, redispatching generation can help relieve line congestion and can be considered a useful action in the RL challenge.

To mimic the physical properties of the power network, each substation and each power line has a *cooldown time*, which is the time needed after activation before this element can be adjusted again. Furthermore, there is a limited number of actions that can be done by an operator or by the technology, therefore, by default, Grid2Op can handle only one substation and one line-switching action in each time step.

In the context of this RL challenge, a *game-over* state will occur when an agent fails to safeguard the power grid, i.e., any network disconnect or isolation of a load or generator. The overarching objective in grid management is to control power flows in a way that prevents overloads and avoids cascading failures that could destabilize the entire network.

The rules described above align with an $N = 0$ setting, meaning no contingencies are anticipated. By adding an adversarial agent, or opponent, in the L2RPN challenge described in Section 2.3, the proposed RL solutions are required to act more securely. In real-world operations, TSOs typically adhere to the stricter $N = 1$ reliability standards, ensuring that the grid remains secure even after the failure of any single grid component.

1.5 Research motivation and contributions

Since the introduction of the L2RPN challenge in 2019 in [1], the field of RL applied to power grid control has seen remarkable growth. Several subsequent challenges have been organized ([1], [34], [35], [33], [36]) and the Grid2Op package has been extended to include larger, more complex networks. These updates introduce additional challenges such as unexpected line attacks, increased penetration of renewable energy, integration of battery storage, and sending timely alerts in case of dangerous contingencies. Each challenge has prompted innovative solutions from the research community, contributing to a rapidly evolving landscape.

This rapid development presents challenges on its own. With a growing number of approaches and techniques emerging from various research efforts, it can be difficult to navigate the landscape and select the most appropriate method. Moreover, some solutions lack thorough documentation, and publicly available codebases often do not clearly convey the rationale behind specific design choices. As a result, replicating or building upon existing methods becomes a cumbersome and time-consuming task.

To address these challenges, this survey paper aims to provide a comprehensive and structured overview of the methods and challenges in reinforcement learning (RL) applied to grid topology control. Our aim is to guide researchers in selecting effective strategies and more easily identifying open research questions in this fast-moving field.

In this paper, we present the following contributions.

1. First, we summarize and analyze the evolution of RL challenges and solutions, providing insights into how the field has progressed over time in Section 2.
2. Next, we review the various techniques that have been proposed to enhance the performance of RL agents in grid topology optimization in Section 3.
3. In Section 4, we discuss benchmarking practices, performance metrics, and baseline methods used in the literature to evaluate RL agents in grid topology optimization, highlighting the best practices.
4. In Sections 5 and 6 we conduct comparative numerical studies to evaluate the effectiveness of selected techniques, providing empirical evidence of their impact on performance.
5. We provide practical guidelines and recommendations to support future research and development in the field, summarized in Section 7.
6. Lastly, Section 8 concludes with a summary of our key insights and directions for future research.

2 Overview of Challenges and Solutions

This chapter provides a comprehensive overview of the L2RPN competitions, highlighting the most significant contributions to each challenge. [37] has been used to help create this overview. Several solutions proposed outside the L2RPN competitions are also discussed in the last part of this section. For a broader understanding and detailed description of the challenges, we direct readers to [11] and the articles mentioned in the specific sections. For many

of the proposed solutions, we introduce acronyms for easier reference and recognition throughout the paper. These acronyms follow the format **AAA-YYYY**, where AAA represents a few descriptive letters and YYYY indicates the year. We provide an overview of all used acronyms and the paper references in [Table 33](#) in [Appendix A](#).

2.1 Competition and solutions L2RPN 2019 IJCNN - Sandbox

The first challenge, introduced in 2019 and described in [\[1\]](#), was based on the IEEE 14-bus system. The platform used for this challenge was PyPowNet ([\[32\]](#)), which has been replaced with Grid2Op in subsequent challenges. In this challenge, the goal was to design an agent that can operate a power grid in real-time over a one-month period using only topology changes. That is, only bus-splitting and line-switching actions were used.

This competition had two winning solutions by [\[38\]](#) (**A3C-2019**), and [\[39\]](#) (**DDQN-2019**).

A3C-2019 is the only solution that does not use the Grid2Op simulation function and is, therefore, purely RL-based. It uses an asynchronous-advantage-actor-critic (A3C) algorithm by [\[40\]](#) with a curriculum learning strategy, see [\[41\]](#), where the idea is that the neural networks learn more effectively when they are first trained on a simpler task. Furthermore, the authors reduce the action space by eliminating all busbar symmetry actions within a substation, since these result in an identical situation and are therefore redundant. This is something that, in subsequent solutions, is done by default and is included when collecting all substation topology actions in the Grid2Op package.

The solution **DDQN-2019** uses *imitation learning* (IL), see [\[42\]](#), combined with a Dueling DQN algorithm, [\[4\]](#), that is trained using importance sampling or prioritized experience replay, [\[43\]](#). They introduce a warning flag based on the loading of a line level. This warning flag can be seen as an *activation threshold* that determines when the agent should interfere with the network to avoid further escalation. When the load on all lines is below the activation threshold, the agent is not activated, but a simple do-nothing action is given to the environment. In all subsequent solutions, an activation threshold is used. In addition, *guided exploration* is used to speed up training. The final solution simulates the top N actions given by the agent and takes the best action.

2.2 L2RPN 2020 WCCI Competition - A feasibility challenge

In spring 2020, the L2RPN challenge was scaled up to a larger grid of 36 substations, representing one-third of the IEEE 118-bus system. The time horizon was increased to one year, with planned maintenance outages that the agent had to account for when operating the network. The set of actions available to the agents in this competition was still limited to topology actions. Compared to the previous challenge in 2019, this challenge is notably much more difficult due to the increased action space size, which is approximately 65k. This underscores the importance of finding an effective method for managing the large action space.

The winning solution of this competition is the approach of [\[44\]](#) (**SMAAC-2021**). This solution combines the Soft Actor-Critic (SAC) algorithm [\[8\]](#) with the use of *afterstates*, which can also be referred to as the post-decision state [\[45\]](#). The afterstate refers to the state after the agent made its move, but before the environment responded. Learning the afterstate instead of the action reduces the output of the agent’s neural network drastically. An intriguing element of this solution is that the SAC features a *Graph Neural Network* (GNN) structure, which is well-suited due to the graphical nature of the power grid. Several other studies have followed up on this idea; for a recent overview of graph reinforcement learning methods applied to power grids, see [\[46\]](#). This survey discusses [\[47\]](#), [\[48\]](#), [\[49\]](#).

Furthermore, the authors of [\[44\]](#) note that the usage of the activation threshold yields a semi-Markov decision problem (semi-MDP) for the RL agent. Together, this leads to the Semi-MDP Afterstate Actor-Critic solution (SMAAC). In **SMAAC-2021**, the RL part of the agent focuses solely on the topology actions of the substations, while the line-switching actions are rule-based and follow a simple rule: reconnect a line whenever it becomes disconnected. This line-reconnection rule is a new addition compared to previous solutions, and has been adopted in most subsequent approaches.

To the best of our knowledge, the solution of the second-place winner, is unfortunately not publicly available nor documented. However, we believe this solution likely served as a precursor to the winning approach of the L2RPN 2020 NeurIPS competition, based on the team’s name, which matches the GitHub handle of a contributor to that solution.

The third-place winners of the 2020 WCCI competition [\[50\]](#) (**A3C-2020**) created a solution inspired by the two winners of the previous competition. **A3C-2020** employs an A3C algorithm. They reduced the action space to 596 actions based on simulation experience and some random selection. Similarly to **DDQN-2019**, they used guided exploration during training. Two agents were trained and combined in their final submission. When one agent fails to propose a good action according to the simulation, the other agent is activated.

2.3 L2RPN 2020 NeurIPS - In a sustainable world

The L2RPN 2020 NeurIPS challenge was held in the summer of 2020. The competition design can be found in [34], and the results of this challenge are discussed in [51]. This challenge consisted of two parts; the robustness challenge and the adaptability challenge. For both parts, the agent now also has access to continuous redispatching and curtailment actions in addition to topological actions.

The robustness challenge uses the same grid employed for the 2020 WCCI challenge. In addition, an adversarial agent is introduced that attacks certain grid lines [52].

In the adaptability challenge, the grid size is increased to 118, thus including the whole IEEE 118-bus system. Additionally, there has been a shift in the energy production mix, with the share of renewable sources increasing from 10% to 30%.

The solution of [53], Search with Action Set (SAS-2021), won first place in both competitions. SAS-2021 works as follows; the stochastic policy $\pi_\theta(a_t|s_t)$, parameterized by θ using a neural network, first outputs a vector of probabilities with which the actions are sampled. Next, the top K actions with the highest probabilities are selected to form the action set A . The final action is greedily selected using a risk function based on the load of the lines. This part can be compared with the guided exploration technique first used in DDQN-2019. To update the policy, they apply evolutionary strategies [54] with black-box optimization to maximize the reward. In this competition, a new rule-based strategy is introduced: The agent SAS-2021 reverses the topology to the reference topology, where all elements are connected to the original busbars, when the network is in a safe state³.

[55] (Binbinchen-2020) won the second-place for the robustness challenge. The authors propose a *Teacher-Tutor-Junior-Senior* framework. Here, the Teacher first defines a smaller action space of 208 actions by applying a greedy agent to the entire action space and saves all used greedy actions in a reduced action space (RAS). The RAS is used by the Tutor, the tutor applies actions, again greedily, and the Junior updates a neural network using IL. Ultimately, the Senior employs this neural network to initialize the actor network for the Proximal Policy Optimization (PPO) algorithm by [6].

The second place in the adaptability challenge was won with an approach that used a straightforward expert system that ran simulations on a limited set of 200 topological actions.

The third place for both challenges was achieved by [56] (D3QN-2020) using a Dueling-Double-DQN algorithm, combining [4] and [5]. Their solution was inspired by the solution A3C-2020 of [50], using guided exploration and combining two agents that work together with different strategies. Similarly to the SAS-2021 solution, they also included the rule-based strategy to revert to the reference topology when in a safe state for the robustness track. Furthermore, they focused on creating a suitable reward function to stimulate the agent to take the right actions.

2.4 L2RPN 2021 ICAPS - Run a power network with trust

In 2021, the L2RPN challenge was organized with the focus on building trust between an agent and the human operator. The grid used was the 36 substation grid as in the 2020 NeurIPS robustness challenge, again including events such as planned maintenance and line attacks. In addition to the previous challenge, the agent can raise an alarm when the assistance of an operator is needed due to a risk of failure. The alarm score function is defined as follows: When a game-over occurs, but the agent raises an alarm within a decent time frame, the agent receives a positive alarm score. Otherwise, the agent will receive a negative score. The grid is divided into three areas, and if the alarm is raised in the correct area, the score will be higher. To avoid overactive agents, the alarm has an attention budget. For more details, see the description of the challenge and the results in [35].

None of the ICAPS 2021 competition winners wrote a paper to our knowledge, making it difficult to find documentation on their solutions, but the presentations of their solutions can be found at [57].

The first place was achieved by [58]. Their solution consists of four modules: an *Expert module*, an *Agent module*, an *Emergency module* and a *Alarm module*. The expert module consists of rule-based parts that are similar to previous solutions, e.g., reconnecting disconnected lines and going back to the reference topology when the grid is in a safe state. The agent module seems to be based on the solution SAS-2021 ([53]), using the same action space and the neural network from this solution for the policy network that recommends actions. They added two extensions to the solution: (1) In case a single topology action is not able to solve the contingency, a multi-step topology action is tried. (2) An emergency module; If topology actions cannot solve the problem, redispatch actions are tried to reduce the overflow.

³Based on their code accessed on 25-10-2024 <https://github.com/PaddlePaddle/PARL/tree/develop/examples/NeurIPS2020-Learning-to-Run-a-Power-Network-Challenge>.

Moreover, if these actions are insufficient and the grid is at risk of failure in the next time step, line disconnection actions are evaluated using simulation. Lastly, the alarm module raises an alarm when the agent cannot solve the overflow.

The team of [59] won second place in the 2021 ICAPS competition. The problem is split into an operational and an alarm strategy. Their solution uses the action space of Binbinchen-2020 ([55]) with only 208 actions. The operational strategy is a simple greedy strategy: If the grid is in danger, all actions are simulated, and the action with the lowest maximum line load $\rho_{\max}$ value is chosen. The alarm strategy is a bit more advanced; they used a combination of rule-based strategies, with a neural network.

The third place in the competition was secured by [60]. This solution served as a precursor to their winning approach for the 2022 WCCI competition AlphaZero-2022 ([61]) also using the AlphaZero approach with Monte Carlo Tree Search (MCTS) sampling by [7], as the core RL algorithm. Notably, in this solution, they preserved the original action space to avoid losing potential beneficial actions. A benefit of their solution is that the MCTS tree already requires looking into the future and, therefore, helps identify critical situations early. This capability is then used for the alarm strategy.

2.5 L2RPN 2022 WCCI and 2023 TU Delft - Energies of the Future and carbon neutrality

In the challenge of 2022 introduced in [33], the energy mix is changed to obtain more realistic future scenarios to reach carbon neutrality in 2050; 40% of the energy mix consists of renewables; wind and solar power and less than 3% is fossil fuels. The use of renewable energy generators is privileged and the use of fossil fuels is penalized to steer agents. The results of the competition can be found in the blog post of [62].

As mentioned in the previous section, [60] improved their solution submitted to the L2RPN 2021 ICAPS competition. Unlike their previous solution, the action space is now narrowed down to 2000 actions using a brute-force selection method. In addition to the learned topology agent, they introduce a redispatching controller that uses Cross-Entropy (CE) optimization by [30]. Topology actions are preferred over redispatch actions, but when both cannot solve the congestion, they rerun the redispatch optimization on the top five proposed topology action candidates. With the improved solution AlphaZero-2022, [61] won the 2022 WCCI competition. The authors also wrote a follow-up paper on how to integrate such an RL-based agent into existing grid operation workflows, [63].

The solution of [64] (BruteForce-2022,) obtained the second place, using a brute-force search for topology actions and the optimization agent optimCVXPY from L2RPN baselines [65]. They used the method of the Teacher in Binbinchen-2020 to gain a reduced action space of 314 actions. The authors also experimented with RL-based agents but were unable to achieve better results, showing the difficulty of this problem.

The third place in the 2022 WCCI competition was achieved by [66] (HRI-EU-2022). Their agent also did not include any learning. They randomly selected 1000 line-switching actions combined with redispatching actions, and at each time step, select the action that reduced the maximum line loads $\rho_{\max}$ the most. Note that they do not make use of substation topology actions.

The competition of [33] was repeated in 2023 [36], which resulted in two new interesting results.

The solution of [67] (LJN-2024) won the competition, using the curriculum agent (Curriculum-2023) approach of [68], discussed in Section 2.6, combined with the optimCVXPY baseline agent of L2RPN. They reduced the action space using three types of Teachers. A general teacher who finds the best action when an overflow occurs on the grid, an attacking teacher who finds the best action in case of an attack on a power line, and finally a $N - 1$ teacher who tries to find the best actions to mitigate the effect of a potential attack on a power line.

The second place was for the team of [69] (Artelys-2024) who used a very similar approach, also building an agent based on the Curriculum-2023 approach of [68]. For the continuous redispatching actions, they used convex optimization (optimCVXPY). Their approach for using redispatching actions is very much the same as in the approach of AlphaZero-2022 ([61]), where an additional note is made that the agent prefers actions that do not limit renewable energy sources.

2.6 Solutions outside of competitions

Quite some researchers proposed interesting solutions outside of competitions; the most relevant solutions to our knowledge will be chronologically discussed in this section.

2021 In the paper of [70] (CEM-2021), the Cross-Entropy Method by [30] is applied to a modified version of the 14-bus power grid, case_14_realistic in Grid2Op. With this simple RL approach and a smart selection of episodes, the authors were able to train an agent to successfully operate 96.5% of the scenarios consisting of one week. The research considers bus-splitting actions only and proposes a reduction of the action space by excluding actions where

only one element is connected to a busbar and actions where none of the connected elements is a line (these are infeasible actions and are currently excluded by default when collecting topology actions in Grid2Op). Furthermore, they show how the number of actions per substation can be computed. An interesting part of this research is the evaluation of the agent, which shows that the agent only needs very few different topologies to manage the grid. Their agent manages the grid with topology actions at substations 1, 3, and 8.

2022 [71] (D3QN-2022) also consider the 14-bus power grid with only bus-splitting actions. They use RL-lib to apply multiple versions of the DQN algorithm and compare their performance, noting that Double Dueling DQN with prioritized replay has the best performance. They use a highly reduced action space considering only 9 actions and a limited part of the observation space compared to other solutions. The final agent is tested on scenarios of one month and, on average, survives 82% of the 8064 time steps.

The topological agent in [72] minimizes line overflow by solving a mixed integer linear programming (MILP) based on linear constraints from DC approximation. [72] propose a three-step pipeline to define a grid segmentation used for a multi-agent approach (a decentralized control technique with topological closed-loop controllers). First, they create an influence graph using *line outage distribution factor* (LODF). Second, a clustering of this influence graph is made. Lastly, clusters are selected using quality criteria.

2023 The work of [73] (PowRL-2022) considers the 2020 NeurIPS robustness challenge, which concerns the 36-bus power grid and an attacking opponent. Similarly to [70], they show a formula to compute the number of feasible topological bus-splitting actions, without the one-element constraint. The RL agent considers bus-splitting actions only. Before training, the action space is reduced to 240 actions using brute-force simulation. The authors describe how they combine heuristics with deep RL. Here, the heuristics are the rule-based parts of the method, which are similar to previously proposed methods: do not activate the RL agent when there is a safe state, reconnect lines when possible, and revert to the default network topology when the contingency ends. The authors introduce one rule that is different from previous solutions: Disconnect lines when there is a sustained period of overflow to avoid permanent damage. The RL agent is trained using PPO with prioritized replay.

The solution [Binbinchen-2020] is extended in the paper of [68] (Curriculum-2023). This work focuses on the robustness track of the 2020 NeurIPS challenge. They extend the action space of [Binbinchen-2020] by introducing an $N - 1$ Teacher and an $N - 1$ Tutor for imitation learning of the Junior agent. The $N - 1$ algorithm used for the Teacher and the Tutor selects the best $N - 1$ secure action, rather than defaulting to a greedy action that assumes all lines remain connected. This is achieved by simulating the combination of each topological action with the disconnection of each line from a predefined subset, and selecting the best $N - 1$ secure actions based on the resulting $\rho_{\max}$ values. This enhancement increased the considered action space of [Binbinchen-2020] from 208 to 508 actions. In addition, they added a topology reversion improvement, as used in solutions [SAS-2021], [D3QN-2020] and [PowRL-2022]. Lastly, the authors made some important code improvements; for details, see [68]. This leads to an improved score of 49.12 where [Binbinchen-2020] got 46.89 according to [51].

A novel approach using hierarchical reinforcement learning is introduced in [74] (HRL-2023). The problem considered is the modified 14-bus network (as in [70]) with and without an adversarial agent. In the hierarchical structure, the authors consider three levels. At the highest level, the agent needs to decide whether an action is needed. This is a rule-based agent that uses the activation threshold as in previous solutions. At the intermediate level, a new agent is introduced that determines where an action is needed. This agent is RL-based, the experiments include both SAC as well as PPO algorithms. At the lowest level, the agent picks the specific bus-splitting action. Experiments are performed with both greedy and RL-based algorithms at the lowest level. The intermediate level forces the lowest level agent to act at the correct substation by applying an action mask.

The authors of [47] (MARL-2023) play with a similar idea, but extend the RL-based lowest level with substation-specific agents in contrast to the method of [HRL-2023] where the lowest level consists of one agent that is steered by the intermediate level agent using an action mask. This concept results in a multi-agent reinforcement learning method (MARL). The authors do experiments using the SACD or PPO algorithm with dependent or independent multi-agents. In this research, the intermediate or mid-level agent is purely rule-based. The concept is applied to the 5-bus power grid, leaving the research applied to larger grids to be explored.

2024 [75] introduce a hierarchical multiobjective Markov decision problem (HMO-MDP) to address more fairness between the power plants. This is done by introducing a suitable reward function and is tested using (Advantage Actor-Critic) A2C and PPO.

In [76] (HUGO-2024), the authors propose new enhancements to their previous [Curriculum-2023] solution. In this approach, the greedy Tutor is used to learn *Target Topologies* (TTs), which are selected based on their robustness. A

target topology is a topology that was found to be safe after a contingency. Here, safe means that the agent is not triggered by the activation threshold to adjust the topology of the network due to a high $\rho_{\max}$ value. The most frequent TTs are saved and used Greedily in the final agent when the maximum load on the network lines is above a certain activation threshold. This activation threshold is slightly lower than usual because often multiple actions are needed to reach the final TT. The authors tested the proposed method on the 118-bus network of the WCCI 2022 challenge. They improved on their previous solution, which had a mean survival time of 1160, to a mean survival time of 1436.

In the study conducted by [48] (IL-2024) IL is evaluated on the 14-bus network with and without planned and unplanned outages. An agent is trained using data from two expert agents, a greedy and an $N - 1$ expert, for details, see [48]. These expert agents can be regarded as the Tutor in Teacher-Tutor-Junior-Senior framework used in Binbinchen-2020 and Curriculum-2023. The agent trained using imitation learning can be compared to the Junior.

An interesting topic that is addressed in the paper of [77] (BDQN-2024) is the imbalance of states and actions in the PNC problem. They note that this leads to degradation in the agents’ performance and propose a Balance DQN algorithm to tackle this issue. This algorithm is built upon the ApeX-DQN framework of [78] and uses k -means-based predefined curriculum learning (KPCL) in combination with Option-DQN, inspired by the option framework in [79], to tackle the imbalance issue.

In [80] (Zonal-2024) the authors address a slightly different approach to the L2RPN problem. Their agent acts as a zonal controller that receives a plan from a planner. The RL-based part of the agent focuses on the continuous actions; curtailment and storage charging plans. A heuristic expert agent manages the topological actions and determines whether an intervention from the RL agent is necessary in emergency situations. The RL-agent, trained with PPO, receives both grid information and the planner’s target plan as state input, aiming to adhere to the plan as closely as possible.

[81] propose a state and action space factorization technique to distribute the PNC task across multiple agents. Unlike previous multi-agent approaches that rely on the network’s graph structure, their method uses a data-driven decomposition of the problem. While this technique has not yet been integrated into an RL-based solution, it shows potential for PNC applications.

2025 The authors of [49] build on their previous work on IL in [48], investigating the use of graph neural networks (GNNs) versus fully connected neural networks (FCNNs) for grid topology control (GNNIL-2025). They propose two GNN variants, referred to as heterogeneous and homogeneous, based on different graph representations. Their results show that both GNN types generalize better than FCNNs, with the heterogeneous GNN achieving the best overall performance.

[82] propose a centrally coordinated multi-agent architecture (CCMA-2025) where multiple regional agents suggest actions, and a central agent selects the final action for the grid topology configuration. The authors explore several variants, differing in whether the agents follow rule-based, RL-based, or greedy strategies. Among these, the RL-Greedy approach — where multiple greedy agents propose actions and an RL-based central agent makes the final decision — performs particularly well. Their experiments focus on case 14, both with and without an opponent, and suggest potential for action space factorization in larger cases.

A recent study on IL for PNC, [83], introduces a *soft-label* IL approach (SoftIL-2025). Instead of directly learning from greedy-based action selection, as in previous methods, all actions are labeled based on the line loads during the simulations, providing a richer learning signal for the agent. They evaluate this method using FCNN and GNN architectures on the 118-bus case (WCCI 2022 L2RPN). The soft-labeled GNN, in particular, achieves strong performance, outperforming current state-of-the-art RL agents.

3 Design choices for RL in Power Grid Control

This chapter explores the key design choices involved in applying RL to PNC, with a focus on the implementation of various techniques and how they differ across solutions. Table 1 provides an overview of the techniques used in a selection of the proposed methods described in Section 2. Due to incomplete documentation, not all solutions are represented in this or subsequent tables.

This section is divided into two distinct sections. Section 3.1 covers RL-specific design choices, such as action and state space definitions, reward function shaping, and other related matters. Section 3.2 focuses on heuristic, rule-based decisions, such as activation thresholds and line reconnection strategies.

Solution Acronym	Competition	Size Grid	With Opponent	Rank	Algorithm used	Imitation learning	Prioritized replay	Simulation used	Activation threshold	Reconnect lines	Revert topology
A3C-2019	IJCNN 2019	14		1/2	A3C				0.6 ⁴		
DDQN-2019	IJCNN 2019	14		1/2	DDQN	✓	✓	✓	0.885		
SMAAC-2021	WCCI 2020	14, 36		1	SAC				0.9	✓	
A3C-2020	WCCI 2020	36		3	A3C			✓	0.8		
SAS-2021	NeurIPS 2020	36, 118	✓	1	ES			✓	1.0 ⁵	✓	✓
Binbinchen-2020	NeurIPS 2020	36	✓	2	PPO	✓		✓	1.0 (Teacher) 0.925 (Tutor) 0.9 (Senior) 0.999 (final agent)	✓	
D3QN-2020	NeurIPS 2020	36, 118	✓	3	D3QN		✓	✓	1.0 ⁶	✓	✓
AlphaZero-2022	WCCI 2022	118	✓	1	MCTS + CE			✓	0.98 ⁶	✓	✓
BruteForce-2022	WCCI 2022	118	✓	2	Greedy + OptimCVXPY			✓	0.95	✓	✓
HRI-EU-2022	WCCI 2022	118	✓	3	Greedy			✓	1.0 ⁵	✓	
LJNAgent-2024	WCCI 2023	118	✓	1	PPO + OptimCVXPY	✓		✓	[0.6; 1.99] (Teachers) 0.99 (final agent)	✓	✓
Artelys-2024	WCCI 2023	118	✓	2	PPO + OptimCVXPY	✓		✓	0.98	✓	✓
CEM-2021	-	14			CEM				0.95	✓	
D3QN-2022	-	14			D3QN		✓		0.95		
PowRL-2022	-	36	✓		PPO		✓	✓	0.95 ^{6,7}	✓	✓
Curriculum-2023	-	36	✓		PPO	✓		✓	0.925 (Teacher) 0.9 (Tutor) 0.9 (Senior) 0.95 (final agent) ⁴	✓	✓
HRL-2023	-	14	✓		PPO / SAC			✓	0.95	✓	
MARL-2023	-	5			MA-PPO / MA-SACD				0.9	✓	
HUGO-2024	-	118	✓		PPO	✓		✓	0.85 (TopoAgent) 0.95 (final Senior)	✓	✓
IL-2024	-	14	✓		IL	✓		✓	0.97		✓
HMO-2024	-	14, 118	✓		A2C/PPO				N.A. ⁸	N.A.	N.A.
BDQN-2024	-	14, 36			ApeX-DQN		✓		N.A.	N.A.	N.A.
Zonal-2024	-	118	✓		PPO				0.9	N.A.	N.A.
GNNIL-2025	-	14	✓		IL	✓		✓	0.97	✓	
CCMA-2025	-	5, 14	✓		PPO			✓	0.95	✓	✓
SoftIL-2025	-	118	✓		IL	✓		✓	0.9	✓	✓

Table 1: Overview of discussed solution methods.

⁴Based on code.⁵ This activation threshold is compared with the $\rho_{\max}(t + 1)$ after a simulated DN-action. So unlike other solutions where the current $\rho_{\max}(t)$ is checked.⁶Although not explicitly documented, this value was used in their ICAPS 2021 solution, suggesting it was applied again here.⁷The specific activation threshold is not documented, but given that the reward function uses a safety threshold of 0.95, likely, this value was also applied as the activation threshold.⁸Not Available. It is either not documented or not used.

3.1 General RL Techniques

3.1.1 Algorithm

As shown in Table 1, the most frequently used RL algorithm for PNC is PPO, featured in 11 of the 26 solutions reviewed in this overview, significantly more often than any other algorithm. Studies comparing PPO with other algorithms ([74, 47, 75]), such as SAC and A2C, consistently show that PPO performs better in PNC. Note, however, that the agent’s performance depends not only on the choice of algorithm but also on other implementation details, which are discussed in the following paragraphs. Additionally, various versions of DQN have been applied to the PNC problem. In [71] (D3QN-2022), the authors demonstrate that the prioritized D3QN, an advanced variant of DQN, significantly speeds up the training. To get an idea of the differences between these methods, we refer the reader to Section 1.3.2.

3.1.2 Action space

As outlined in Section 1, the actions available to operate the power network include bus splitting, line switching, generator re-dispatch or curtailment and, optionally, storage changes (introduced in [33]). Topological actions – bus splitting and line switching – serve as a low-cost option to alleviate thermal overloads on the network, as highlighted in [11]. However, the large, non-linear, and combinatorial nature of the action space makes optimal control of the grid topology beyond current state-of-the-art capabilities ([11, 34]), leaving these topological actions as a significant, yet underutilized, source of operational flexibility in power networks.

The inability of existing methods to navigate such a vast action space has motivated the L2RPN competitions (see [11, 11, 34]), where the primary goal is to use RL, to explore the full range of switching options to control power flows. As a result, topological actions are the main focus of agents used in L2RPN. Table 2 provides an overview of the number of feasible bus-splitting and line-switching actions per case. It shows how the number of bus-splitting actions increases drastically when going from case 14 to case 36.

Note that Grid2Op considers two types of actions for bus splitting and line switching; *set* actions and *change* actions. A *set bus* (or *set status*) action specifies the configuration (or status) that the substation (or line) gets, regardless of its current state. This means that if a substation (or line) is already in the specified configuration (or status), executing the action leaves the configuration unchanged. In contrast, a *change bus* (or *change status*) action defines which elements of a substation (or which lines) to switch to the other bus (or status). This results in an action that depends on the current configuration (status) of the substation (line). Executing the change action will always affect the topology (unless the affected element is in a cool-down state). It is easier to consider only the set actions since the feasibility of a change action depends on the current state of the affected element(s). Therefore, the action spaces discussed in the following refer only to set actions.

To describe the size of the action spaces, we first introduce some useful notations. As mentioned earlier, the power grid can be represented by a graph $G = (V, E)$, where V is the set of substations (nodes) and E is the set of transmission lines (edges). The size of substation $v \in V$ is denoted as $|v|$ and is equal to the total number of elements connected to the substation, that is $|v| = g(v) + l(v) + b(v) + e(v)$, where $g(v)$ is the number of generators, $l(v)$ the number of loads, $b(v)$ the number of storage (batteries), and $e(v)$ the number of lines connected to substation v . A line (edge) $e \in E$ that, when switched off, directly disconnects part of the grid is referred to as a *bridge* (or *cut-edge*), denoted as $b \in B \subset E$. The total number of lines and bridges in the grid is denoted by $|E|$ and $|B|$, respectively.

In most Grid2Op environments and L2RPN challenges, the environment allows for only one new substation configuration at each time step. Therefore, the formula for the total number of feasible bus-splitting actions $|A_{bus}|$ is simply the sum of all possible configurations per substation, that is

$$|A_{bus}| = \sum_{v \in V} |A(v)|, \quad (1)$$

where $|A(v)|$ is the number of topology actions that can be performed on substation v .

Computing all possible configurations of a substation v with two busbars simply leads to $|A(v)| = 2^{|v|}$ options. However, symmetric actions and actions that directly result in a game over by disconnecting a non-line element, i.e., load, generator or battery, from the rest of the grid can be excluded. This simple reduction leads to $|A(v)| = |A_{sym}(v)|$, which can be computed as

$$|A_{sym}(v)| = 2^{|v|-1} - (2^{g(v)+l(v)+b(v)} - 1), \quad (2)$$

also presented by [73]. Note that this computation also includes the actions of substations where only one configuration is possible due to connectivity constraints. Therefore, the result of this computation is slightly higher compared to what is presented in Table 2 which excludes these actions.

The number of feasible line-switching actions is given by

$$|\mathcal{A}_{line}| = |E| - |B|. \quad (3)$$

$|\mathcal{A}_{line}|$ excludes all line-switching actions involving bridges, as they would isolate parts of the network.

Environment	# Subs	# Lines	# Gens	# Loads	# Bridges	$ \mathcal{A}_{bus} $	$ \mathcal{A}_{line} $
case 14 realistic	14	20	5	11	1	150	19
case 14 sandbox	14	20	6	11	1	178	19
case 36 all versions	36	59	22	37	1	66810	58
case 118 2020 NeurIPS	118	186	62	99	8	72107	178
case 118 2022 WCCI ⁹	118	186	62	91	8	72957	178

Table 2: Overview of action spaces per environment¹⁰

Due to the large number of feasible actions and the even greater number of possible network topology states, a majority of the proposed solutions incorporate some form of action space reduction. Tables 3 to 5 provide an overview of the actions considered (both by the RL agent and the rule-based heuristics), the reduced action space sizes, and the reduction strategies used in the solutions discussed in Section 2. The action space sizes refer specifically to those controlled by the RL agent, excluding actions handled by the heuristics. For most solutions, the RL agent focuses on handling the discrete topology actions, in some cases with, but in most cases without, line-switching actions. Combining line-switching and bus-splitting actions is possible, but this naturally yields an even larger action space of size $|\mathcal{A}_{tot}| = |\mathcal{A}_{bus}| \times |\mathcal{A}_{line}|$, which might unnecessarily complicate the problem. Only DDQN-2019, one of the solutions proposed for the first challenge, tried to combine both action spaces in its solution. In all other solutions, the agents stick to only one action per time step, either a bus-splitting or a line-switching action.

Solution Acronym	Actions for RL agent	Original Size Action Space	Reduced Size Action Space	% of actions included	Reduction method	Actions for rule-based heuristics
A3C-2019	Topology actions	$312 + 20 = 332$ ¹¹	$156 + 20 = 176$	50.0%	Symmetry reduction.	N.A.
DDQN-2019	Combined topology actions: bus splitting and line switching.	$156 \times 20 = 3120$	$155 + 19 + 76 = 251$ ¹²	8.0%	N.A.	N.A.
CEM-2021	Bus splitting	150	112	74.7%	$(N - 0)$ -reduction.	Line switching.
D3QN-2022	Bus splitting	178	9	5.1%	$(N - 1)$ -reduction, after which a subset of actions is selected that lead to "desirable power network topologies". Not documented what this entails.	N.A.
HRL-2023	Bus splitting	150	106	70.7%	$(N - 0)$ -reduction. Next, the actions for substations where only one configuration is possible are removed, and one explicit do-nothing action is added.	Line switching.
IL-2024	Bus splitting	150	112	74.7%	$(N - 0)$ -reduction.	N.A.
GNNIL-2025	Bus splitting	150	112	74.7%	$(N - 0)$ -reduction.	N.A.
CCMA-2025	Bus splitting	178	73	41.0%	$(N - 1)$ -reduction.	line switching

Table 3: Action space reductions for case 14, the PyPowNet, realistic, and sandbox version. The first two solutions use case 14 in PyPowNet, which is not mentioned in Table 2.

⁹This environment also includes 7 storage units.

¹⁰Some papers report a different number of bus-splitting actions. This might be related to different Grid2Op versions. The numbers reported here refer to the version adopted in this paper, that is Grid2Op 1.9.3.

In [Tables 4](#) and [5](#) we omitted the ‘‘Original Size Action Space’’ column, as these solutions primarily focus on bus-splitting actions, $\mathcal{A}_{bus}$, for which the action space sizes are already detailed in [Table 2](#). As can be seen in [Table 4](#), the exceptions to this are [A3C-2020](#) and [D3QN-2020](#).

Based on the current solutions, we distinguish seven methods applied to reduce the bus-splitting action space, each of which is explained in detail below.

1. **Symmetry reduction**: This method includes all *feasible* actions while excluding symmetric ones. Although it is now the default approach, it was first introduced during the 2019 L2RPN challenge. The formula for computing the size of this action space is given in [Eq. \(1\)](#).
2. **$(N - 0)$ -reduction** introduced by [\[70\]](#) (solution [CEM-2021](#)): This approach only includes actions where at least two elements (or zero) are connected to a busbar, with at least one of these being a line. According to experts, this strategy yields more stable actions. The number of actions for a substation $v \in V$ now reduces to $|\mathcal{A}(v)| = |\mathcal{A}_{n-0}(v)|$, where

$$|\mathcal{A}_{n-0}(v)| = |\mathcal{A}_{sym}(v)| - (e(v) - \delta_{|v|,2} \cdot \delta_{e(v),2} - \delta_{e(v),1}), \quad (4)$$

with $\delta_{m,n}$ the Kronecker delta function, that is $\delta_{m,n} = 1$ if $m = n$ and 0 otherwise.^{[13](#)}

3. **$(N - 1)$ -reduction** used in [D3QN-2022](#) and [CCMA-2025](#): This method only includes actions where at least two elements (or zero) are connected to a busbar of which at least two elements are lines. The resulting reduced action space is more robust to potential line failures or planned maintenance. Using this strategy, [\[82\]](#) showed that the number $|\mathcal{A}(v)| = |\mathcal{A}_{n-1}(v)|$ of feasible substation configurations in substation v is equal to

$$|\mathcal{A}_{n-1}(v)| = |\mathcal{A}_{sym}(v)| - 2^{g(v)+l(v)+b(v)} \cdot (e(v) - \delta_{e(v),2} - \delta_{e(v),1}). \quad (5)$$

4. **Greedy reduction**: This method narrows the action space to include only the highest-performing actions identified by brute-force search. Extensive simulations are run with a greedy agent that selects actions based on the ρ_{max} value or, in some cases, based on the reward function. The most frequently selected actions across multiple scenarios are then retained. For robustness, the scenarios can include an opponent, as in [Binbinchen-2020](#). Another method to select more robust actions used in [Curriculum-2023](#) and [LJNAgent-2024](#), is with an $N - 1$ Teacher, as described in [Section 2.6](#).
5. **Expert reduction**: The action space can be reduced according to domain knowledge or experience, as in [A3C-2019](#), [D3QN-2020](#), and [HUGO-2024](#). Unfortunately, it is difficult to check what this entails. The experience could also be based on greedy simulations.
6. **Mask reduction** introduced by [\[44\]](#): This approach restricts to actions from substations larger than a specified size M . Let $V_{>M} := \{v \in V : |v| > M\}$. The total size of the action space is now defined by

$$|\mathcal{A}_{bus}| = \sum_{v \in V_{>M}} |\mathcal{A}(v)|. \quad (6)$$

Note that, as mentioned in [Section 2.2](#) in the solution [SMAAC-2021](#) the output of the neural network was mainly reduced by learning the afterstates instead of the specific actions. The size of the output of the neural network, denoted by $|\mathcal{N}_{output}|$ scales therefore linearly, as shown in [Eq. \(7\)](#).

$$|\mathcal{N}_{output}| = \sum_{v \in V_{>M}} |v| - 1. \quad (7)$$

7. **Stable Topo reduction**: This method selects actions that result in a stable topology. For example, in [HUGO-2024](#), the target topologies (TTs) are identified by recording all the topologies that brought the network in a *safe state*; i.e., where $\rho_{max} < \text{activation threshold}$. The 500 most frequently used TTs are saved for the final agent. Another example can be found in [BDQN-2024](#), where each topology action is tested and considered successful if it remains stable for at least four time steps. After an extensive number of simulations, actions with a high success rate are kept, while others are filtered out based on a set threshold.

¹¹This action space includes 312 bus-splitting and 20 line-switching actions.

¹²This action space includes 155 bus-splitting, 19 line-switching, 76 bus-splitting and line-switching combinations and 1 do-nothing action(s).

¹³[Eq. \(4\)](#) is a slightly improved version of the formula presented by [\[70\]](#).

¹⁴Two action spaces are used, $\mathcal{A}_{normal}$, which is used by default and $\mathcal{A}_{exception}$, which is used only when specific lines are disconnected. Here, $|\mathcal{A}_{normal}| = 500$ and $|\mathcal{A}_{exception}| = 232$.

¹⁵This action space includes *change* bus actions. This is different from *set* bus actions which most solutions use. See <https://grid2op.readthedocs.io/en/latest/>.

¹⁶In addition, the action space contains 58 line switching, 40 redispatching and one explicit do-nothing action.

Solution Acronym	Actions for RL agent	Reduced Size Action Space	% of actions included	Reduction method	Actions for rule-based heuristics
SMAAC-2021	Bus splitting	66674	99.8%	Mask reduction with $N = 5$.	Line switching.
A3C-2020	Topology actions	300	0.4%	200 based on <i>experience</i> , 100 randomly selected. In addition, the action space contains 295 line-switching actions.	Line switching.
SAS-2021	Bus splitting	732 ^[14]	1.0%	N.A.	Line switching and redispatching.
Binbinchen-2020	Bus splitting	208 ^[15]	0.3%	Greedy reduction.	Line switching when line is disconnected.
D3QN-2020	Topology actions or redispatch actions	786 ^{[15][16]}	1.2%	Expert reduction.	Line switching when line is disconnected.
PowRL-2022	Bus splitting	240	0.4%	Greedy reduction.	Line switching.
Curriculum-2023	Bus splitting	508 ^[15]	0.8%	Greedy reduction.	Line switching.
BDQN-2024	N.A., but only bus splitting actions seem to be used	1080	1.6%	Stable Topo reduction.	N.A.

Table 4: Action space reductions for case 36.

Although the $(N - 0)$ - and $(N - 1)$ -reduction methods may be more sensible, they are only applied to the smaller case 14 instances. Applying the $(N - 1)$ -reduction to case 36 would reduce the action space to 65985, see [82], which is still nearly 99% of the action space. Hence, for the larger cases, such as 36 and 118, more drastic reduction strategies are typically employed to deal with the exponential growth of the action space. For case 36 and case 118, the percentage of selected bus-splitting actions generally falls within $[0.3 - 1.6]$ and $[0.4 - 2.8]$, respectively, excluding the exceptional cases HRI-EU-2022 and SMAAC-2021.

The challenge in effective action space reduction lies in balancing two critical factors: ensuring that the reduced space still encompasses all the necessary actions to mitigate potential contingencies and avoiding an overly large action space that hinders the RL agent from converging. Most current solutions employ a greedy reduction method, which quickly selects the most promising actions. However, this approach may exclude actions that, while not optimal on their own, could prove valuable when combined with others. For instance, an action might appear less rewarding in the short term, but when considered in conjunction with other actions, it could yield a higher long-term benefit.

Solution Acronym	Actions for RL agent	Reduced size action space	% of actions included	Reduction method	Actions for rule-based heuristics
SAS-2021	Bus splitting	1000	1.0%	N.A.	Line switching.
D3QN-2020	Topology actions	978	1.4%	<i>Expert reduction</i> ¹⁵ . In addition, the action space contains 185 line switching and one do-nothing action.	Line switching.
AlphaZero-2022	Bus splitting	2000	2.7%	<i>Greedy reduction</i> .	Line switching and redispatching.
BruteForce-2022		314	0.4%	<i>Greedy reduction</i> .	Bus splitting, line switching and redispatching.
HRI-EU-2022		N.A.	N.A.	Randomly draw 1000 line switch combined with redispatch actions each time the grid is in danger.	Line switching combined with redispatch.
LJNAgent-2024	Bus splitting	1516	2.1%	<i>Greedy reduction</i> .	Line switching and redispatching.
Artelys-2024	Bus splitting	500	0.7%	<i>Greedy reduction</i> .	Line switching and redispatching.
HUGO-2024	Bus splitting	2030	2.8%	Apply the action space of AlphaZero-2022 with an additional 30 expert actions selected by RTE (<i>Expert reduction</i>). <i>Stable Topo Reduction</i> for the target topologies (TTs).	Line switching.
SoftIL-2025	Bus splitting	2030	2.8%	Apply the action space of HUGO-2024	Line switching.

Table 5: Action space reductions for case 118.

3.1.3 Observation space

The observation or state space of the power network includes all relevant information about the physical network needed for PNC, described in Section 1.1. It should be noted that while extensive information may provide a comprehensive view of the system, it may also introduce unnecessary complexity, thereby posing a challenge for a Deep-RL (DRL) agent to focus on the most crucial aspects. Conversely, an overly reduced observation space might result in suboptimal or uninformed decisions due to missing key information.

Based on the approaches evaluated in Table 6, it can be noted that most authors incorporate key variables such as active power of generators, loads, and lines (gen_p , $load_p$, p_or , p_ex), topology configuration ($topo_vect$) and consistently across all solutions, the line loading (ρ) as features representing the state S_t observed by the RL agent. Notably, the solutions DDQN-2019, AlphaZero-2022 and Zonal-2024 utilize the entire observation space without applying any reductions in state representation referred to as the CompleteObservation¹⁷.

The solution SMAAC-2021 introduces an additional feature *danger*, a binary vector that flags lines $e \in E$ when the loading ρ_e exceeds a predefined danger threshold $\rho_{danger} = 0.9$. Moreover, this solution enriches the state S_t by incorporating a history of previous states, $S_t = [s_{t-n}, \dots, s_t]$, enabling the agent to discern trends, such as rising or falling feature values. Like other solutions that utilize a GNN architecture — such as MARL-2023, GNNIL-2025 and SoftIL-2025 — SMAAC-2021 structures feature values into a *feature matrix*, alongside an *adjacency matrix* that represents the current topology configuration of the graph.

In CCMA-2025, the coordination agent receives, in addition to the information on the grid, the proposed actions and/or action values from the regional agents.

¹⁷<https://grid2op.readthedocs.io/en/latest/observation.html>

The most heavily reduced state space appears in [D3QN-2022](#), where only four features are retained: the voltages of the lines (v_{or} , v_{ex}), the currents or line flows (a_{or} , a_{ex}), the line loads (ρ), and the topology configuration.

It should be noted that not all authors explicitly documented which parts of the observation space were included in the state space provided to the RL agent. In such cases, we extracted this information from publicly available code repositories, if available. Solutions for which the exact observation space could not be determined have been excluded from [Table 6](#).

None of the reviewed solutions explicitly applies systematic feature selection techniques to reduce the state space. Instead, most authors rely on domain knowledge and intuition to select the most relevant features. While this approach has been effective in previous work, it raises the question of whether more principled feature selection methods could further improve performance.

Solution Acronym	Complete Observation	Time stamp	Active power		Reactive power		Loads		Voltages		Line flows	Line loads	Time step overflow	Line status	Topo config (<i>topo_vect</i>)	Cooldown line sub	Maintenance	Extra
			Loads (<i>load_p</i>)	Gens (<i>gen_p</i>)	Lines (<i>p_ex, p_or</i>)	Loads (<i>load_qp</i>)	Gens (<i>gen_q</i>)	Lines (<i>q_ex, q_or</i>)	Loads (<i>load_v</i>)	Gens (<i>gen_v</i>)	Lines (<i>v_ex, v_or</i>)	(<i>a_or, a_ex</i>)	(ρ)					
A3C-2019	✓		✓	✓		✓				✓		✓	✓	✓	✓	✓		
DDQN-2019			✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓		✓	
A3C-2021			✓	✓	✓						✓	✓	✓	✓	✓		✓	
SMAAC-2021			✓	✓	✓							✓	✓	✓	✓		✓	
D3QN-2021			✓	✓	✓							✓	✓	✓	✓		✓	
Bibbichen-2020		✓	✓	✓	✓							✓	✓	✓	✓		✓	
SAS-2021		✓	✓	✓	✓							✓	✓	✓	✓		✓	
CEM-2021			✓	✓	✓							✓	✓	✓	✓		✓	
D3QN-2022			✓	✓	✓							✓	✓	✓	✓		✓	
PowRL-2022		✓	✓	✓	✓							✓	✓	✓	✓		✓	
AlphaZero-2022			✓	✓	✓							✓	✓	✓	✓		✓	
Curriculum-2022	✓	✓	✓	✓	✓							✓	✓	✓	✓		✓	
HRL-2023			✓	✓	✓							✓	✓	✓	✓		✓	
MARL-2023			✓	✓	✓							✓	✓	✓	✓		✓	
LJNAgent-2024			✓	✓	✓							✓	✓	✓	✓		✓	
IL-2024			✓	✓	✓							✓	✓	✓	✓		✓	
BDQN-2024			✓	✓	✓							✓	✓	✓	✓		✓	
Zonal-2024			✓	✓	✓							✓	✓	✓	✓		✓	
GNIL-2025	✓		✓	✓	✓							✓	✓	✓	✓		✓	
CCMA-2025			✓	✓	✓							✓	✓	✓	✓		✓	

Table 6: Selected observation features for the RL agent per solution. For documentation on each feature, see [84].

^aBased on code, not (clearly) documented

3.1.4 Normalization

Once the features from the observation space have been selected, it is important to transform the data to train the model efficiently. *Normalization* entails rescaling the input features to ensure that the values fall within a standard range, such as $[-1, 1]$. This step is essential in machine learning, as neural networks tend to train faster and more effectively when features are on a similar scale. Although most Deep RL solutions for PNC apply some form of normalization, this aspect is often under-reported in the documentation and sometimes entirely overlooked in implementations.

Selecting the appropriate normalization technique can be a complex task in its own right; see [85], [86], [87], [88] and [89]. This paragraph highlights two common techniques used in L2RPN that are effective and accessible for scaling features when training a neural network.

A widely used normalization technique, applied by [50], [68] and [74] is *min-max* (or *linear*) *scaling*. This method is effective when the data is relatively uniformly distributed and does not contain any extreme outliers. The formula for computing the normalized value $X_{normalized}$ of an observed feature value X using the min-max scaling is described in Eq. (8).

$$X_{normalized} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}. \quad (8)$$

The method guarantees $X_{normalized} \in [0, 1]$. For the computation, approximate lower and upper bounds ($X_{\min}$ and $X_{\max}$) are required, which can be obtained by running simulations, for example, using a do-nothing agent. For some features, such as generator production, these bounds are predefined and can be directly derived from Grid2Op.

Another common approach, used by [44], [60], [58], [59], [61] and [47], is *Z-score normalization* (or *standardization*). This technique is more appropriate when the data is roughly Gaussian distributed. In this case, the normalized value is obtained by:

$$X_{normalized} = \frac{X - \mu(X)}{\sigma(X)}, \quad (9)$$

where $\mu(X)$ and $\sigma(X)$ represent the empirical mean and standard deviations, which can be estimated through simulations. This method results in Z-normalized data with a mean of 0 and a standard deviation of 1.

While both techniques make assumptions about the underlying data distribution, real-world data often does not strictly conform to these assumptions. Exploring alternative normalization methods could offer improvements in the training process.

3.1.5 Reward

Selecting an appropriate reward function is essential for a reinforcement algorithm to effectively learn the desired behavior, as emphasized by [90]. This section provides an overview of the reward functions proposed for the various L2RPN challenges. As mentioned in Section 1.3, reward can be a function of the previous state, the action and the new state. All rewards discussed in this section are based only on the state s_t observed at time t , and simply denoted by $r(t)$.

As shown in Table 7, the reward function typically involves two components: the reward during *normal operations* and the reward at the end of an episode. The latter usually incorporates a negative reward upon a game over and sometimes includes a positive reward upon the agent's successful completion of the episode. During normal operations, the reward function should steer the agent in order to avoid a game over. Having a negative or positive reward at the end of an episode encourages the agent to maintain the stability of the network. However, relying solely on terminal rewards can delay learning, as it may take many episodes for the value functions to converge and reflect long-term rewards. Providing meaningful rewards during normal operations offers more immediate feedback, helping the agent differentiate between beneficial and detrimental actions. In the following, we describe various reward functions that have been used to guide agent behavior during *normal operation*.

[1] introduce a commonly used reward function referred to as $R_{Margins}$, detailed in Eq. (10). This reward has been applied (directly or in a scaled fashion) in several solutions, including DDQN-2019, A3C-2020, CEM-2021, HRL-2023, and CCMA-2025 as shown in Table 7. $R_{Margins}$ is one of the default rewards available in the Grid2Op package. This reward is designed to encourage a balanced power flow across the transmission lines by maximizing the margins between the power flow used and the *thermal limits* (maximum capacity) of the lines. Let $F_e(t)$ denote the power flow on transmission line $e \in E$ at time step t (in Amperes), and L_e represent its thermal limit. At time step t , the margin $M_e(t)$ is computed as

$$M_e(t) = \max \left\{ 0, 1 - \frac{F_e(t)}{L_e} \right\}, \quad \forall e \in E,$$

and the reward as

$$R_{Margins}(t) = \sum_{e \in E} (1 - (1 - M_e(t))^2). \quad (10)$$

This reward function encourages keeping the power flows on each line below their thermal limits by rewarding larger safety margins.

Another frequently used reward function is R_ρ , a simple linear function based on the maximum line load $\rho_{\max}(t) := \max\{\rho_e(t) : e \in E\}$, defined as:

$$R_\rho(t) = \begin{cases} 2 - \rho_{\max}(t), & \text{if } \rho_{\max}(t) < 0.95, \\ 2 - 2\rho_{\max}(t), & \text{otherwise.} \end{cases}$$

This reward function was first introduced in [Binbinchen-2020](#) and later applied in [PowRL-2022](#), [Curriculum-2023](#), and [LJNAgent-2024](#). As illustrated in [Table 7](#), it is often combined with a high negative penalty for game over events or for performing illegal actions¹⁸ and a strong positive reward when the agent successfully completes the episode without failure.

The solution [SMAAC-2021](#) uses a reward function based on energy losses rather than transmission line loading, namely

$$R_{Loss}(t) = \frac{\sum_{i \in \text{loads}} p_i(t)}{\sum_{i \in \text{gens}} p_i(t)} - 0.9,$$

where p_i is the active power production or consumption of a load or generator i , respectively. Notably, this reward function departs from the other line-load-based reward functions by focusing on minimizing energy losses, indirectly encouraging lower line loads since overloaded lines tend to increase energy losses. Despite being fundamentally different, this reward function has shown good performance in [\[44\]](#).

Lastly, we discuss the reward designed by [\[61\]](#) and used in [AlphaZero-2022](#). Their reward is based on the cumulative sum over all overloaded lines. The authors introduce a function u as in [Eq. \(11\)](#) and utilize it to calculate the reward as:

$$R_{Alpha}(t) = e^{-u(t) - 0.5 \cdot n_{\text{offline}}},$$

where

$$u(t) = \begin{cases} \max\{\rho_{\max}(t) - 0.5, 0\}, & \text{if } \rho_{\max} \leq 1.0, \\ \sum_{l \in L: \rho_l > 1} \rho_l, & \text{otherwise.} \end{cases} \quad (11)$$

This reward function considers the number of disconnected lines, n_{offline} , to decrease the maximum achievable reward at that time step.

The solutions [BruteForce-2022](#) and [HRI-EU-2022](#) rely on greedy simulations to determine the next action. The selection of greedy actions can be guided by the reward $r(t+1)$, but these approaches base their decision on the maximum line load $\rho_{\max}(t+1)$. Specifically, the greedy agent selects the action that minimizes $\rho_{\max}(t+1)$. [BruteForce-2022](#) and [HRI-EU-2022](#) are therefore excluded from [Table 7](#).

Since [IL-2024](#), [GNNIL-2025](#) and [SoftIL-2025](#) do not use an RL agent and thus do not require a reward function to guide their behavior toward a stable state, these approaches are also excluded from [Table 7](#).

Lastly, we note that not all authors documented the reward function used. In this case, the reward function was derived from the code if it was publicly available. Solutions for which we were unable to determine the reward function are excluded from [Table 7](#).

Solution Acronym	Reward function
A3C-2019	$r(t) = \begin{cases} -100, & \text{if game over,} \\ \sum_{l \in L} R_l(t), & \text{otherwise,} \end{cases} \text{ where } R_l(t) = \begin{cases} 0.95 - \frac{F_l(t)}{L_l}, & \text{if } F_l(t) \leq 0.95 \cdot L_l, \\ \alpha \cdot (0.95 - \frac{F_l(t)}{L_l}), & \text{otherwise.} \end{cases}$
DDQN-2019	$r(t) = \begin{cases} -1, & \text{if game over,} \\ \frac{1}{n} \cdot R_{Margins}(t), & \text{otherwise.} \end{cases}$

Table continues on the next page...

¹⁸Illegal actions are actions that are currently unavailable, for example, due to maintenance of a substation or power line.

Solution Acronym	Reward function
A3C-2020	$r(t) = \begin{cases} -200, & \text{if game over,} \\ R_{Margins}(t) + R_{violated}(t), & \text{otherwise,} \end{cases} \quad \text{where} \quad R_{violated}(t) = -\sum_{l \in L} \beta_l \cdot ReLu(\frac{F_l(t)}{L_l} - 1) \quad \text{with} \quad \beta_l = 50, \quad \forall l \in L.$
SMAAC-2021	$r(t) = \begin{cases} -1, & \text{if action results in game over,} \\ -0.5, & \text{if action is illegal,} \\ R_{Loss}(t), & \text{otherwise.} \end{cases}$
D3QN-2020	$r(t) = w_1 \cdot R_{Sandbox}(t) + w_2 \cdot R_{CloseToOverflow}(t) + w_3 \cdot R_{Distance}(t) + w_4 \cdot R_{LineCapa}(t),$ where $R_{Sandbox}(t)$ based on costs for redispatch and loss, $R_{CloseToOverflow}(t)$ based on the overflow of lines; less overflow gives higher reward, $R_{Distance}(t)$ based on the distance from the reference topology; smaller distance gives higher reward, $R_{LineCapa}(t)$ based on the lines capacity usage; more available capacity gives a higher reward, - and $w_1 = 30$, $w_2 = 200$, $w_3 = 20$, and $w_4 = 3.0$.
Binbinchen-2020	$r(t) = \begin{cases} -300, & \text{if action results in game over,} \\ -10, & \text{if action is illegal,} \\ 500, & \text{if finished complete episode,} \\ R_\rho(t), & \text{otherwise.} \end{cases}$
SAS-2021	No reward function used for training. They estimate the value function (which should be the expected discounted future reward of a state) with the current risks of the lines defined as $R_{isk}(t) = \max \left\{ \frac{F_l(t)}{L_l} \right\}, \forall l \in L$.
CEM-2021	$r(t) = \begin{cases} 0, & \text{if action results in game over,} \\ R_{Margins}(t), & \text{otherwise.} \end{cases}$
D3QN-2022	$r(t) = \begin{cases} 1, & \text{if survived time step } t, \\ 0, & \text{if game over.} \end{cases}$
PowRL-2022	$r(t) = \begin{cases} -300, & \text{if action results in game over,} \\ 500, & \text{if finished complete episode,} \\ R_\rho(t), & \text{otherwise.} \end{cases}$
AlphaZero-2022	$r(t) = R_{Alpha}(t).$
Curriculum-2023	$r(t) = \begin{cases} -10, & \text{if action illegal or results in game over,} \\ R_\rho(t), & \text{otherwise.} \end{cases}$
HRL-2023	$r(t) = \begin{cases} \frac{1}{n} \cdot R_{Margins}(t), & \text{if survived time step } t, \\ -0.5, & \text{if game over.} \end{cases}$
MARL-2023	$r(t) = \begin{cases} -1, & \text{if action results in game over,} \\ -0.5, & \text{if action is illegal,} \\ R_{Loss}, & \text{otherwise.} \end{cases}$
LJNAgent-2024	$r(t) = \begin{cases} -10, & \text{if action illegal or results in game over,} \\ R_\rho(t) + R_{action}(t), & \text{otherwise,} \end{cases}$ where $R_{action}(t) = \begin{cases} 4.0, & \text{if do-nothing action is used,} \\ 0.0, & \text{otherwise.} \end{cases}$ The greedy search uses a simple linear reward based on the maximum ρ value: $R_{Greedy} = 2 - \rho_{\max}$.
BDQN-2024	$r(t) = \begin{cases} -20, & \text{if action results in game over,} \\ 0.7R_e(t) + 0.3R_c(t), & \text{otherwise,} \end{cases} \quad \text{where} \quad R_e(t) = \frac{\sum_{i \in gens} c_i \cdot (P_i^{\max} - P_i(t))}{\sum_{i \in gens} c_i \cdot P_i^{\max}},$ with c_i the costs of generator $i \in gens$ and $P_i^{\max}$ and $P_i(t)$ the maximum and given active power of generator $i \in gens$, respectively, and $R_c(t) = 1 - \frac{1}{ O } \sum_{i \in O} \frac{I_i(t)}{I_i^{\max}}$, with $O \subseteq E$ the set of connected transmission lines and $I_i^{\max}$ and $I_i(t)$ the maximum and the given current of line $i \in O$, respectively.

Table continues on the next page...

Solution Acronym	Reward function
CCMA-2025	$r(t) = \begin{cases} \frac{1}{n} \cdot R_{Margins}(t), & \text{if survived time step } t, \\ -0.5, & \text{if game over.} \end{cases}$

Table 7: Overview of the reward functions, $r(t)$, used by the solutions discussed.

3.1.6 Imitation learning

As shown in Table 1, eight out of twenty-two solutions employ *Imitation Learning* (IL), see [42, 91, 92], a machine learning technique in which a task is learned by observing an expert. In IL, the agent tries to mimic the behavior of an expert by predicting the actions $a(t) \in A$ the expert would choose in state $s(t) \in S$. This differs from RL, where the agent learns through exploration, receiving rewards to guide it towards a policy that maximizes long-term cumulative reward. Although RL is effective in many scenarios, learning an optimal policy can be difficult, for example, when the rewards are sparse (e.g., given only at the end of an episode), or the reward function is undefined, which is the case in power grid maintenance. As discussed earlier, different reward functions have been designed to provide the agent with the right feedback. However, manually designing a reward function that satisfies the desired behavior can be extremely complicated. IL could be a good solution to address this issue. Moreover, even when a proper reward function is available, IL can speed up the learning process by reducing the number of samples required for RL.

IL has two main classes: *behavioral cloning* (BC), described in [93, 94], and *inverse reinforcement learning* (IRL), introduced in [95]. In BC, data trajectories (τ^*) of the expert agent are collected, consisting of state-action pairs $(s_0, a_0^*), (s_1, a_1^*), \dots$, and supervised learning is used to learn the policy π_θ , parameterized by θ using a neural network, that minimizes the loss $\mathcal{L}(a^*, \pi_\theta(s))$. In contrast, IRL tries to determine the reward function $r_\theta(s, a)$ that the expert agent is optimizing, based on observed demonstrations, rather than directly learning the policy.

The IL-based solutions proposed for L2RPNall involve BC. In [Binbinchen-2020], a greedy expert (referred to as Tutor) selects the action that minimizes the maximum line load $\rho_{\max}(t+1)$, while the agent (Junior) attempts to replicate this policy. In [Curriculum-2023], the experts' strategy is extended with an $N-1$ secure action selection, as described in Section 2.6. This extended experts' strategy has also been used in [LJNAgent-2024, Artelys-2024, HUGO-2024, IL-2024, GNNIL-2025] and [SoftIL-2025]. Five of these papers use IL as a warm start to initialize RL-based agents, similar to the approach used in the AlphaGo algorithm [96]. In contrast, [IL-2024, GNNIL-2025] and [SoftIL-2025] explore IL as the primary method without subsequently deploying an RL algorithm. Nevertheless, these experiments show promising results, being able to improve on the expert Greedy baseline while reducing the computation time needed.

As pointed out by [48], BC can result in a *distribution shift*: small deviations in the agents' actions from the experts can lead to unseen scenarios for which the behavior is undefined. Consequently, the agent makes more mistakes, which could lead to catastrophic failures. To mitigate this issue, an improved version of BC can be used, where training alternates between the agents' policy and the expert [97, 98].

3.1.7 Prioritized replay

In its simplest form, RL algorithms discard experience after one update. *Experience replay*, a technique designed for *off-policy* algorithms (see Section 1.3.2), addresses two key issues: the risk of strongly correlated updates, and the tendency to forget useful but rare experiences immediately. Transitions, (s_t, a_t, r_t, s_{t+1}) , experienced by the agent, are stored in a replay buffer and sampled uniformly at random to be revisited. In *prioritized experience replay* (PER) [43], the idea is to replay transitions that are important more frequently and, in this way, make more efficient and effective use of the stored experiences. The priority is typically determined by the temporal difference (TD) error, which measures the difference between the predicted Q-value of a state-action pair and the target Q-value based on the current policy. Essentially, the TD error indicates how much the new experience impacts the learning process. As mentioned in [43], this prioritization might introduce a bias, which is corrected through importance sampling.

Several solutions in Table 1 that employ DQN-based algorithms, [DDQN-2019, D3QN-2020] and [D3QN-2022, [39, 56, 71]], make use of PER. In [71], the authors demonstrate the performance improvement when using PER for D3QN applied to the L2RPN problem.

The authors of [PowRL-2022, [73]] also applied PER in their solution, but combined with PPO. This is noteworthy since PPO is an on-policy algorithm, and as previously mentioned, experience replay is primarily designed for off-policy methods. The use of PER in this context shows potential benefits outside its conventional use.

3.1.8 Exploration strategies

A DRL agent needs exploration to learn more about the environment and discover potentially good actions. A common exploration strategy used for off-policy algorithms (see [Section 1.3.2](#)) is *Epsilon-Greedy*. Here, the agent selects a random action with probability ϵ and with probability $1 - \epsilon$ the action best evaluated, $a^* = \arg \max_{a \in A} \{Q(s_t, a)\}$, according to the current estimated Q -values.

Due to the large action space it can be difficult for the agent to converge to a good policy. [DDQN-2019](#) and [D3QN-2020](#) therefore, introduce a *guided exploration* method. In this approach, the agent selects the top N estimated Q -values for the current state $Q(s_t, a)$ and simulates the performance of each action to pick the action with the highest reward. This leads to a more stable training process and significantly increases training efficiency. It should be noted that this approach uses a greedy action selection based on simulations and, therefore, might overlook actions that do not directly yield a high reward but do result in a new state s_{t+1} for which the cumulative discounted future reward is profitable.

3.1.9 Curriculum learning or prioritized training

It can be noted that PNC is a very complex task. To understand and learn the concepts of a complex task, humans and animals often start with smaller, easier sub-problems and gradually move to the more complex ones. *Curriculum learning*, introduced by [\[41\]](#), essentially copies this idea as a training strategy in the field of machine learning.

[A3C-2019](#) ([\[38\]](#)) apply this technique by defining three difficulty levels and training the agent from easy to hard. The difficulty levels are defined by adjusting the Grid2Op²¹ configuration parameters: *SOFT_OVERFLOW_THRESHOLD* (*SOT*), *NB_TIMESTEP_OVERFLOW_ALLOWED* (*TOA*²²) and *HARD_OVERFLOW_THRESHOLD* (*HOT*). When a transmission line $e \in E$ has a soft overflow, i.e., $\rho_e(t) > SOT$, for more than *TOA* consecutive time steps, the line will be disconnected. In case $\rho_e(t) > HOT$, the line e immediately disconnects. In level-1 (the easiest) all parameters, *SOT*, *TOA*, and *HOT* are set in such a way that lines would never disconnect, level-3 applies the default parameters of the environment, which are *SOT* = 1.0, *TOA* = 3 time steps and *HOT* = 1.5 and level-2 is set in such a way that the strictness of the environment gradually increases. The authors show that adding this curriculum training strategy to the A3C algorithm applied to PNC enhances the training progress and improves the results of the final agent when tested on the evaluation episodes.

The authors of [BDQN-2024](#) ([\[77\]](#)) introduce K-means based predefined curriculum learning. The initial states that can be used as a start during the training process are grouped into clusters using K-means. One of the clusters is then sampled based on a calculated sampling weight, which is defined by the cluster's difficulty and visiting frequency. With this sampling weight, the authors try to balance learning in a curriculum strategy, from easy to hard, while ensuring state variety.

Another solution that orders the data given to the DRL agent is [SMAAC-2021](#) ([\[44\]](#)). In this approach, the training algorithm gives priority to more difficult episodes. Contrary to what curriculum learning suggests. The difficulty of an episode is determined by how well the do-nothing agent performed and how well the DRL agent has done so far. This has the advantage that the agent enters situations where it could learn more frequently, since difficult scenarios require more interference from the agent, whereas too easy scenarios might barely exceed the thermal limit of the transmission lines.

3.1.10 State and action space factorization

One of the key challenges of PNC is the large state and action space which is known as the *curse of dimensionality*. To deal with this problem, several authors propose to decompose the PNC problem into multiple subtasks, referred to as *factorization* (or decomposition) of the (state and) action space.

Four of the evaluated papers, [\[44\]](#), [\[74\]](#), [\[47\]](#), [\[82\]](#), suggest a hierarchical setting where one agent is responsible for determining the area, in this case, the substation where an action is needed, and another agent decides the specific action, i.e., the substation configuration, executed. In [SMAAC-2021](#) ([\[44\]](#)) and [MARL-2023](#) ([\[47\]](#)), the agent that determines the actionable substation is a rule-based agent, and the agent that defines the specific substation configuration is one or multiple RL-based agent(s). On the other hand, the solution of [HRL-2023](#) ([\[74\]](#)) proposes an RL-based agent to select the substation together with a Greedy or RL-based agent for the substation configurations. The solution [CCMA-2025](#) ([\[82\]](#)) reverses the order of the agents. The regional agents first propose an action, after which one central coordinating agent decides in which area the proposed action will be executed.

²¹The research of [\[38\]](#) has been applied on the predecessor of Grid2Op, PyPowNet. We discuss the equivalent parameters here.

²²In [\[38\]](#) referred to as consecutive overload limit, COL.

In [BDQN-2024](#) ([\[77\]](#)), the authors integrate the option model inspired by [\[79\]](#). In this approach, the problem is split into two subtasks based on the action frequencies. One of the neural networks of the agent is specialized on high-frequency actions and the other on low-frequency actions.

A limitation of the previously mentioned approaches is that an agent i with a certain subtask, i.e., that only controls a subset $\mathcal{A}_i \subset \mathcal{A}$, still observes the entire grid, and therefore their state space is not reduced, as pointed out by [\[81\]](#). Creating a good segmentation of the state and action for PNC is a difficult task, as the highly interconnected structure of the grid and the nature of power systems can cause local actions to have an effect on a very distant portion of the grid, as shown by [\[99, 100\]](#) with *influence graphs*.

In [\[72\]](#), the authors use an influence graph based on the *Line Outage Distribution Factors*, a metric that measures the effect of disconnecting a transmission line i to the power flow of transmission line j , to define a segmentation of the power grid. The resulting segmentation is then used to define the areas of the topological agents, which show promising performance when compared with one central agent, both MILP-based.

[\[81\]](#) propose a state and action factorization method that can also be used outside the PNC context, without the use of influence graphs or domain-specific knowledge. The factorization that was found using this method has not yet been applied to an RL agent for PNC at the time of writing, so this is still open for future research.

3.2 Rule Based Techniques

This section discusses different rule-based techniques used to enhance the agents’ performance. The rule-based techniques considered are the activation threshold, reconnection of lines, and reverting to the reference topology. These expert rules are based on what is known to work well in the power grid system. As seen in [Table 1](#), all solutions use expert rules to enhance agents’ performance.

3.2.1 Activation threshold

When the power grid is operating safely and there are no contingencies, it is best not to interfere (too often) as this will result in more instability. To accommodate for this, [DDQN-2019](#) ([\[39\]](#)) introduced a warning flag which we refer to as *activation threshold* (AT), denoted ρ_{act} . The agent is activated only when one of the lines exceeds the AT, i.e., $\rho_{\text{max}}(t) > \rho_{\text{act}}$. As shown in [\[39\]](#), this significantly improves the training process, as the agent does not have to learn what to do during a period when the system is in a safe state. [SMAAC-2021](#) ([\[44\]](#)) note that the MDP now becomes a semi-MDP setting for the RL-agent [\[79\]](#).

From [Table 1](#), it can be seen that the values used for the AT range between 0.6 – 1.0 and there is no golden rule. On average, the AT used for the final agent is $\rho_{\text{act}} \approx 0.93$ and from [Table 8](#) it can be seen that on average a higher AT is used in the larger cases. Some agents use a lower value for ρ_{act} during the training process. This can be argued by the fact that the agent otherwise might take a long time to train due to its infrequent activation. By having a lower AT, data is collected more frequently, and more training updates can take place. However, having a low AT harms the training process, as the agent needs to learn to do nothing when the network is in a safe state.

Case	Mean AT ρ_{act}	Median AT ρ_{act}
All cases	0.93	0.95
Case 14	0.90	0.95
Case 36	0.94	0.95
Case 118	0.96	0.98

Table 8: Overview of mean and median activation thresholds ρ_{act} used for the RL agents.

The AT is not always compared to the current $\rho_{\text{max}}(t)$, instead some approaches estimate $\rho_{\text{max}}(t+1)$ using a simulation of a DN-action and activate the agent when $\rho_{\text{max}}(t+1) > \rho_{\text{act}}$.

²¹The documentation in the slides and the code do not agree. This equation shows how it is implemented. According to their documentation they cut it off at a minimum of -10 to avoid “fear”.

²²See https://github.com/lujasone/NeurIPS_2020_L2RPN_Comp_An_Approach/blob/fdb01cc42253e948562453d2fb3249fe1ded37e0/Adaptability_Track/MyRewards.py for more information on the exact reward functions.

3.2.2 Line reconnections and disconnections

As described in [Section 1.1](#), when a line is overloaded for too long, it will automatically disconnect and cannot be reconnected for the next 10 time steps, defined in Grid2Op by `NB_Timestep_Reconnection`. Other reasons a line could be disconnected are due to planned maintenance, introduced in the WCCI 2020 challenge, see [Section 2.2](#), opponent attacks, which are part of the NeurIPS Robustness Track in 2021, see [Section 2.3](#), and later challenges, or simply due to a line switching action of the agent. After such a disconnection the line cannot be reconnected until the cooldown time of the line has passed, defined in Grid2Op by `NB_Timestep_CoolDown_Line`.

When a line is disconnected, it makes sense to reconnect it as soon as possible, as the network is usually more stable and robust when most lines are connected. Using this assumption, it makes sense to implement this straightforward rule to enhance the agent instead of letting the RL algorithm of the agent learn when to reconnect lines. From [Table 1](#) it can be seen that, starting from the WCCI 2020 competition, most solutions use this enhancement.

In [PowRL-2022](#) ([\[73\]](#)), an additional rule for line switching is introduced: When a line is overloaded for a consecutive number of time steps, the line will be manually disconnected to avoid permanent damage.

3.2.3 Revert to reference topology

This expert rule is based on the assumption that the grid is most stable in its *reference topology*, i.e., the topology where all elements are connected to the same busbar in each substation [\[68\]](#). Based on this assumption, some solutions add a heuristic that proposes actions to revert to the reference topology whenever the grid is in a safe state. To define a safe state, a *revert threshold* (RT), denoted ρ_{rev} , is usually introduced. When $\rho_{max}(t) < \rho_{rev}$ the network is assumed to be safe. The actions to revert to the reference topology are often first simulated and then greedily selected based on $\rho_{max}(t+1)$. The code of [AlphaZero-2022](#) reveals that the authors also introduced a minimum period between each reversion, which prevents the grid from going back and forth between topologies and, therefore, provides a more stable grid. The approach [CCMA-2025](#) uses a different additional condition for topology reversion. The revert to reference topology rule is only implemented between 03:00AM and 06:00AM, as the grid tends to be more stable at night.

The solutions [IL-2024](#) and [GNNIL-2025](#) do not use an RT, but instead, the scenarios are split into individual days. The authors highlight two benefits of this approach: (1) shortening the scenarios increases the amount of usable data for learning, as fewer data can be used from scenarios where the agent fails quickly, and (2) this mimics topology reversal, which is assumed to be beneficial for operating power grids, as stated earlier.

The last column of [Table 1](#) shows which solutions revert to the reference topology in their implementation. The mean value used as RT is 0.89, see [Table 9](#).

Revert threshold (ρ_{rev})	Solutions
0.95	SAS-2021 ²³ , D3QN-2020 ²³ , PowRL-2022
0.9	AlphaZero-2022 ²³ , LJNAgent-2024 , CCMA-2025
0.85	BruteForce-2022 ²³
0.8	Curriculum-2023 , SoftIL-2025

Table 9: Overview of values chosen as revert threshold ρ_{rev} in different solutions.

4 Benchmarks, performance metrics and baselines

This chapter discusses the evaluation framework used for assessing reinforcement learning (RL) agents in the context of power network control (PNC), by covering three main components: benchmarks ([Section 4.1](#)), performance metrics ([Section 4.2](#)), and lastly baseline agents ([Section 4.3](#)).

4.1 Benchmarks

Grid2Op provides a variety of benchmarks that can be used for the training and evaluation of an (RL) agent for PNC. See [Table 10](#) for an overview of all environments that we refer to in this paper. Starting from a toy size, case 5, with only five substations, the difficulty of the benchmark environments increases to a real-size grid of 118 substations, case

²³Based on code.

118. Furthermore, additional stochasticity, more sustainable energy sources, and an adversarial agent are added to some environments to increase the difficulty. Each of the environment datasets consists of episodic scenarios with 5-minute intervals.

The different environment datasets that can be used are more elaborately discussed in [Section 2](#), and more information can be found in [\[84\]](#).

Case	Grid2Op name	Maintenance	Opponent	Redispatch	Storage units
Case 5 example	<i>rte_case5_example</i>				
Case 14 realistic	<i>rte_case14_realistic</i>			✓	
Case 14 sandbox	<i>l2rpn_case14_sandbox</i>			✓	
Case 36 WCCI 2020	<i>l2rpn_wcci_2020</i>	✓		✓	
Case 36 NeurIPS 2020	<i>l2rpn_neurips_2020_track1</i>	✓	✓	✓	
Case 118 NeurIPS 2020	<i>l2rpn_neurips_2020_track2</i>	✓		✓	
Case 36 ICAPS 2021	<i>l2rpn_icaps_2021</i>	✓	✓	✓	
Case 118 WCCI 2022	<i>l2rpn_wcci_2022</i>	✓	✓	✓	✓
Case 118 IDF 2023	<i>l2rpn_idf_2023</i>	✓	✓	✓	✓

Table 10: Overview of the available benchmark environments in Grid2Op, [\[84\]](#).

4.2 Performance metrics

Training curve A good measure of the performance of an agent is the *episode return*, i.e., the accumulated reward. To measure the performance of a novel RL algorithm, researchers often compare *training curves* of the algorithm with other state-of-the-art RL algorithms. The training curve is usually a plot with the mean episode return on the y-axis and the number of time steps played in the environment on the x-axis. The training curve provides a good visualization of how agent performance improves as the RL algorithm has more interactions with the environment.

These training curves are frequently used in the research conducted on RL for PNC. However, it is difficult to compare the results of one paper with the results of the other since not everyone uses the same reward function. Therefore, it is necessary to regenerate the results of other agents for a fair comparison. To use a more general metric, one could use the mean episode *duration* (or mean survival time) instead of the return in the training curve, as in [\[61\]](#).

Another aspect to consider for the training curve is that an RL-agent that uses an AT does not interact with the environment at each time step, see [Section 3.2](#). Therefore, it is more sensible to use the number of *environment interactions* of the RL-agent on the x-axis instead of the number of time steps played in the environment as done in [\[44, 74, 47\]](#). Alternatively, the number of agent updates can be put on the x-axis as in [\[53\]](#), but this quantity critically depends on the batch size chosen for the updates.

Metrics for the final agent To evaluate the performance of the final (trained) agent for the L2RPN challenge, [\[34\]](#) introduced a *score function*. The resulting scores range between $[-100, 100]$ where -100 corresponds to an immediate blackout, 0 corresponds to the performance of a do-nothing baseline agent, 80 corresponds to safely finishing the scenario, and 100 corresponds to an oracle agent that also optimizes all energy losses. The operational cost, the costs of energy losses, and the costs of a blackout are used to define the overall L2RPN score.

Another frequently used metric to assess the final performance of the agent is the average number of time steps survived, as in [\[68, 61, 74\]](#). In this case, it can be good to also include the redispatch and curtailment costs, as in [\[61\]](#), since this metric does not account for the costs of agents' actions.

Some papers also report the mean episode reward, but since the reward function is different per proposed solution, this metric is less informative. Other examples of statistics that, for example, are included in [\[1, 44, 70, 68, 74\]](#) are the number of (un)solved scenarios, the topology depth, the (number of) unique topologies or actions, and the (number of) substations changed. Where the *topology depth* defines the distance from the reference topology, i.e., how many substations are not in the reference topology.

Lastly, an interesting factor in the performance of the agent is the computation time needed. For example, a greedy agent performs quite well on the PNC problem, as shown by the solutions [BruteForce-2022](#) and [HRI-EU-2022](#). However, a

greedy agent that has to iterate over all possible options requires a lot of computation time. [61, 76] therefore include computation time as a performance measure in their evaluation.

4.3 Baselines

To aid a good start and help researchers evaluate their agents, the authors of [34] provide a dedicated package with baselines for the L2RPN competition²⁴. The package includes a *Do-Nothing Agent*, which is a frequently used baseline and is also directly accessible in the Grid2Op package. As the name suggests, the Do-Nothing Agent does not change anything in the network and gets a corresponding L2RPN score of 0, as described above. This is a simple baseline to confirm that your agent learned the intended behavior. The same package also includes an *Expert Agent* described by [12], which is used as a baseline by [68]. This agent ranks the topology actions with the information of an *overload distribution graph* based on LODF, and greedily selects the best action after simulation of the top actions according to this rank.

It is also good practice to benchmark the results of an agent with some current SOTA solutions, as done in [44, 73]. Most of the proposed solutions are publicly accessible via the baseline package. However, it might be time-consuming and difficult to replicate the results of previous solutions even when the code is publicly available due to the changes to Grid2Op or other packages required by the proposed solution. Another option is to compare the results with earlier, different, or simpler versions of the agent to show how the proposed enhancements of the agent affect the final results, as in [39, 71, 68, 74, 47, 61, 77].

A baseline agent that only uses continuous actions is *OptimCVXPY*. This agent is included in a couple of proposed solutions, as can be seen in Table 1. It is interesting to compare the saved costs of redispatch and curtailment actions and the computation time of a proposed agent with this agent. [33] introduces another baseline agent that uses only continuous actions, trained using PPO.

Lastly, the *Greedy Agent* is an interesting baseline agent to discuss. In this baseline, used in [61, 74, 48], the agent selects the best action based on the reward $r(t+1)$ or the maximum line load $\rho_{\max}(t+1)$ simply by simulating the network at the next time step $t+1$. Similar to the RL-based agents, the Greedy Agent is activated to perform an action only when the activation threshold is exceeded. As stated earlier, this agent gives good results, and an RL agent that can get close to or improve this baseline is usually quite good. However, the Greedy Agent does not consider the potential future rewards and, therefore, omits actions that do not directly give the best reward but can result in beneficial states after execution. Additionally, simulating all possible actions is computationally expensive. Therefore, a reduction of the action space is needed when applying this to cases 36 and 118, as done in [61].

5 Experimental setup

This chapter presents our experimental setup for our numerical studies. We evaluate a selection of modeling choices that have been discussed in Section 3, by analyzing their impact on the performance of the (RL-based) agent.

First, Section 5.1 describes the general experimental setup, including the benchmark environment, performance metrics, and baselines used. The subsequent sections explain the setup for each individual evaluated modeling choice:

- Action space in Section 5.2
- Observation space in Section 5.3
- Reward function in Section 5.4
- Curriculum training in Section 5.5
- Activation threshold in Section 5.6
- Line reconnections and disconnections in Section 5.6 and
- Revert to reference topology in Section 5.6

The results of all experiments are presented in Section 6. Each experiment adjusts one modeling choice, while the other parts remain as described for the baseline presented in the following. Finally, we create a ‘Rainbow’ agent along the same ideas and principles of [101], combining our best findings to see how much we can improve the baseline and if the combined configurations will help each other.

²⁴These baselines are available at <https://github.com/Grid2op/l2rpn-baselines> and documentation is available at [65]

5.1 General setup

Our experiments focus on a single PPO agent, the RL algorithm most frequently applied in the L2RPN challenge, see Table 1. Since PPO is an on-policy method, techniques designed for off-policy algorithms (e.g., prioritized replay and exploration strategies) are not included in the evaluation studies. The comparison of the techniques in our experiments can still be relevant when using other algorithms such as DQN or SAC. Furthermore, *imitation learning* and *factorization* are also interesting research areas. However, exploring these in depth would require a dedicated study and is outside the scope of this work.

All our experiments use the open-source library RLlib from [102]. The PPO algorithm in this library is slightly adjusted to facilitate Semi-MDP. The entire setup of our experiments is included in our RL4PNC package²⁵. Before training the PPO algorithm, all episodes are split into train (80%), test (10%), and validation scenarios (10%) to avoid overfitting. Validation scenarios are used to compare the final trained agents with the other agents (Do Nothing, Greedy, or other PPO agents).

Benchmarks All experiments are tested on the benchmark case 14 sandbox with and without an added opponent, where the opponent is configured as in [74]. This test case allows for extensive experimentation while maintaining manageable computational costs. Furthermore, introducing an opponent provides a more realistic assessment, as larger environments typically involve either an opponent or maintenance constraints.

We train the PPO agents on both environments. We will refer to the agent trained on the environment without an opponent as PPO and the agent trained on an environment with an opponent as PPO*.

(Performance) metrics To understand how techniques affect the RL-based agent’s training process, we plot a training curve with the mean survival time (the solid lines) and the corresponding standard deviation (shaded area) against the number of environment interactions, averaged over five different model seeds. The curves are smoothed by using a moving average of 1500 environment interactions.

The final trained agents and the Greedy and Do-Nothing agents are validated on the validation scenarios. The results are presented in tables in which we compare the following metrics.

- Percentage completed episodes,
- percentage time steps survived,
- percentage time steps overloaded,
- agent execution time: mean computation time needed when the agent is activated, i.e., when $\rho_{max} > \rho_{act}$,
- maximum topology depth,
- number of unique actions (substation configurations) used,
- number of unique lines in danger,
- number of unique substations changed,
- specific substations changed.

Most of these metrics have been examined in previous papers and are introduced in Section 4.2. Here we introduce two new metrics: (1) percentage of time steps overloaded, tracking how often the grid operates in danger, which helps us compare agents with similar survival percentages, and (2) number of unique lines in danger, tracking how many different lines have been overloaded.

Each PPO agent is trained with five different seeds, and all are evaluated on the validation scenarios. Metrics are recorded for each agent and the average is reported.

Baselines The baselines used for validation are (1) the Do-Nothing Agent, (2) a Greedy Agent w.r.t. $\rho_{max}(t+1)$, and (3) a single PPO agent as in HRL-2023 ([74]) which is configured as described below.

- The action space is $\mathcal{A}_{n-0}$, described in Section 3.1.2,
- the observation space is as described for HRL-2023 in Table 6. Without the maintenance attribute, since this is not relevant for case 14,
- the reward function is based on $R_{Margins}$, as described for HRL-2023 in Table 7.

²⁵<https://github.com/EricavanderSar/rl4pnc-survey>

- the AT is $\rho_{\text{act}} = 0.95$, see Table 1,
- lines are reconnected when disconnected,
- revert to reference topology is not applied,
- normalization is applied as in Eq. (8),
- the hyperparameters are defined as in Table 11,
- the neural network consists of three *fully connected layers* (FCNN) of $[256 \times 256 \times 256]$, with ReLu activation and,
- Train for 100.000 environment interactions.

This configuration is used as a baseline because [74] demonstrated good performance with a purely PPO-based agent, trained without simulation-based steering.²⁶

Hyperparameter	Value without opponent (PPO)	Value with opponent (PPO*)
Discount (γ)	0.99	0.99
Learning-rate	0.0001	0.0001
VF coeff. (c_1)	0.5	0.5
Entropy coeff. (c_2)	0.0	0.01
Clipping param. (ϵ)	0.3	0.3
GAE param. (λ)	0.95	0.95
SGD iterations	5	15
Minibatch size	256	256
Batch size	1024	1024

Table 11: Hyperparameters used for experiments training a PPO agent on case 14.

Where applicable, the baseline Greedy agent uses the same configurations as the PPO agent, allowing for a fair comparison. More specifically, the action space is defined as $\mathcal{A}_{n-0}$, the AT $\rho_{\text{act}} = 0.95$, and lines are reconnected whenever disconnected.

The chosen baselines allow us to assess how different modeling choices impact PPO performance relative to (1) a passive strategy (Do Nothing), (2) a heuristic-based method (Greedy), and (3) an established PPO approach from prior work.

Although we maintain consistent hyperparameters across experiments to control computational costs, we note that optimal values may vary depending on the modeling choices, such as action and observation space, which impact the output and input of the neural network used. Future work could explore adaptive tuning to further enhance performance.

We conducted additional experiments with PPO agents trained without an opponent with the hyperparameters described in the third column “Value with opponent (PPO*)” of Table 11. A summary of these results is presented in Appendix C to demonstrate the importance of hyperparameter tuning.

5.2 Action space experiments

As mentioned in Section 3.1.2, most approaches reduce the action space to manage complexity in larger networks. The baseline Greedy and PPO agents use the $\mathcal{A}_{n-0}$ action space, which retains approximately 75% of the original $\mathcal{A}_{\text{sym}}$ action space (Table 3). A more restrictive reduction, $\mathcal{A}_{n-1}$, results in an action space of 40% of $\mathcal{A}_{\text{sym}}$. However, in the larger cases, 36 and 118, the action spaces are often reduced to around 1%–2%, most often using a Greedy reduction method, as described in Section 3.1.2. Therefore, in the experiments, we will also consider the smallest action space for case 14, containing roughly 5% of $\mathcal{A}_{\text{sym}}$, proposed by D3QN-2022, which we denote by $\mathcal{A}_{d3qn}$.

To assess the number of actions required for effective network management, experiments are conducted with both the Greedy and PPO agent using four different action spaces:

²⁶It is worth mentioning that the experiments in [74] used case 14 realistic instead of case 14 sandbox. Case 14 realistic is currently outdated, and therefore the case 14 sandbox has been used in this paper. The results may differ due to this.

- $\mathcal{A}_{sym}$: 178 actions,
- $\mathcal{A}_{n=0}$: 133 actions,
- $\mathcal{A}_{n=1}$: 73 actions and
- $\mathcal{A}_{d3qn}$: 9 actions.

5.3 Observation space experiments

These experiments examine how different observation spaces impact the training and final performance of the RL agent. We compare the following observation spaces:

- **Baseline**: active power, line loads (ρ), time-step overflow, and topology configuration.
- **Complete**: time stamp, active power, reactive power, voltages, voltage angles (θ), line flows, line loads, time-step overflow, line status, and topology configuration.
- **D3QN-2022** by [71]: voltages of lines only (v_{or}, v_{ex}), line flows, line loads, and topology configuration. This is interesting because this was the smallest observation space in the solutions discussed.
- **Line loads**: Including only the line loads ρ , which is the common feature across all proposed solutions.
- **Danger**: The baseline observation space with an additional *danger* parameter as in [SMAAC-2021] [44].
- **History**: Extends the previous observation space with six historical time steps, as in [SMAAC-2021]. Since an FCNN is used, there is no need to structure the features in a feature matrix, which would be beneficial for a GNN architecture.

For an overview of the observation spaces considered in the experiments and their sizes, see Table 35 in Appendix B. The features *cooldown time* and *maintenance* are omitted as they are irrelevant for case 14.

Continuous features are normalized in all experiments, as detailed in Eq. (8). Since timestamp transformation is not explicitly discussed in prior solutions, we apply a cosine transformation, which better captures cyclic patterns than min-max normalization. The time of day (*time_of_day*) and the day of the year (*day_of_year*) are handled as separate cycles in view of the well-known daily and seasonal patterns in power usage; see Eqs. (12) and (13).

$$\text{time_of_day} = \cos(2\pi \cdot \text{minute_of_day} / 60 \cdot 24) \quad (12)$$

$$\text{day_of_year} = \cos(2\pi \cdot \text{day_of_year} / 365) \quad (13)$$

5.4 Reward function experiments

It is essential to choose a suitable reward function to help the RL algorithm learn the right behavior, as mentioned in Section 3.1.5. Table 7 shows how each proposed solution uses a (sometimes slightly) different reward function. In the experiments, the following reward functions are compared for the training process of a PPO agent:

- **Baseline**: The reward from [HRL-2023] using the $R_{Margins}$ function.
- **Base + Bonus/Penalty**: The baseline reward, with a bonus of 500 for episode completion and a penalty of -300 in case of game over.
- **Constant**: A constant reward, as used by [D3QN-2022].
- **Binbinchen**: The reward used in [Binbinchen-2020] using the R_p function.
- **SMAAC**: The reward used in [SMAAC-2021] using the R_{Loss} function.
- **AlphaZero**: The reward used in [AlphaZero-2022] using the R_{Alpha} function.

5.5 Curriculum training

Different from previous design choices, curriculum training is an optional enhancement of the RL algorithm. There are many possibilities for how to apply curriculum training to the PNC problem. We compare the baseline PPO agent with a curriculum-trained PPO agent using the method proposed by [38], described in Section 3.1.9. The configuration parameters for the three levels, *SOT*, *TOA*, *HOT*, discussed in Section 3.1.9, are shown in Table 12, along with an additional parameter *NOD*:NO_OVERFLOW_DISCONNECTION. With this setup, level 1 corresponds to a level

where there is no line limit enforcement, level 2 disconnects after 15 time steps of high overload of 2, and level 3 corresponds to the default environment behavior²⁷

Level	<i>NOD</i>	<i>SOT</i>	<i>TOA</i>	<i>HOT</i>
1	✓	99	99	999
2		2	15	99
3		1	2	2

Table 12: Setup curriculum levels.

We transition from level 1 to level 2 after 1/5 of the total training time, and from 2 to 3 (the final level) after 7/15 of the training time, following [38].

5.6 Rule-based experiments

For each rule-based decision described in Section 3.2, we test different application strategies: applying the rule during both training and the final validation of the agent, applying the rule only at one stage, or using a more strict version in one phase in which the rule is applied more frequently. This allows us to examine how the rule timing and intensity affect the agent’s performance.

Activation threshold As mentioned in Section 3.2.1, there is no golden rule to choose the activation threshold ρ_{act} . This motivates us to examine its impact by testing different values of ρ_{act} . We tested different values $\rho_{act} \in \{0.8, 0.9, 0.95, 0.99\}$, evaluating both the training and validation performance in eight combinations of the training and final agent thresholds.

Line switching: Reconnection and disconnection The PPO and Greedy baselines include a rule that reconnects the lines if their reconnection improves the current action, tested using simulation. We measure the frequency of these reconnections during training and compare performance with and without this rule. Additionally, we test a rule that disconnects overloaded lines after multiple consecutive time steps. This rule is motivated by the default reconnection delay of 10 steps²⁸, which can make early disconnection beneficial. The rule is implemented similarly to reconnection, meaning that only beneficial disconnections are executed. We test incorporating this rule in combination with the line reconnection rule. Testing the line disconnection by itself is not considered, as permanent disconnections would be detrimental.

Revert to reference topology The ‘revert to reference topology’ rule restores substations to their reference topology (i.e., all elements connected to the same bus) when the network is in a ‘safe state’. Each reversion action is simulated, after which the best one is executed if it improves on the current action. Safe states are defined by different revert thresholds (RTs), which we compare in training and validation experiments: $\rho_{rev} \in \{0.8, 0.9, 0.95\}$. If no RT is applied, the agent follows the baseline behavior.

6 Results, recommendations and guidelines

This chapter describes the results for the experiments described in Section 5. The results will be presented in the same order as described in the experimental setups in Section 5. First, we describe the results of the Greedy, Do-Nothing and PPO baselines in Section 6.1. The results of the numerical experiments on each of the modeling choices follow in the subsequent sections Sections 6.2 to 6.8. Lastly, Section 6.9 presents the setup and the results of the Rainbow agent, configured based on the results of the previous experiments.

6.1 Baselines

The Greedy agent consistently outperforms the other baselines in maintaining grid safety, as the results in Tables 13 and 14 demonstrate, where the best values for “Completed episodes”, “Steps survived” and “Steps overloaded” are highlighted in bold, following the convention used throughout the paper.

²⁷The exact setup slightly deviates from [38] as the Grid2Op environment changed over time.

²⁸As defined by the Grid2Op parameter: NB_Timestep_Reconnection.

As expected, the Greedy agent requires significantly more execution time than the RL-based agents and, of course, the Do-Nothing agent. The PPO baseline did not complete all episodes but survived 87% of time steps on average – significantly improving on the Do-Nothing agent, which confirms it learned meaningful behavior – while using fewer than half the actions required by the Greedy agent (see ‘Unique actions’ column). PPO* performed worse than PPO in the environment without an opponent but slightly better in the environment with an opponent. Fig. 5 provides a visual comparison of the validation results, further illustrating that environments with an opponent pose a significantly greater challenge.

Agent	Completed episodes	Steps survived	Steps loaded over-	Agent execution time [ms]	Maximum topology depth	Unique actions	Unique lines in danger	Unique subs changed	Substations changed
Do Nothing	0.0%	20.90%	0.185%	0.063	1	1	4	0	[]
Greedy	100.0%	100.00 %	0.014%	369.669	5	65	12	5	[1, 3, 4, 5, 8]
PPO	78.6%	87.31%	0.008%	1.9668	5	27.8	11.8	6	[1, 2, 3, 4, 5, 8, 12]
PPO*	35.4%	57.89%	0.041%	1.9658	4.6	80.6	17	6.8	[1, 2, 3, 4, 5, 8, 12]

Table 13: Validation results of baseline agents, described in Section 5.1, applied to the environment case 14 sandbox without an opponent.

Agent	Completed episodes	Steps survived	Steps loaded over-	Agent execution time [ms]	Maximum topology depth	Unique actions	Unique lines in danger	Unique subs changed	Substations changed
Do Nothing	0.0%	1.920%	4.137%	0.062	1	1	12	0	[]
Greedy	0.0%	9.030%	1.217%	365.313	5	88	14	5	[1, 3, 4, 5, 8]
PPO	0.0%	4.690%	0.638%	2.1618	4	12.8	12.2	5.4	[1, 2, 3, 4, 5, 8, 12]
PPO*	0.0%	5.740%	0.683%	2.2398	4.6	91	15	7	[1, 2, 3, 4, 5, 8, 12]

Table 14: Validation results of baseline agents, described in Section 5.1, applied to the environment case 14 sandbox with an opponent.

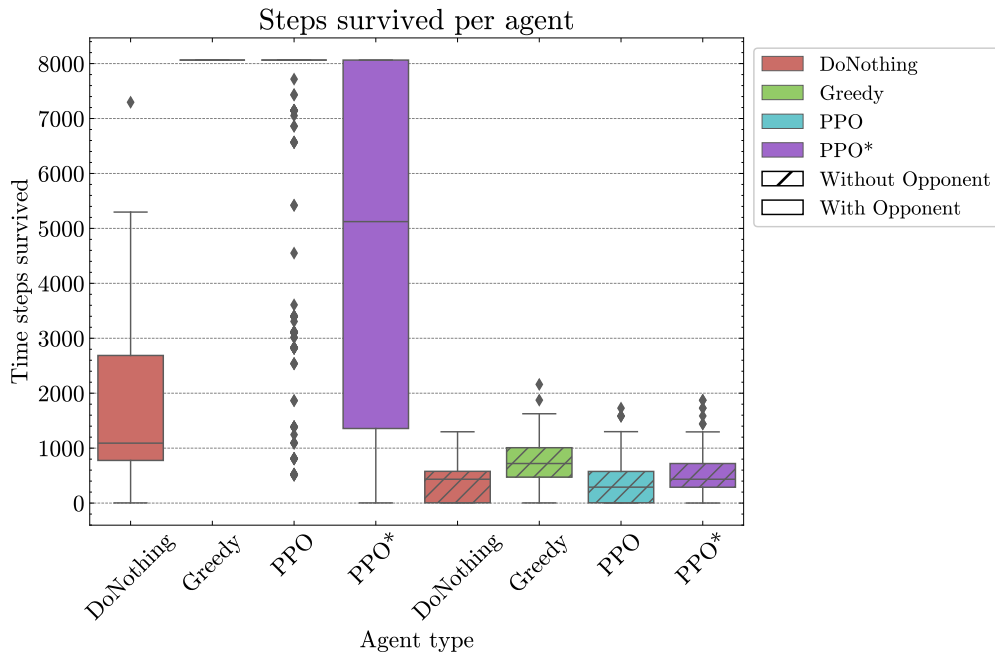


Figure 5: Boxplot of the time steps survived for the baseline agents applied to the validation episodes.

Fig. 6 illustrates the training progress of the PPO agent, showing that it converges at approximately 7000 time steps survived ($\sim 87\%$), consistent with the validation results in Table 13. The training curve suggests that PPO* does not learn effectively. Training was extended to 500.000 interactions (right figure in Fig. 6), but performance plateaued after 100.000 interactions. Therefore, our subsequent experiments stop after 100.000 interactions when training with an opponent.

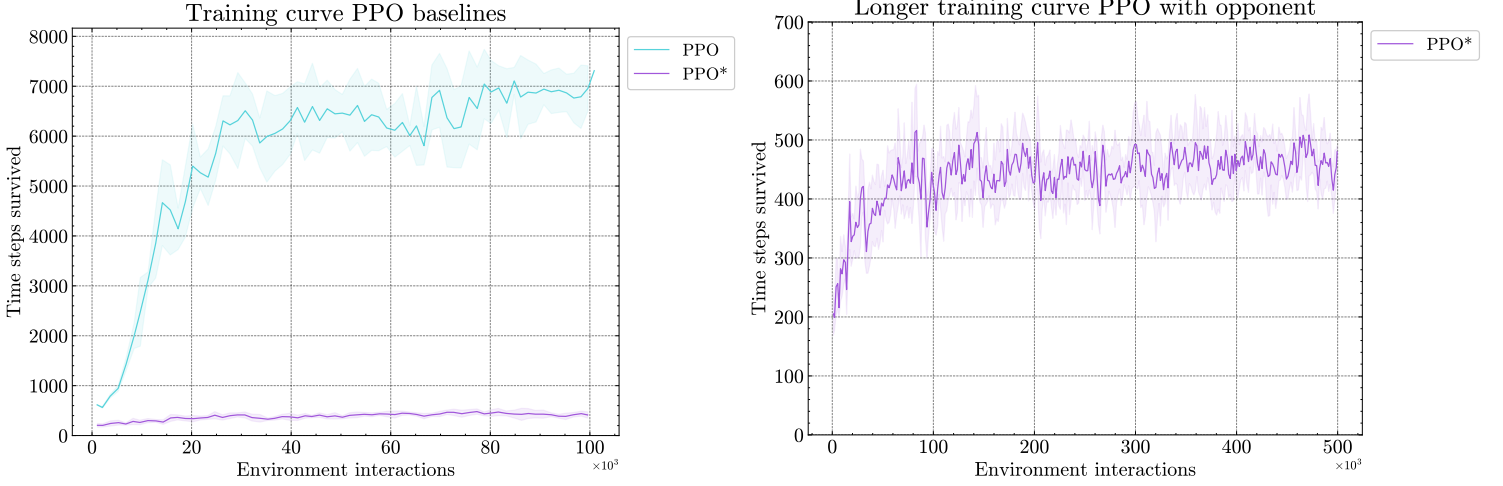


Figure 6: Training curves of the PPO agent trained without opponent (PPO) and the PPO agent trained with opponent (PPO*).

6.2 Action spaces

The choice in action can have a significant impact on the performance of the (PPO) agents. Fig. 7 illustrates substantial differences in the training progress when training the PPO agent with different action spaces. In an environment without an opponent, Fig. 7a, the agent trained with the action space $\mathcal{A}_{d3qn}$ yields the worst performance, whereas in an environment with an opponent, Fig. 7b, it achieves the best result. This suggests that expert-curated actions in $\mathcal{A}_{d3qn}$, which emphasize robustness, are particularly beneficial against line attacks. The results in Table 16 support this observation, showing that $\mathcal{A}_{d3qn}$ consistently outperforms other action spaces across all agents (Greedy, PPO, and PPO*).

Despite eliminating less robust actions, the training progress of the agent with the $\mathcal{A}_{n-1}$ action space does not exceed that of the agents trained with the $\mathcal{A}_{n-0}$ and $\mathcal{A}_{sym}$ action spaces.

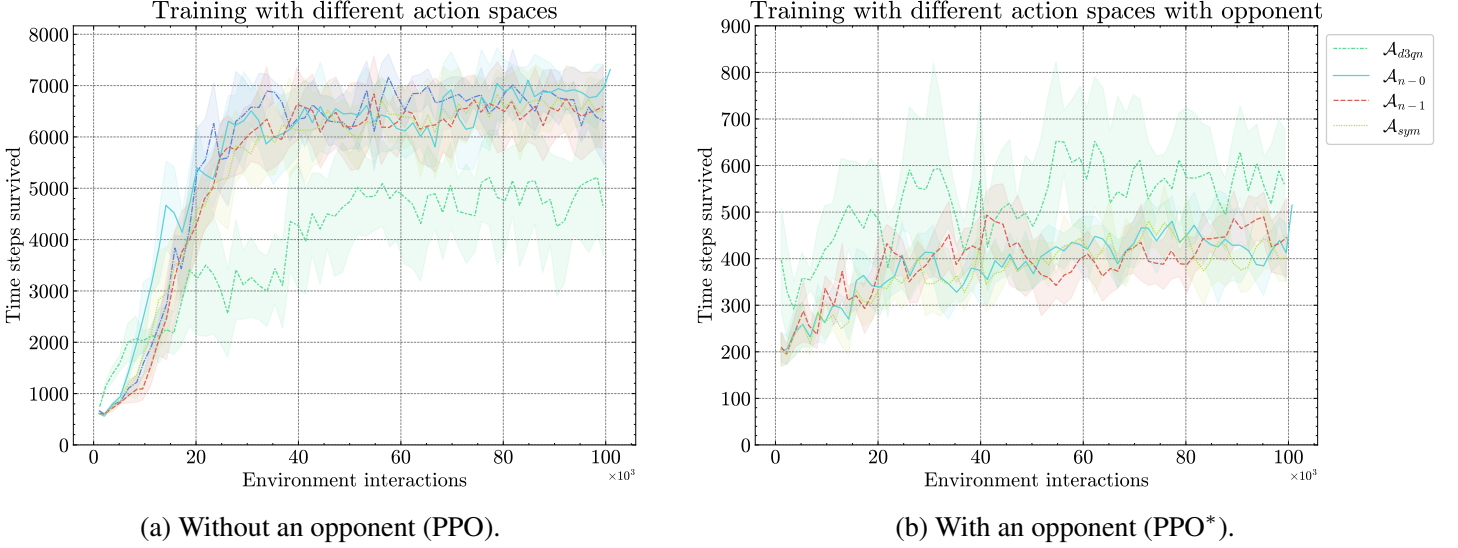


Figure 7: Training curves of PPO agents trained with different action spaces.

Examining the validation results of the environment without an opponent reported in [Table 15](#), we observe that both the Greedy and PPO agents exhibit slightly reduced performance when the baseline action space $\mathcal{A}_{n-0}$ is reduced to $\mathcal{A}_{n-1}$. However, this reduction comes with a substantial decrease in execution time for the Greedy agent. The agent with $\mathcal{A}_{d3qn}$, the smallest action space containing only 5% of the available actions, survives only half of the time steps. Both the Greedy agent and PPO agents trained with $\mathcal{A}_{n-0}$ and $\mathcal{A}_{n-1}$ act only in substations 1, 3, 4, 5, and 8, suggesting that a reduced action space incorporating these substations could be effective. Furthermore, the Greedy agent with the action space $\mathcal{A}_{sym}$ exhibits performance comparable to that of $\mathcal{A}_{n-0}$, while the agent with the action space $\mathcal{A}_{n-0}$ employs significantly fewer distinct configurations.

[Table 36](#) in [Appendix C](#) shows that training the PPO agent using different hyperparameters can give substantially different results. Specifically, the reduced action space $\mathcal{A}_{n-1}$ achieves an overall better score in [Table 36](#) compared to [Table 15](#). This fact highlights the importance of hyperparameter tuning when changing the number of actions an RL agent needs to learn.

Agent	action space	completed episodes	steps survived	steps loaded	over-	agent execution time [ms]	maximum topology depth	unique actions	unique lines in danger	unique changed	subs	substations changed
Greedy	$\mathcal{A}_{sym}$	100.0%	100.00%	0.017%		487.991	6	90	14	10		[0, 1, 2, 3, 4, 5, 8, 9, 11, 12]
	$\mathcal{A}_{n-0}$	100.0%	100.00%	0.014%		369.669	5	65	12	5		[1, 3, 4, 5, 8]
	$\mathcal{A}_{n-1}$	98.0%	99.74%	0.022%		210.361	5	52	16	5		[1, 3, 4, 5, 8]
	$\mathcal{A}_{d3qn}$	19.0%	52.05%	0.088%		31.585	3	8	4	3		[1, 3, 4]
PPO	$\mathcal{A}_{sym}$	74.6%	85.68%	0.011%		2.2462	5.4	43	12.4	9.4		[0, 1, 2, 3, 4, 5, 6, 8, 9, 10, 11, 12, 13]
	$\mathcal{A}_{n-0}$	78.6%	87.31%	0.008%		1.9668	5	27.8	11.8	6		[1, 2, 3, 4, 5, 8, 12]
	$\mathcal{A}_{n-1}$	53.6%	77.87%	0.030%		2.2202	4	35.2	15.8	4.8		[1, 3, 4, 5, 8]
	$\mathcal{A}_{d3qn}$	27.0%	56.23%	0.112%		4.8118	1	5.4	5.4	2.2		[1, 3, 4]
PPO*	$\mathcal{A}_{sym}$	41.6%	61.68%	0.048%		2.2738	4.8	91	15.8	12.2		[0, 1, 2, 3, 4, 5, 6, 8, 9, 10, 11, 12, 13]
	$\mathcal{A}_{n-0}$	35.4%	57.89%	0.041%		1.9658	4.6	80.6	17	6.8		[1, 2, 3, 4, 5, 8, 12]
	$\mathcal{A}_{n-1}$	16.2%	43.36%	0.114%		2.38	4.2	63.8	16	5		[1, 3, 4, 5, 8]
	$\mathcal{A}_{d3qn}$	9.0%	41.94%	0.117%		2.1576	2.8	9	8.2	3		[1, 3, 4]

Table 15: Validation results of different action spaces for the agents, applied to the environment case 14 sandbox without an opponent. The baseline results are highlighted in gray.

Agent	Action space	Completed episodes	Steps survived	Steps loaded	over-	Agent execution time [ms]	Maximum topology depth	Unique actions	Unique lines in danger	Unique changed	subs	Substations changed
Greedy	$\mathcal{A}_{sym}$	0.0%	9.550%	1.252%		479.783	7	115	15	12		[0, 1, 2, 3, 4, 5, 8, 9, 10, 11, 12, 13]
	$\mathcal{A}_{n-0}$	0.0%	9.030%	1.217%		365.313	5	88	14	5		[1, 3, 4, 5, 8]
	$\mathcal{A}_{n-1}$	0.0%	9.330%	1.691%		208.128	5	54	17	5		[1, 3, 4, 5, 8]
	$\mathcal{A}_{d3qn}$	0.0%	9.940%	0.947%		30.668	3	8	13	3		[1, 2, 3]
PPO	$\mathcal{A}_{sym}$	0.0%	4.738%	0.681%		2.3484	3.4	17.8	10.4	7.2		[0, 1, 2, 3, 4, 5, 8, 9, 10, 11, 12, 13]
	$\mathcal{A}_{n-0}$	0.0%	4.690%	0.638%		2.1618	4	12.8	12.2	5.4		[1, 2, 3, 4, 5, 8, 12]
	$\mathcal{A}_{n-1}$	0.0%	4.476%	1.840%		2.2628	3.4	33.8	15.4	4.8		[1, 3, 4, 5, 8]
	$\mathcal{A}_{d3qn}$	0.0%	5.376%	1.639%		2.8682	1	4.6	14.8	1.8		[1, 3, 4]
PPO*	$\mathcal{A}_{sym}$	0.0%	6.220%	0.692%		2.4682	3.8	99.2	13.6	13		[0, 1, 2, 3, 4, 5, 6, 8, 9, 10, 11, 12, 13]
	$\mathcal{A}_{n-0}$	0.0%	5.740%	0.683%		2.2398	4.6	91	15	7		[1, 2, 3, 4, 5, 8, 12]
	$\mathcal{A}_{n-1}$	0.0%	6.658%	0.912%		2.9604	3.6	65.4	15.2	5		[1, 3, 4, 5, 8]
	$\mathcal{A}_{d3qn}$	0.0%	6.904%	1.127%		2.57	2.6	8.8	14	3		[1, 3, 4]

Table 16: Validation results of different action spaces for the agents, applied to the environment case 14 sandbox *with* an opponent. The baseline results are highlighted in gray.

From [Table 16](#), we can conclude that more robust action spaces, such as $\mathcal{A}_{d3qn}$, help the agent perform on a network where lines can be disrupted and, similar to the results in [Table 14](#), the agent trained with an opponent performs better compared to the agent trained without an opponent.

6.3 Observation spaces

In contrast to the action space selection, the choice of observation space has a limited impact on performance. [Fig. 8a](#) shows that the smallest observation space, “Line loads”, achieves the highest survival time near the end of training,

suggesting that additional input features do not enhance learning progress. In the environment with an opponent (Fig. 8b), training curves for different observation spaces largely overlap, indicating that the choice of observation space has little impact in this setting. These results raise the question of whether the additional input features beyond line loads are irrelevant for decision-making or if adjustments, such as changes to the neural network architecture, are necessary to fully utilize the extra information. Similar conclusions can be obtained by observing the validation results in Tables 17 and 18.

In the experiments where we used alternative hyperparameters for the PPO agent the observation space of D3QN-2022 gave the best results, see Table 36 in Appendix C in contrast to the findings in Table 17, highlighting the impact of hyperparameter choices.

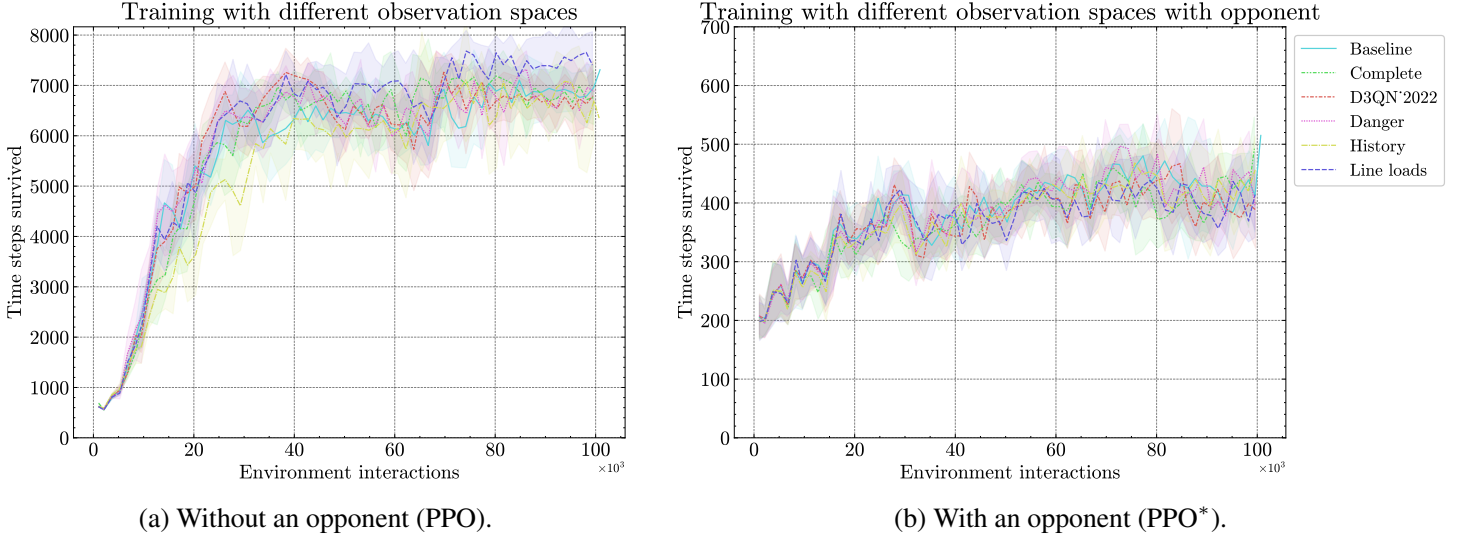


Figure 8: Training curves of PPO agents trained with different observation spaces.

Agent	Observation space	Completed episodes	Steps survived	Steps over-loaded	Agent execution time [ms]	Maximum topology depth	Unique actions	Unique lines in danger	Unique subs changed
PPO	Baseline	78.6%	87.31%	0.008%	1.9668	5	27.8	11.8	6
	Complete	78.0%	87.71%	0.010%	2.5374	4.4	32.8	11	6.2
	D3QN_2022	72.0%	83.52%	0.012%	2.2846	4.4	35.4	12.2	6.4
	Danger	77.2%	87.78%	0.013%	2.2118	4.6	33	12.8	6.8
	History	72.6%	84.42%	0.009%	3.0548	4.8	30.4	11.2	6
	Line loads	92.8%	96.01%	0.005%	1.9328	4.6	12.2	7.2	4.8
PPO*	Baseline	35.4%	57.89%	0.041%	1.9658	4.6	80.6	17	6.8
	Complete	25.2%	52.59%	0.067%	3.0234	5	95.2	17.8	7
	D3QN_2022	38.4%	61.58%	0.040%	2.9538	4.6	78.6	16.4	7
	Danger	43.4%	67.73%	0.037%	2.7828	5	74.4	15.6	7
	History	32.0%	54.57%	0.052%	4.2958	4.8	89.4	17.6	7
	Line loads	52.0%	68.02%	0.036%	2.5792	4.4	61	13.6	7

Table 17: Validation results of different observation spaces for the agents, applied to the environment case 14 sandbox without an opponent.

Agent	Observation space	Completed episodes	Steps survived	Steps over-loaded	Agent execution time [ms]	Maximum topology depth	Unique actions	Unique lines in danger	Unique subs changed
PPO	Baseline	0.0%	4.690%	0.638%	2.1618	4	12.8	12.2	5.4
	Complete	0.0%	4.718%	0.804%	3.9826	3.4	66.4	14.8	7
	D3QN_2022	0.0%	4.304%	0.770%	3.7278	3.6	64.8	16.8	6.8
	Danger	0.0%	4.758%	0.652%	5.712	3.4	36	12.4	6.6
	History	0.0%	4.400%	0.766%	4.7906	3.2	26.6	12.8	5.8
	Line loads	0.0%	4.184%	0.814%	3.4236	3.2	45.8	14.6	6.4
PPO*	Baseline	0.0%	5.740%	0.683%	2.2398	4.6	91	15	7
	Complete	0.0%	5.296%	0.674%	5.0958	3.4	97.8	15.6	7
	D3QN_2022	0.0%	5.468%	0.672%	3.9996	4	89.2	14.8	7
	Danger	0.0%	5.586%	0.718%	3.96	4.4	93.4	15.8	6.8
	History	0.0%	5.288%	0.744%	5.2616	3.4	91	16	7
	Line loads	0.0%	5.250%	0.762%	3.5642	3.6	77.2	15	7

Table 18: Validation results of different observation spaces for the agents, applied to the environment case 14 sandbox *with* an opponent.

6.4 Reward functions

Training curves for different reward functions show little difference in the number of time steps survived (see Fig. 9). The plots in Fig. 10 show the mean episode reward to highlight the difference between the experiments. Although episode rewards differ significantly, their impact on survival time is negligible. In the opponent-based training, the Binbinchen reward agent appeared to still be learning at the end of the 100,000 interactions. A similar trend is observed for the agent using the SMAAC reward when we zoom in on the mean episode reward for this agent. These results suggest that extended training could further improve the agents’ performance.

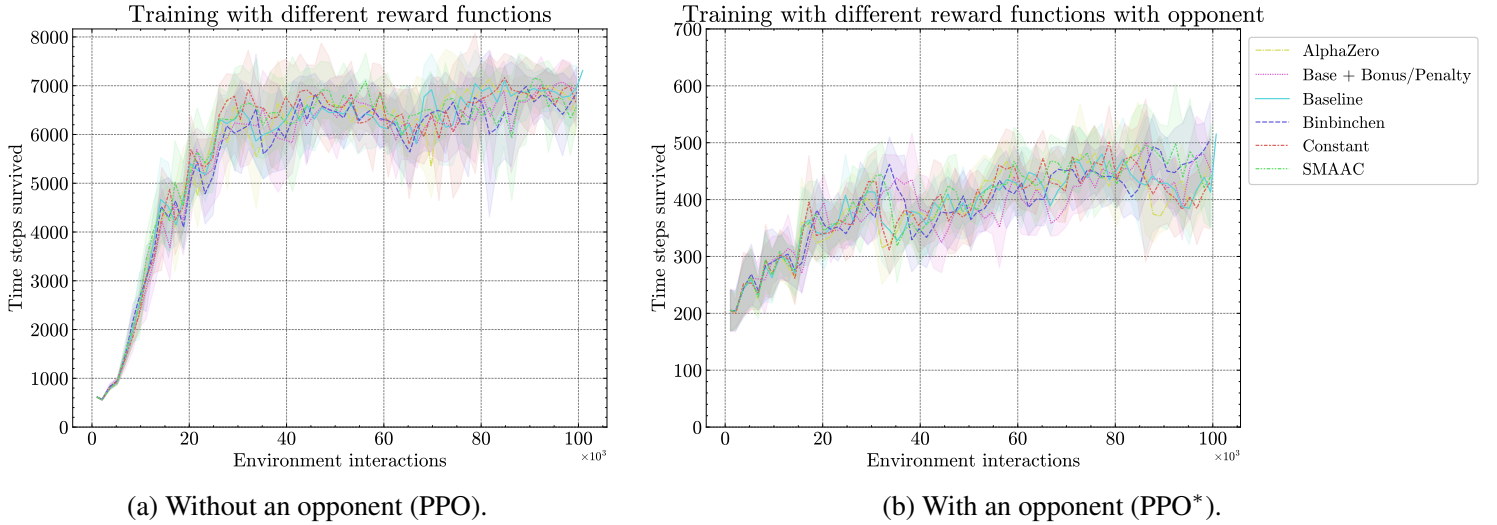


Figure 9: Training curves of PPO agents trained with different reward functions.

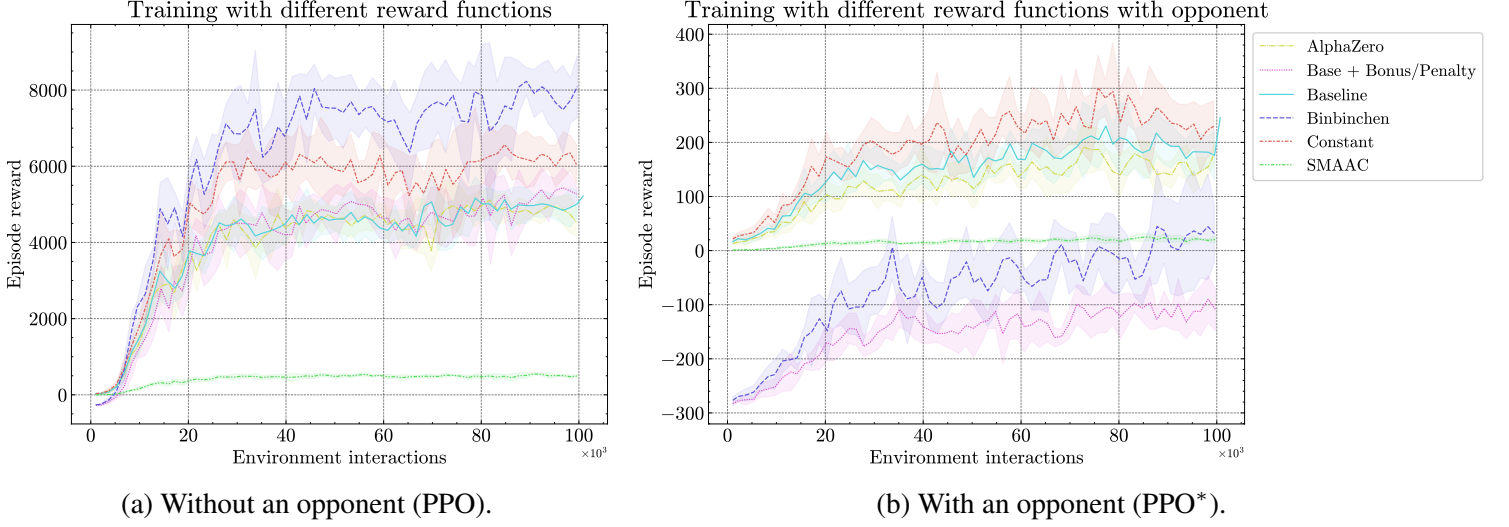


Figure 10: Curves of the mean episode reward of the PPO trained with different reward functions.

The validation results in [Tables 19](#) and [20](#) confirm that the impact of the compared reward functions on the agent’s final performance is minimal. Notably, the AlphaZero reward agent trained *with* an opponent (PPO*) performs relatively well in the environment *without* an opponent.

Agent	Reward function	Completed episodes	Steps survived	Steps over-loaded	Agent execution time [ms]	Maximum topology depth	Unique actions	Unique lines in danger	Unique subs changed
PPO	Baseline	78.6%	87.31%	0.008%	1.9668	5	27.8	11.8	6
	AlphaZero	75.6%	86.46%	0.015%	2.2298	4.8	33	12.8	6.2
	Constant	78.4%	87.83%	0.009%	2.3478	5	31.8	12	6
	SMAAC	71.6%	84.49%	0.012%	2.2804	5	39.8	13	6.6
	Binbinchen	74.8%	86.09%	0.016%	2.2614	4.2	34.6	13.2	6
	Base + Bonus/Penalty	81.0%	89.09%	0.010%	2.2558	5	30	12.4	6
PPO*	Baseline	35.4%	57.89%	0.041%	1.9658	4.6	80.6	17	6.8
	AlphaZero	58.2%	74.19%	0.023%	2.0352	4.8	66.6	15.4	6.8
	Constant	48.0%	68.69%	0.033%	2.084	5.2	74.8	15.8	7
	SMAAC	35.8%	57.21%	0.060%	2.1178	4.6	81.4	16.2	7
	Binbinchen	26.2%	46.07%	0.091%	2.072	4.8	94.8	16.8	7
	Base + Bonus/Penalty	21.1%	44.93%	0.111%	2.0162	4.6	91.6	17.4	7

Table 19: Validation results of different reward functions for the agents, applied to the environment case 14 sandbox without an opponent.

Agent	Reward function	Completed episodes	Steps survived	Steps over-loaded	Agent execution time [ms]	Maximum topology depth	Unique actions	Unique lines in danger	Unique subs changed
PPO	Baseline	0.0%	4.690%	0.638%	2.1618	4	12.8	12.2	5.4
	AlphaZero	0.0%	4.928%	0.518%	2.5898	3.4	12.2	11.4	4.6
	Constant	0.0%	4.816%	0.607%	2.4422	3.6	15.6	11.6	5.4
	SMAAC	0.0%	4.338%	0.817%	2.3208	3.4	17	13.2	6
	Binbinchen	0.0%	4.534%	0.785%	2.4766	3.8	19.4	13	5.4
	Base Bonus/Penalty +	0.0%	4.608%	0.673%	2.301	3.4	12.6	11.2	5.8
PPO*	Baseline	0.0%	5.740%	0.683%	2.2398	4.6	91	15	7
	AlphaZero	0.0%	5.970%	0.638%	2.4438	3.6	85.8	15.2	7
	Constant	0.0%	5.484%	0.669%	2.3988	3.8	90.4	14.4	6.8
	SMAAC	0.0%	5.508%	0.790%	2.5324	4	94	15.4	7
	Binbinchen	0.0%	5.878%	0.758%	2.545	4	100.6	16.4	7
	Base Bonus/Penalty +	0.0%	5.676%	0.750%	2.5292	3.8	98.2	15.6	6.8

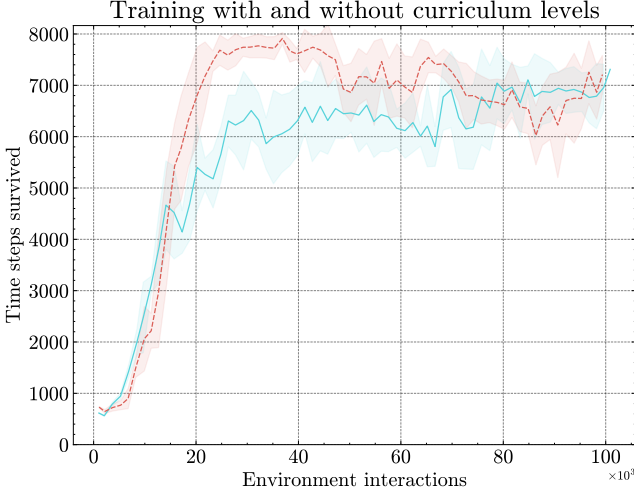
Table 20: Validation results of different reward functions for the agents, applied to the environment case 14 sandbox *with* an opponent.

6.5 Curriculum training

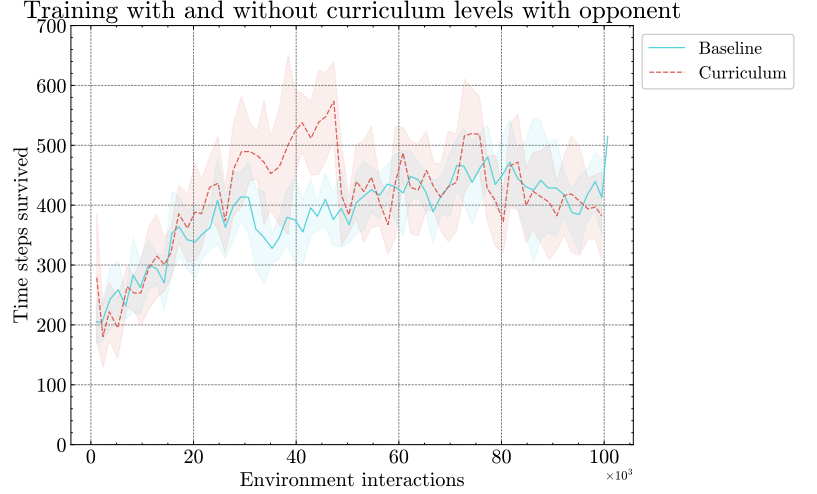
Curriculum training, which starts with relatively easy environment settings, initially provides a clear performance benefit. However, once the difficulty increases, the agent struggles to maintain its high score or recover its performance.

As shown in [Fig. 11](#), the training progress of the curriculum agent begins promising, but its performance rapidly decreases when the final difficulty level starts around 46.667 time steps, reaching a level similar to the baseline agent.

While the final performance does not surpass the baseline, curriculum training offers a clear advantage in training efficiency. Specifically, training without an opponent for the baseline agent takes 350-460 minutes, whereas training for the curriculum version takes 301-360 minutes, reducing the training time by ~ 15 -20%. Looking at the training process with an opponent, we see that the baseline agent takes 87-124 minutes and the curriculum agent takes 46-60 minutes of training, reducing the training time by ~ 50 %.



(a) Without an opponent (PPO).



(b) With an opponent (PPO*).

Figure 11: Comparison of the training curves of PPO agents trained with and without curriculum levels setup as described in [Section 5.5](#)

[Tables 21](#) and [22](#) show that the curriculum scores slightly, but not significantly, higher than the baseline agent in the environment without an opponent. However, no improvement is visible for the curriculum agents validated in the environment with an opponent.

Agent	Curriculum Training	Completed episodes	Steps survived	Steps over-loaded	Agent execution time [ms]	Maximum topology depth	Unique actions	Unique lines in danger	Unique subs changed
PPO		78.6%	87.31%	0.008%	1.967	5.0	27.8	11.8	6.0
	✓	81.6%	90.07%	0.012%	2.108	4.8	31.8	12.2	6.4
PPO*		35.4%	57.89%	0.041%	1.966	4.6	80.6	17	6.8
	✓	20.2%	38.39%	0.100%	2.241	4.4	93.2	16.8	7.0

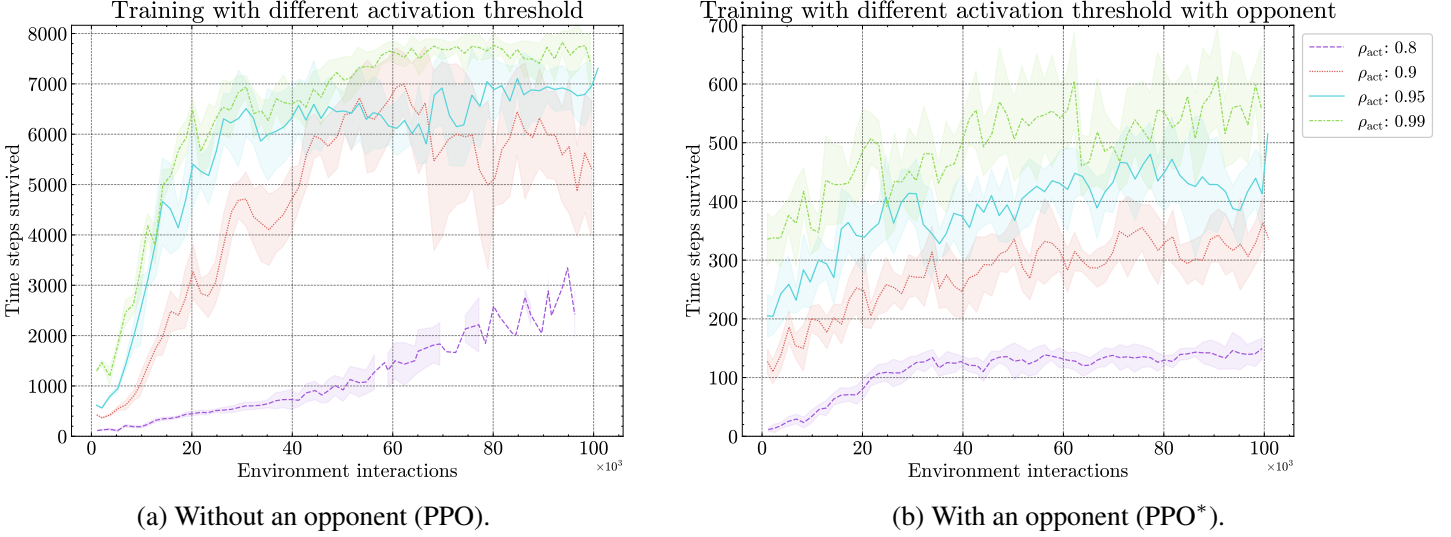
Table 21: Validation results of agents trained with curriculum levels compared with the baseline agents, applied to the environment case 14 sandbox without an opponent.

Agent	Curriculum Training	Completed episodes	Steps survived	Steps over-loaded	Agent execution time [ms]	Maximum topology depth	Unique actions	Unique lines in danger	Unique subs changed
PPO		0.0%	4.690%	0.638%	2.1618	4	12.8	12.2	5.4
	✓	0.0%	4.50%	0.664%	2.171	3.8	17.4	11.2	5.4
PPO*		0.0%	5.740%	0.683%	2.2398	4.6	91	15	7
	✓	0.0%	5.69%	0.948%	2.318	4.2	82.0	16.6	6.8

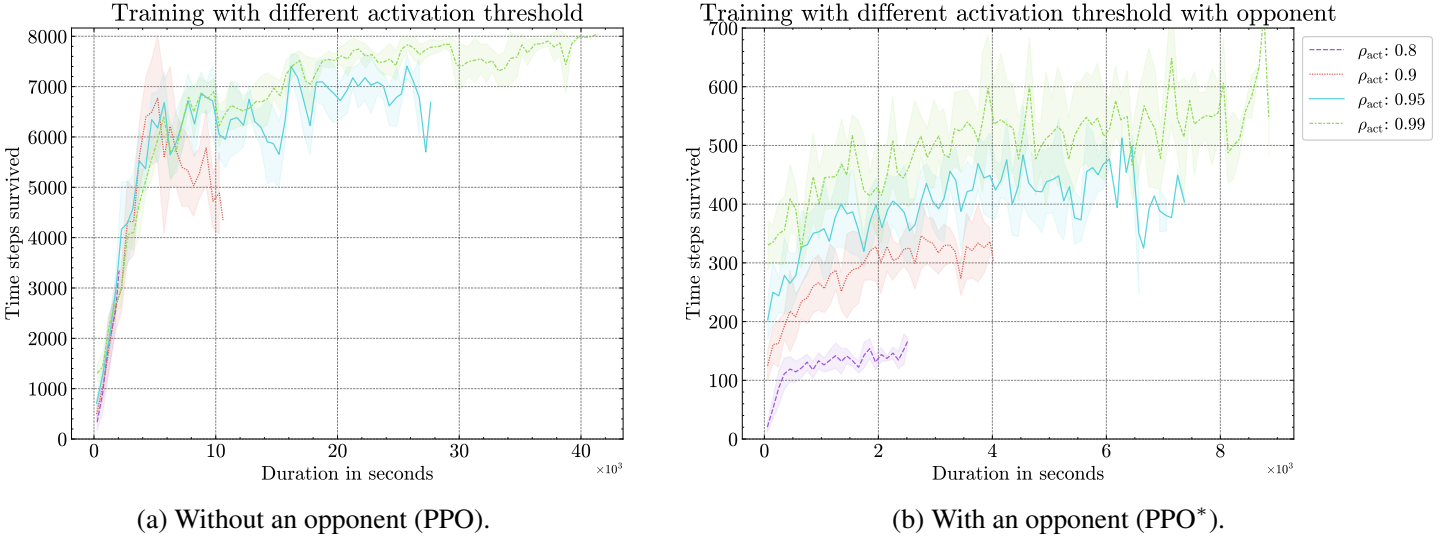
Table 22: Validation results of agents trained with curriculum levels compared with the baseline agents, applied to the environment case 14 sandbox with an opponent.

6.6 Activation threshold

[Fig. 12](#) illustrates the significant impact of the activation threshold (AT) on training progress. The plot suggests that increasing the activation threshold improves performance up to a certain point. However, beyond a certain threshold $\rho_{act} > 1.0$, performance is likely to degrade, as the agent may act too late, allowing lines to break before intervention.

Figure 12: Training curves of PPO agents trained with different activation thresholds ρ_{act} .

We remark that when the threshold is higher, the agent interacts less with the environment per episode. Therefore, as illustrated in Fig. 13, training with a higher threshold also takes longer. Figs. 12 and 13 suggest that the training curve for $\rho_{act} = 0.8$ may continue to improve in the no-opponent setting. To verify whether longer training improves performance, we conducted an additional experiment in which all agents were trained for the same duration, see Fig. 14. However, the upward trend for $\rho_{act} = 0.8$ did not persist in the extended training experiment.

Figure 13: Training curves plotting the survival time against the duration of the training process. All agents are trained for 100,000 time steps. The training curves show the PPO trained with different activation thresholds ρ_{act} .

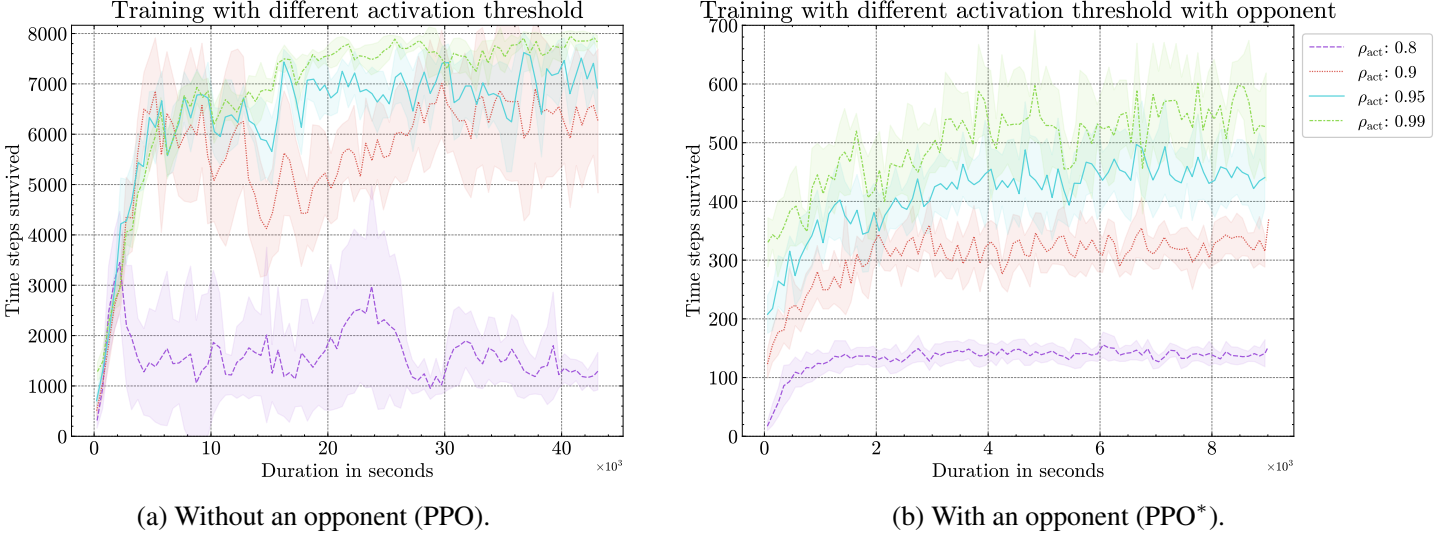


Figure 14: Training curves plotting the survival time against the duration of the training process. All agents are trained for the same duration. The training curves show the PPO trained with different activation thresholds ρ_{act} .

Intuitively, one might expect that in the presence of an opponent, anticipating earlier (i.e., using a lower AT) would be beneficial. However, Fig. 12 shows that, regardless of the opponent’s presence, the highest AT ($\rho_{act} = 0.99$) achieves the longest survival time.

Tables 23 and 24 show that the best performing agents are trained and evaluated with the highest AT ($\rho_{act} = 0.99$). The agent trained with $\rho_{act} = 0.95$ but using $\rho_{act} = 0.99$ during validation performs competitively, indicating some flexibility in selecting the AT.

Lastly, we observe that a higher AT leads to fewer redundant actions that do not alter the grid configuration. This fact indicates that a higher AT encourages more deliberate and impactful interventions, potentially improving overall stability.

Our results suggest that a relatively high AT ($\rho_{act} \in [0.95, 0.99]$) is optimal for both training and validation, leading to longer survival times and reducing redundant interventions.

Agent	AT	Train AT	Completed episodes	Steps survived	Steps over-loaded	Agent execution time [ms]	Maximum topology depth	Unique actions	Unique lines in danger	Unique subs changed
PPO	0.8	0.8	13.0%	44.95%	0.149%	1.8564	5	76.8	18.2	7
		0.9	10.8%	26.80%	0.271%	2.0542	5	67	17.4	7
		0.95	1.8%	19.05%	0.289%	1.7026	5	63.8	18.2	6.8
		0.99	2.4%	17.82%	0.389%	1.9152	5.2	46.8	17	7
	0.9	0.8	56.8%	74.66%	0.036%	2.1218	4.8	32.6	12.6	6.4
		0.9	46.6%	68.44%	0.035%	2.1662	4.8	43	14	6.6
		0.95	42.4%	66.69%	0.035%	1.783	5.2	46.8	15	6.6
		0.99	32.0%	57.98%	0.056%	2.1034	4.8	34.8	13.6	6.6
	0.95	0.8	64.6%	78.62%	0.025%	2.2682	4	20.2	8.4	6.4
		0.9	74.6%	85.21%	0.014%	2.4084	4.2	30.2	11	6.4
		0.95	78.6%	87.31%	0.008%	1.967	5.0	27.8	11.8	6.0
		0.99	73.8%	87.33%	0.013%	2.3442	4.6	24.2	12	6.4
	0.99	0.8	60.6%	75.55%	0.032%	2.3064	3.6	13.8	5.4	5.8
		0.9	75.6%	84.86%	0.021%	2.349	3.8	17.8	10	6.2
		0.95	91.6%	95.80%	0.010%	2.2824	4	16.6	7.8	6
		0.99	92.6%	97.07%	0.009%	2.4236	3.8	14.2	6.8	5
PPO*	0.8	0.8	0.0%	3.85%	0.975%	2.0288	5	112.4	18.8	7
		0.9	0.0%	5.37%	0.746%	2.0548	5	103.6	19.6	7
		0.95	0.0%	6.58%	0.491%	2.0866	5	97.4	19.2	7
		0.99	0.0%	9.95%	0.442%	1.9956	5.2	82	19.2	6.8
	0.9	0.8	0.4%	9.35%	0.341%	1.9694	4.2	106.8	18.4	7
		0.9	2.2%	22.78%	0.173%	2.0276	4.6	105.6	18.6	7
		0.95	5.6%	32.63%	0.113%	1.9982	5.2	94.6	17.8	6.6
		0.99	24.4%	48.98%	0.070%	2.111	5	67.4	16	6.8
	0.95	0.8	4.8%	19.80%	0.160%	2.0542	3.8	103.4	18	7
		0.9	25.0%	50.13%	0.063%	2.2582	4.8	96.6	17.8	7
		0.95	35.4%	57.89%	0.041%	1.9658	4.6	80.6	17	6.8
		0.99	64.8%	79.36%	0.018%	2.0402	4.4	41.6	12.8	6.2
	0.99	0.8	7.2%	24.92%	0.145%	2.2372	3.6	100.8	17.4	7
		0.9	39.0%	57.58%	0.052%	2.143	3.8	80.6	15.6	6.8
		0.95	46.4%	66.50%	0.043%	2.0842	4.2	72.4	16.2	6.4
		0.99	68.0%	81.01%	0.029%	2.1568	3.8	39.8	11.2	6

Table 23: Validation results of different activation thresholds used for the final agent agents (the column ‘AT’) combined with different activation thresholds used for the training of the agent (column ‘Train AT’), applied to the environment case 14 sandbox without an opponent. The baseline results are highlighted in grey.

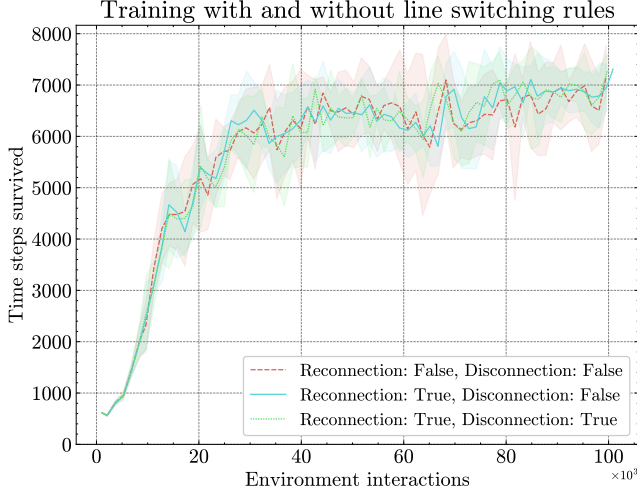
Agent	AT	Train AT	Completed episodes	Steps survived	Steps over-loaded	Agent execution time [ms]	Maximum topology depth	Unique actions	Unique lines in danger	Unique subs changed
PPO	0.8	0.8	0.0%	2.022%	1.156%	2.359	4	21.2	14.4	6.2
		0.9	0.0%	1.952%	1.126%	2.3764	4.4	29	14.4	6
		0.95	0.0%	1.972%	0.554%	1.8766	4.6	26.8	14	6
		0.99	0.0%	2.024%	0.549%	2.2802	4.6	24.4	14.6	6
	0.9	0.8	0.0%	3.498%	1.094%	2.373	3.8	17.2	13	6
		0.9	0.0%	3.488%	0.810%	2.4138	4.2	16.6	12	6
		0.95	0.0%	3.690%	0.514%	1.9192	4	14.6	11.6	6
		0.99	0.0%	3.634%	0.571%	2.4456	4.4	17	11.8	5.4
	0.95	0.8	0.0%	4.424%	0.940%	2.3154	3.4	15.6	13.4	5.4
		0.9	0.0%	4.312%	0.863%	2.253	3.2	12	12.2	5.6
		0.95	0.0%	4.690%	0.638%	2.1618	4	12.8	12.2	5.4
		0.99	0.0%	4.682%	0.712%	2.5526	3.2	12.8	12	5.6
	0.99	0.8	0.0%	5.488%	0.928%	2.2912	3	16.4	13.8	5.8
		0.9	0.0%	4.982%	0.824%	2.3132	2.6	13.2	13	5.6
		0.95	0.0%	5.450%	0.652%	2.7746	3.2	10.2	10.8	5.2
		0.99	0.0%	4.752%	0.841%	2.227	2.8	6.6	10.2	4.4
PPO*	0.8	0.8	0.0%	1.704%	1.575%	2.3988	4.6	93.8	18.2	7
		0.9	0.0%	2.150%	1.345%	2.2218	4.6	97	18.4	7
		0.95	0.0%	2.670%	1.092%	2.364	5.2	94.2	18.2	7
		0.99	0.0%	2.808%	1.062%	2.2186	5	82.2	18.2	7
	0.9	0.8	0.0%	2.872%	1.124%	2.582	4.2	94	16.2	7
		0.9	0.0%	3.662%	0.907%	2.534	4.2	95.8	16.6	6.8
		0.95	0.0%	4.714%	0.685%	2.6614	5	96.6	16.4	7
		0.99	0.0%	4.834%	0.682%	2.301	4.2	79	16	7
	0.95	0.8	0.0%	3.784%	1.002%	2.3184	3.6	86.4	15.4	6.8
		0.9	0.0%	5.352%	0.776%	3.1196	4.2	91.4	16.2	7
		0.95	0.0%	5.740%	0.683%	2.2398	4.6	91	15	7
		0.99	0.0%	5.852%	0.692%	2.2794	4	79.4	14.6	6.8
	0.99	0.8	0.0%	5.318%	0.840%	2.3642	3.2	89.8	15.4	6.8
		0.9	0.0%	5.934%	0.788%	2.4672	3.8	92	13.4	6.8
		0.95	0.0%	6.540%	0.684%	2.5556	3.8	90.8	14.6	7
		0.99	0.0%	7.042%	0.672%	2.5936	4	80.6	15.6	6.8

Table 24: Validation results of different activation thresholds used for the final agent agents (the column ‘AT’) combined with different activation thresholds used for the training of the agent (column ‘Train AT’), applied to the environment case 14 sandbox *with* an opponent. The baseline results are highlighted in grey.

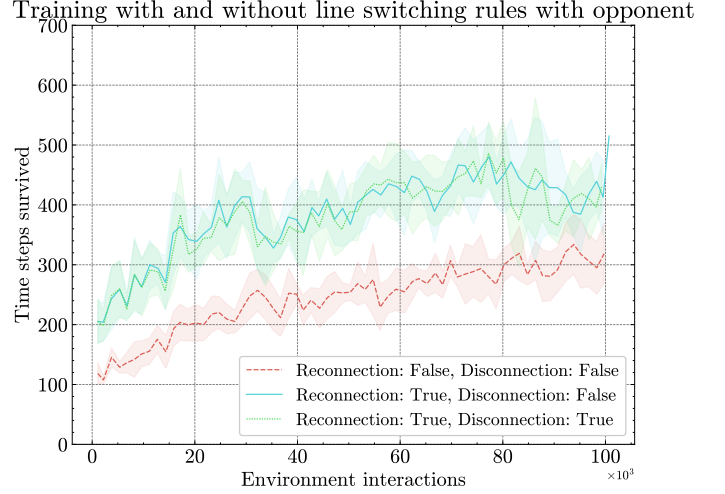
6.7 Line switching: Reconnection and disconnection

Line-switching rules, such as reconnecting or disconnecting lines, can improve the agents performance as we avoid lines being disconnect or overloaded for an unnecessary amount of time. However, as expected, the impact depends heavily on the presence of an opponent.

In an environment without an opponent (Fig. 15a and Table 25), incorporating line-switching rules does not significantly affect the agent’s survival time or overall performance. Tracking the frequency of line reconnections and disconnections during training (where applicable) reveals that these actions are rarely used. At the start of training, they are activated fewer than 0.05 times per episode on average, decreasing further to below 0.005 times per episode once the agent is fully trained. This low activation frequency explains their negligible effect on performance.



(a) Without an opponent (PPO).



(b) With an opponent (PPO*).

Figure 15: Training curves of PPO agents trained with and without line-switching rules. The baseline corresponds to the setting where the line reconnection is applied, but the line disconnection is not.

In the environment with an opponent (Fig. 15 and Table 26), line reconnections provide a clear performance benefit. However, including the line disconnection rule again has no very significant impact on the agents’ performance. The frequency of line reconnections increases significantly in the opponent environment, with agents using them approximately 3–4 times per episode on average. The line disconnection is only used around 0.01 times on average per episode for a trained agent, which is slightly more than in an environment without an opponent, but this still does not reflect in the agent’s performance.

Agent	Recon- nect	Discon- nect	Train Reconnect	Train Dis- connect	Completed episodes	Steps sur- vived	Steps over- loaded	Agent execution time [ms]	Maximum topology depth	Unique actions	Unique lines in danger	Unique subs changed
PPO	✓	✓			79.7%	89.49%	0.009%	2.057	4.7	28.33	12.2	5.8
			✓		80.8%	90.42%	0.009%	2.051	4.4	25.8	11.8	6.4
				✓	64.6%	82.21%	0.013%	2.212	5.0	58.6	14	6.8
			✓		80.3%	90.12%	0.009%	2.055	4.7	27.33	11.5	6
				✓	78.6%	87.31%	0.008%	1.967	5.0	27.8	11.8	6
			✓		81.4%	90.41%	0.009%	2.089	4.6	27.0	11.4	6.6
				✓	78.7%	89.08%	0.009%	2.411	4.5	30.7	11.3	5.8
			✓		76.2%	86.46%	0.008%	2.150	5.0	29.4	10.6	6
				✓	81.4%	89.24%	0.011%	2.587	4.4	26.8	11.6	6.4
PPO*	✓	✓			17.8%	42.03%	0.093%	1.998	4.6	88.8	17.6	7
			✓		36.8%	58.19%	0.049%	2.150	4.8	80.4	17.8	7
				✓	41.0%	63.21%	0.039%	2.138	4.6	78.2	16.0	7
			✓		20.4%	44.37%	0.093%	1.937	4.6	89.2	17.2	7
				✓	35.4%	57.89%	0.041%	1.966	4.6	80.6	17	6.8
			✓		41.8%	62.00%	0.046%	2.060	4.6	76.4	16.6	7
				✓	17.8%	41.41%	0.090%	2.910	4.6	89.6	17.8	7
			✓		35.4%	57.72%	0.046%	2.349	4.4	75.8	16.8	7
				✓	44.2%	63.93%	0.041%	2.531	4.8	78.4	16.8	6.8

Table 25: Validation results of final agents with and without reconnection and disconnection rule (columns ‘Reconnect’ and ‘Disconnect’), and trained with and without reconnection and disconnection rule (columns ‘Train Reconnect’ and ‘Train Disconnect’) applied to the environment case 14 sandbox without an opponent. The baseline results are highlighted in grey.

Agent	Recon- nect	Discon- nect	Train Reconnect	Train Dis- connect	Completed episodes	Steps sur- vived	Steps over- loaded	Agent execution time [ms]	Maximum topology depth	Unique actions	Unique lines in danger	Unique subs changed
PPO	✓	✓			0.0%	2.442%	1.671%	2.170	4.0	17.8	12.6	6.2
			✓		0.0%	2.396%	1.386%	1.985	3.6	13.8	12.2	6.0
				✓	0.0%	2.354%	1.489%	2.507	3.6	17.0	13.4	5.8
			✓		0.0%	4.814%	0.658%	2.707	3.6	12.0	10.8	4.6
				✓	0.0%	4.690%	0.638%	2.162	4.0	12.8	12.2	5.4
			✓		0.0%	4.452%	0.777%	2.485	3.2	11.0	12.0	5.2
				✓	0.0%	4.832%	0.654%	3.535	3.8	12.6	11.8	5.4
			✓		0.0%	4.604%	0.626%	3.315	3.4	11.8	11.8	5.2
				✓	0.0%	4.816%	0.631%	3.536	3.6	12.6	13.4	6.0
PPO*	✓	✓			0.0%	3.754%	1.567%	2.071	4.0	93.6	16.4	7.0
			✓		0.0%	2.542%	1.995%	1.951	4.0	86.6	13.8	7.0
				✓	0.0%	2.622%	2.149%	2.331	3.6	88.4	14.4	7.0
			✓		0.0%	5.478%	0.867%	2.244	4.0	95.2	15.8	7.0
				✓	0.0%	5.740%	0.683%	2.2398	4.6	91	15	7
			✓		0.0%	5.644%	0.640%	2.372	3.6	90.6	15.4	6.6
				✓	0.0%	5.658%	0.808%	3.265	3.8	94.6	16.4	7.0
			✓		0.0%	5.834%	0.631%	3.329	3.6	89.0	15.4	7.0
				✓	0.0%	5.870%	0.671%	3.732	4.2	96.0	16.8	7.0

Table 26: Validation results of final agents tested with and without reconnection and disconnection rule (columns ‘Reconnect’ and ‘Disconnect’), and trained with and without reconnection and disconnection rule (columns ‘Train Reconnect’ and ‘Train Disconnect’) applied to the environment case 14 sandbox *with* an opponent. The baseline results are highlighted in grey.

6.8 Revert to reference topology

Training with or without the revert-to-reference-topology rule has a minor effect on the agent’s survival time during training. For agents trained without an opponent, the training curve with the highest revert thresholds (RTs) is slightly higher near the end, as shown in Fig. 16 where the training curves in exhibit similar trends across all configurations.

By tracking the frequency of the revert actions, we see that in the environment without an opponent, trained agents revert substations to the reference topology approximately twice per episode for $\rho_{\text{rev}} = 0.8$, and around 3-4 times per episode for $\rho_{\text{rev}} = 0.9$ or $\rho_{\text{rev}} = 0.95$. With a survival time of around 7,000 time steps, this is not very frequent and, therefore, explains the minimal effect on the training curves. In the environment with an opponent, substations are reverted to the reference topology between 0.2 and 0.4 times per episode, depending on the RT value.

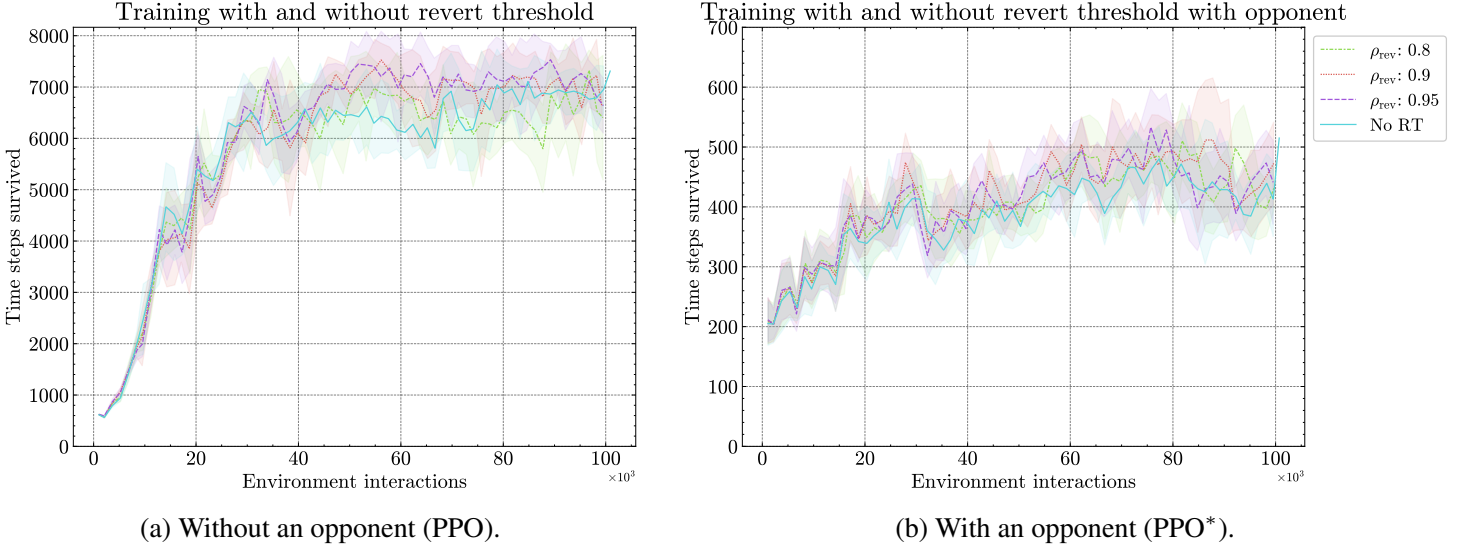


Figure 16: Training curves of PPO agents trained with and without the revert to reference topology rule. The baseline is without reverting to the reference topology.

During validation in the environment without an opponent, the best-performing configuration uses an $\rho_{\text{rev}} = 0.9$ during training and an $\rho_{\text{rev}} = 0.95$ for the final agent, see Table 27.

Table 28 shows that the PPO* agent trained with $\rho_{\text{rev}} = 0.8$ and evaluated with $\rho_{\text{rev}} = 0.9$ achieves the highest performance among all the configurations tested on the environment with an opponent.

In general, training with an RT but omitting it for the final agent decreases performance. This is expected as the agent is trained with a rule that it does not ultimately use. Furthermore, the results suggest that performance improves when the RT during training is slightly lower (making the rule less strict), leading to less frequent activation compared to the RT used in the final agent.

Agent	RT	Train RT	Completed episodes	Steps survived	Steps over-loaded	Agent execution time [ms]	Maximum topology depth	Unique actions	Unique lines in danger	Unique subs changed
PPO	0.0	0.0	78.6%	87.31%	0.008%	1.967	5.0	27.8	11.8	6.0
		0.8	54.4%	72.90%	0.040%	2.286	4.6	23.4	11.4	5.8
		0.9	34.0%	58.87%	0.069%	2.221	4.8	18.2	8.4	6.2
		0.95	38.2%	62.72%	0.064%	2.195	5.2	17.2	8.6	5.6
	0.8	0.0	77.6%	87.64%	0.008%	2.097	4.6	28.8	12.2	6.2
		0.8	77.2%	87.50%	0.011%	2.205	4.6	24.4	11.4	6.0
		0.9	81.0%	89.88%	0.020%	2.174	4.6	24.6	12.0	6.6
		0.95	82.2%	91.02%	0.017%	2.164	5.0	25.0	9.6	6.4
	0.9	0.0	78.4%	87.04%	0.007%	2.143	4.6	28.2	11.4	6.2
		0.8	80.6%	89.49%	0.011%	2.510	4.6	25.8	11.6	5.6
		0.9	87.8%	93.45%	0.016%	2.376	5.0	24.4	10.8	6.4
		0.95	88.2%	93.90%	0.013%	2.433	5.0	25.8	10.2	6.4
	0.95	0	78.0%	87.47%	0.009%	2.190	4.8	31.0	10.6	6.0
		0.8	79.2%	88.42%	0.011%	2.415	4.6	25.8	12.2	5.8
		0.9	88.8%	94.24%	0.016%	2.297	5.0	26.8	10.6	6.4
		0.95	87.6%	92.97%	0.014%	2.410	5.0	25.6	10.6	6.4
PPO*	0.0	0.0	35.4%	57.89%	0.041%	1.9658	4.6	80.6	17	6.8
		0.8	33.4%	55.64%	0.045%	2.1132	4.6	80.4	17.2	6.8
		0.9	31.6%	51.95%	0.060%	2.2036	4.8	80.4	16.6	6.8
		0.95	59.6%	74.64%	0.019%	2.0416	5	56.6	14.8	6.4
	0.8	0.0	32.0%	55.43%	0.050%	2.132	4.6	86	17.2	7
		0.8	36.0%	57.72%	0.043%	2.0768	4.6	79.8	16.4	6.8
		0.9	29.4%	54.00%	0.051%	2.1914	4.4	80.4	16.2	6.8
		0.95	54.6%	73.19%	0.023%	2.6348	4.8	60.6	14.8	6.6
	0.9	0.0	31.6%	54.92%	0.055%	1.9902	4.8	82.8	17	7
		0.8	35.8%	57.04%	0.041%	2.047	4.2	80.8	16.6	6.6
		0.9	29.2%	55.33%	0.049%	2.1774	4.2	83.6	17.4	6.6
		0.95	57.2%	76.04%	0.024%	2.1238	4.6	59.8	16.2	6.6
	0.95	0.0	31.0%	56.75%	0.050%	2.6844	4.4	88.4	16.6	6.8
		0.8	36.0%	57.37%	0.045%	2.1208	4.2	80.6	16	6.8
		0.9	31.2%	56.11%	0.049%	2.2034	4.2	80.2	16.8	6.8
		0.95	56.8%	77.12%	0.024%	2.1824	5	63.4	15.6	6.6

Table 27: Validation results of different revert thresholds used for the final agent agents (the column ‘RT’) combined with different revert thresholds used for the training of the agent (column ‘Train RT’), applied to the environment case 14 sandbox without an opponent. RT=0.0 means that no reverting to topology is applied. The baseline results are highlighted in gray.

Agent	RT	Train RT	Completed episodes	Steps survived	Steps over-loaded	Agent execution time [ms]	Maximum topology depth	Unique actions	Unique lines in danger	Unique subs changed
PPO	0.0	0.0	0.0%	4.690%	0.638%	2.162	4.0	12.8	12.2	5.4
		0.8	0.0%	4.854%	0.610%	2.733	3.8	9.4	11.4	4.6
		0.9	0.0%	4.966%	0.703%	2.297	4.4	10.2	11.0	5.4
		0.95	0.0%	4.594%	0.755%	2.413	4.0	10.8	12.0	5.0
	0.8	0.0	0.0%	4.650%	0.712%	2.212	3.6	12.6	11.6	5.6
		0.8	0.0%	4.590%	0.662%	2.425	3.2	8.8	11.0	4.6
		0.9	0.0%	4.670%	0.650%	2.420	4.0	9.6	11.4	5.4
		0.95	0.0%	4.676%	0.658%	2.367	4.0	12.4	11.6	5.4
	0.9	0.0	0.0%	4.650%	0.581%	2.064	3.4	11.2	12.4	5.6
		0.8	0.0%	4.726%	0.668%	2.628	3.6	9.0	11.6	5.0
		0.9	0.0%	4.722%	0.652%	2.426	3.6	8.2	11.4	4.8
		0.95	0.0%	4.694%	0.688%	2.489	4.2	11.8	11.2	5.4
	0.95	0.0	0.0%	4.620%	0.658%	2.082	3.6	12.8	11.4	5.4
		0.8	0.0%	4.918%	0.651%	2.523	3.0	9.2	11.8	4.6
		0.9	0.0%	4.588%	0.673%	2.411	3.8	9.2	11.2	4.4
		0.95	0.0%	4.810%	0.632%	2.349	3.8	9.0	12.2	5.4
PPO*	0.0	0.0	0.0%	5.740%	0.683%	2.2398	4.6	91	15	7
		0.8	0.0%	2.644%	2.285%	2.1626	4.2	86	15.6	7
		0.9	0.0%	2.692%	2.019%	2.262	4.4	87.2	15.6	7
		0.95	0.0%	2.564%	2.141%	2.379	4	88	15.8	6.8
	0.8	0.0	0.0%	5.190%	0.813%	1.9972	3.6	89.2	16.2	6.8
		0.8	0.0%	5.620%	0.692%	2.8124	3.6	89.8	15.6	7
		0.9	0.0%	6.066%	0.633%	2.497	3.4	93	15.2	7
		0.95	0.0%	6.180%	0.619%	2.3248	4	89.6	15.6	7
	0.9	0.0	0.0%	5.586%	0.688%	1.9358	3.6	88.6	14.4	7
		0.8	0.0%	6.460%	0.596%	2.8822	3.6	91.6	15.4	7
		0.9	0.0%	6.032%	0.689%	2.3828	3.2	94.6	15.2	7
		0.95	0.0%	6.088%	0.639%	2.467	3.6	89.8	14.2	7
	0.95	0.0	0.0%	5.716%	0.652%	2.073	3.8	88.6	14.6	6.8
		0.8	0.0%	6.146%	0.639%	2.3474	3.2	87.4	14.2	6.8
		0.9	0.0%	5.858%	0.675%	2.3884	3.6	90.4	14.8	7
		0.95	0.0%	6.256%	0.629%	2.4976	3	91	14.2	7

Table 28: Validation results of different revert thresholds used for the final agent agents (the column ‘RT’) combined with different revert thresholds used for the training of the agent (column ‘Train RT’), applied to the environment case 14 sandbox *with* an opponent. RT=0.0 means that no reverting to topology is applied. The baseline results are highlighted in grey.

6.9 Rainbow agent

This section showcases the performance of RL agents that combine the best modeling choices in view of the results described in previous sections. Inspired by [101], we refer to such an agent as ‘Rainbow agent’.

6.10 Agent configuration

To configure the Rainbow agent, we compare on the validation results in Tables 15 to 28. For the agents trained and tested in an environment without an opponent (PPO), we compared the percentages of completed episodes and steps that survived to select the most promising configuration. The setup selected for the PPO Rainbow agent is reported in Table 29.

	Train	Validate	Completed episodes	Steps survived
Action space	$\mathcal{A}_{n-0}$ (Baseline)		78.6%	87.31%
Observation space	Line loads		92.8%	96.01%
Reward function	Base + Bonus/Penalty		81.0%	89.09%
Curriculum	Curriculum training (yes)		81.6%	90.07%
Activation threshold (ρ_{act})	0.99	0.99	92.6%	97.07%
Line switching	Reconnect + Dis-connect	Reconnect + Disconnect ²⁹	81.4%	90.41%
Revert threshold (ρ_{rev})	0.9	0.95	88.8%	94.24%

Table 29: Setup for the PPO Rainbow agent trained and evaluated on an environment without an opponent. The column ‘Train’ shows the setup used during training and the column ‘Validate’ shows the setup used when running the validation of the final trained agents. The columns ‘Completed episodes’ and ‘Steps survived’ show the scores obtained in the previous experiments.

For agents trained and tested on an environment *with* an opponent (PPO*), we compare the values in the columns “Steps survived” and “Steps overloaded”, since the percentage of completed episodes is 0.0% for all agents in an environment with an opponent. The setup selected for the PPO* Rainbow agent is provided in Table 30.

	Train	Validate	Steps survived	Steps overloaded
Action space	$\mathcal{A}_{d3qn}$		6.904%	1.127%
Observation space	Baseline		5.740%	0.683%
Reward function	AlphaZero		5.970%	0.638%
Curriculum	Baseline (no)		5.740%	0.683%
Activation threshold (ρ_{act})	0.99	0.99	7.042%	0.672%
Line switching	Reconnect + Dis-connect	Reconnect + Dis-connect	5.870%	0.671%
Revert threshold (ρ_{rev})	0.8	0.9	6.460%	0.596%

Table 30: Setup for the PPO* Rainbow agent trained and evaluated on an environment *with* an opponent. The column “Train” shows the setup used during training and the column “Validate” shows the setup used when running the validation of the final trained agents. The columns “Completed episodes” and “Steps survived” show the scores obtained in the previous experiments.

²⁹Although validation results without the disconnection rule slightly outperformed those with it (Section 6.7), we chose to use the rule during both training and validation to maintain consistency, as the performance difference was negligible.

For each of the selected configurations, we test its contribution to the Rainbow agents by reverting it to its default configuration from the baseline while maintaining all other configurations for the Rainbow agent, referred to as *ablation studies* in [10].

6.11 Evaluating the Rainbow Agent

This subsection shows the results of the PPO Rainbow agent trained and validated in an environment without opponent and the PPO* agent trained and validated in an environment with an opponent.

From Figs. 17 and 18 we see that the Rainbow agent, configured with all best found configurations, indeed improves the mean episode survival time during training for both the PPO and the PPO* agent, when compared to the Baseline agent. The comparable training curves of the ablation studies reveal that the performance of reverting to a single configuration closely matches that of the Rainbow agent. For example, we see that the impact of not including the change in activation threshold (Rainbow - $\rho_{\text{act}} = 0.95$) is not as big as one might expect based on the previous results for the activation threshold.

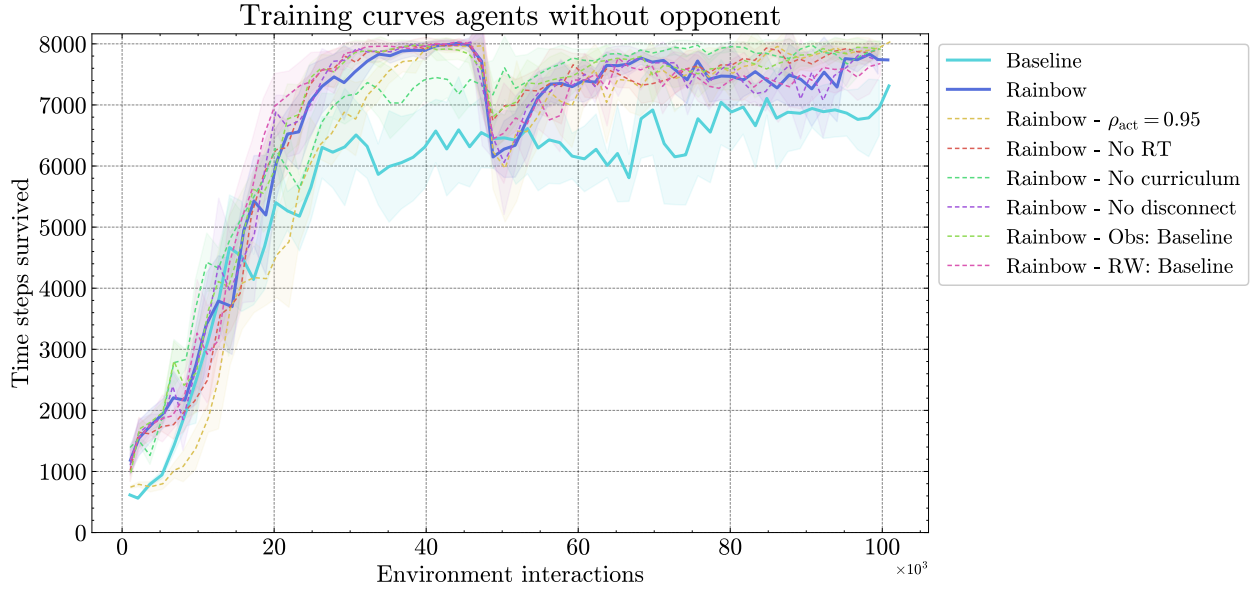


Figure 17: Training curves of the PPO Rainbow agent (trained on an environment without an opponent) compared to the Baseline and six different ablations (dashed, thinner lines).

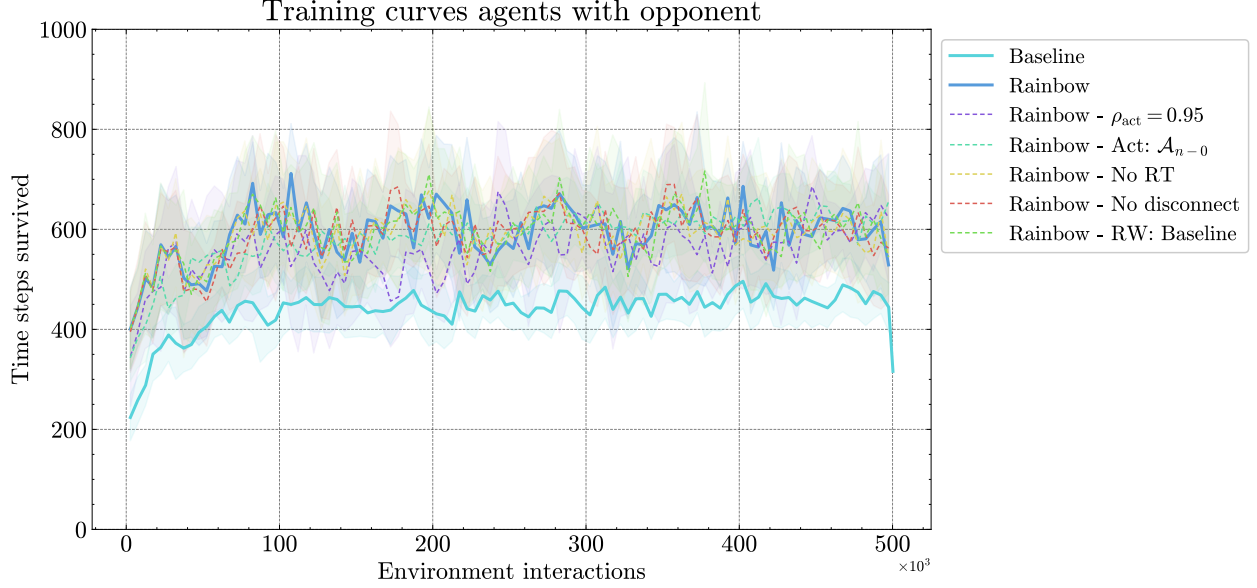


Figure 18: Training curves of the PPO* Rainbow agent (trained on an environment *with* an opponent) compared to the Baseline and five different ablations (dashed, thinner lines). These curves are smoothed with a moving average over 5.000 environment interactions to make the visualization clearer (instead of 1500).

In Table 31, the Rainbow agent achieves a notable improvement over the Baseline, completing 94.0% of episodes and surviving 96.85% of steps on average, corresponding to relative increases of 15.6% and 9.54%, respectively. However, the ablation variants have comparable or even better performance. The agent using the original observation space and the agent without curriculum training are specifically performing well. This outcome suggests that the improvements observed with the Rainbow agent may be primarily driven by a subset of enhancements, or that interactions between the enhancements produce unexpected effects. A more in-depth analysis is needed to disentangle the contributions of individual components and understand their combined impact on performance.

Agent	Completed episodes	Steps survived	Steps over-loaded	Agent execution time [ms]	Maximum topology depth	Unique actions	Unique lines in danger	Unique subs changed
Baseline	78.6%	87.31%	0.008%	1.967	5.0	27.8	11.8	6.0
Rainbow	94.0%	96.85%	0.013%	2.728	4.0	7.0	5.8	4.4
Rainbow - Obs: Baseline	97.8%	98.68%	0.010%	2.805	4.0	9.6	7.8	4.8
Rainbow - RW: Baseline	92.6%	96.25%	0.012%	2.477	4.0	6.2	5.0	4.2
Rainbow - No curriculum	97.2%	98.50%	0.010%	2.439	3.8	6.2	4.4	4.0
Rainbow - AT: 0.95	96.3%	98.22%	0.003%	2.129	4.0	9.3	6.3	5.3
Rainbow - No disconnect	95.4%	97.86%	0.014%	2.190	4.0	8.8	6.6	5.2
Rainbow - No RT	91.2%	95.15%	0.009%	2.745	3.6	6.6	5.4	4.0

Table 31: Validation results of the PPO Rainbow agent and the six ablations compared to the baseline agent.

Table 32 shows more convincingly that combining all the best-found modeling choices results in the best performance for the final agent. With a mean survival percentage of 8.65%, the Rainbow agent improves on the Baseline agent by 2.63%. Furthermore, compared to the best-scoring agent including only one enhancement, the agent with AT=0.99 (Table 30), the Rainbow agent improves by 1.60%. Using the original larger action space $\mathcal{A}_{n-0}$ (agent Rainbow - Act: $\mathcal{A}_{n-0}$ in Table 32), performed relatively well on the mean steps survived while keeping the percentage of steps overloaded low, making this configuration of modeling choices an interesting option. However, we observed that the training duration of this agent took twice as long compared to the Rainbow agent with the reduced action space ($\mathcal{A}_{d3qn}$), underlining the importance of action space reduction for larger grids for computational efficiency.

Agent	Completed episodes	Steps survived	Steps over-loaded	Agent execution time [ms]	Maximum topology depth	Unique actions	Unique lines in danger	Unique subs changed
Baseline (long)	0.0%	6.018%	0.684%	2.2808	4.2	98.6	15.8	7
Rainbow	0.0%	8.646%	1.294%	3.7762	2.6	9	14.4	3
Rainbow - Act: $\mathcal{A}_{n-0}$	0.0%	8.368%	0.694%	4.4156	3.2	91.4	15.2	7
Rainbow - RW: Baseline	0.0%	7.882%	1.294%	3.8726	2.4	9	13.2	3
Rainbow - AT: 0.95	0.0%	7.674%	1.343%	3.036	3	9	14.8	3
Rainbow - No disconnect	0.0%	8.270%	1.238%	2.2876	2.6	9	13.6	3
Rainbow - No RT	0.0%	7.978%	1.302%	4.2582	2.6	9	14.6	3

Table 32: Validation results of the PPO* Rainbow agent and the five ablations compared to the baseline agent. All agents are trained for 500.000 environment interactions, explaining the slightly higher score for the baseline agent compared to previous results.

While the Rainbow agents show a notable improvement over the RL baseline, they still fall short of outperforming the Greedy baseline. Further research should explore advanced techniques and fine-tuning hyperparameters to bridge this performance gap.

7 Discussion and Future Work

The application of reinforcement learning (RL) to power network control (PNC) has evolved rapidly since the introduction of the Learning to Run a Power Network (L2RPN) challenge in 2019. This survey has highlighted a significant body of research in this field, demonstrating the potential of RL and other machine learning techniques, such as imitation learning (IL). However, several challenges remain open. In this chapter, we discuss key issues and propose directions for future research.

Scalability of RL methods for large-scale grids One of the main challenges in applying RL to PNC is the exponential growth of the action and the state spaces as the grid size increases. Currently, researchers have explored two main approaches.

- **Action space reduction:** As shown in [Tables 4](#) and [5](#) most researchers employ a greedy reduction method for larger grids (cases 36 and 118). While effective and relatively simple to implement, this approach may exclude beneficial actions that could be valuable when combined with others. Future research should explore more sophisticated reduction techniques.
- **Factorization of the state and action space:** Most of the solutions that propose decomposition of the PNC problem define a (hierarchical) multi-agent architecture where each agent is responsible for a subset of the grid. These solutions define a factorization of the action space. However, as noted in [\[81\]](#), these agents still observe the entire grid, leading to excessive information processing. [\[81, 72\]](#) propose potentially interesting state and action factorization that could enhance scalability. Both have yet to be tested in MARL settings.

Another challenge with MARL-based approaches is the need for agents to accumulate sufficient experience. Transfer learning, which enables agents to reuse knowledge across tasks [\[103, 104\]](#), could significantly accelerate training in this multi-agent setup. While promising, transfer learning has not yet been explored in RL for PNC. Future studies could investigate its applicability in MARL setups and for adapting policies to unseen grid configurations.

Given the complexity of power grids, additional approaches beyond action reduction and factorization could be considered [\[105\]](#). The scalability of RL remains a key open challenge in this domain.

Advancing RL Techniques for Power Grid Optimization Our numerical studies in [Sections 5](#) and [6](#) evaluated a selection of proposed modeling choices for an RL-based agent applied to case 14 in Grid2Op [\[31\]](#). While the Rainbow agent, which combined our best findings, shows a notable improvement over the original RL-based baseline, it still did not surpass the Greedy baseline. Several avenues remain open for exploration:

- **Imitation Learning (IL):** Prior work [\[39, 55, 68, 67, 69, 76, 48, 49, 83\]](#) suggests that IL can significantly boost RL performance or perhaps even be used on its own. Future research should explore the benefits of IL further.

- **Off-policy techniques:** Since our experiments relied on PPO, an on-policy algorithm, we did not explore off-policy methods such as prioritized experience replay or advanced exploration strategies. These techniques could be tested using DQN-based approaches.
- **Curriculum Learning:** Curriculum training has potential but requires careful design choices. Future research could explore different difficulty design choices and adaptive hyperparameter tuning during training.
- **Neural Network Architectures:** While FCNNs are most commonly used, recent studies [49, 83] indicate that graph neural networks (GNNs) can improve performance in IL settings. The benefits of GNNs in RL-based PNC warrant further investigation. Additionally, recurrent architectures, such as LSTMs, could improve learning in environments with temporal dependencies.
- **Model-Based RL:** Most studies reviewed focus on model-free RL. The approach [AlphaZero-2022] in [61], which integrates Monte Carlo Tree Search (MCTS), demonstrated strong results in L2RPN 2022. Extending model-based planning approaches could offer new insights.

Although our numerical studies explored only a subset of possible RL techniques, we hope our methodology and our overview provide a foundation for future research.

Extend findings to larger and realistic grids Our experiments focused on Case 14 due to computational constraints. Future research should extend these findings to larger test cases (36 and 118) in Grid2Op. However, full-scale experiments with large action spaces may be infeasible. Our results indicate that environments with opponents benefit from smaller, more robust action spaces, suggesting that extensive action-space experiments may not be necessary for larger grids.

A significant challenge remains in bridging the gap between Grid2Op test cases and real-world power grids. A recent enhancement in Grid2Op provides the option to adjust the number of busbars per substation, bringing simulations closer to reality. However, to the best of our knowledge, this new setup has not yet been explored. Future studies should evaluate their impact on RL training and performance.

Lastly, in [48], the lines attacked by the opponent differ from [74]. [48] found that by disabling some of the lines the network becomes almost entirely inoperable. Therefore, they consider a particular subset of lines for disablement. Our experiments confirm that agents struggle in the current adversarial settings, as proposed by [74]. Re-running these experiments with a more balanced opponent configuration could yield further insights.

8 Conclusion

In this survey, we have explored the application of reinforcement learning (RL) to power grid topology optimization, highlighting key methods, challenges, and recent advancements in the field. Since the introduction of the L2RPN challenge in 2019, research in this area has grown significantly, with diverse approaches emerging to address the complexities of power network control (PNC). Our study has categorized and analyzed various techniques, including action space reduction, hierarchical multi-agent systems, and advanced RL algorithms, offering insights into their respective advantages and limitations. Our comparative study further provides quantitative insights into commonly used methods, offering a clearer understanding of their impact on performance.

Despite notable progress, several challenges remain, including the need for more scalable RL algorithms, improved generalization across networks, and bridging the gap between simulated test cases and existing grid operation frameworks. Addressing these challenges is important to realize the full potential of RL in power grid optimization.

Nevertheless, given the progress already achieved, the rapid growth of research in RL, and the many underexplored techniques with potential, such as imitation learning, off-policy techniques, curriculum learning, different neural network architectures and model-based RL mentioned in Section 7, we believe RL remains a promising direction to support grid operators in PNC.

We hope that this survey provides a valuable foundation for researchers and practitioners interested in RL-based power grid topology optimization and inspires future work to address the outstanding challenges in this field.

Acknowledgements We want to thank Jan Viebahn (TenneT) and Davide Barbieri (TenneT) for the interesting discussions. Furthermore, we would like to thank Barbera de Mol for the collaboration at the beginning of this project on a joint codebase. Lastly, we thank RTE France for developing and providing the Grid2Op environment, which facilitates the RL-based research on power network control.

References

- [1] A. Marot, B. Donnot, C. Romero, B. Donon, M. Lerousseau, L. Veyrin-Forrer, I. Guyon, Learning to run a power network challenge for training topology controllers, *Electr. Power Syst. Res.* 189 (2020) 106635.
- [2] T. TSO, Rapport monitoring leveringszekerheid. tech. rep., <https://www.tennet.eu/nl/over-tennet/publicaties/rapport-monitoring-leveringszekerheid>, accessed 07-08-2024 (2024).
- [3] ENTSO, Entso-e market report 2023, https://eepublicdownloads.entsoe.eu/clean-documents/nc-tasks/ENTSO-E_Market_Report_2023.pdf, accessed 07-08-2024 (2023).
- [4] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, N. Freitas, Dueling network architectures for deep reinforcement learning, in: *International conference on machine learning*, PMLR, 2016, pp. 1995–2003.
- [5] H. Van Hasselt, A. Guez, D. Silver, Deep reinforcement learning with double q-learning, in: *Proceedings of the AAAI conference on artificial intelligence*, Vol. 30, 2016.
- [6] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, *arXiv preprint arXiv:1707.06347* (2017).
- [7] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, et al., Mastering the game of go without human knowledge, *nature* 550 (7676) (2017) 354–359.
- [8] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine, Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, in: *ICML*, PMLR, 2018, pp. 1861–1870.
- [9] M. Glavic, R. Fonteneau, D. Ernst, Reinforcement learning for electric power system decision and control: Past considerations and perspectives, *IFAC-PapersOnLine* 50 (1) (2017) 6918–6927.
- [10] M. Glavic, (deep) reinforcement learning for electric power system control and related problems: A short review and perspectives, *Annual Reviews in Control* 48 (2019) 22–35.
- [11] A. Kelly, A. O’Sullivan, P. de Mars, A. Marot, Reinforcement learning for electricity network operation, *arXiv preprint arXiv:2003.07339* (2020).
- [12] A. Marot, B. Donnot, S. Tazi, P. Panciatici, Expert system for topological remedial action discovery in smart grids, in: *Mediterranean Conference on Power Generation, Transmission, Distribution and Energy Conversion (MEDPOWER 2018)*, IET, 2018, pp. 1–6.
- [13] R. Bacher, H. Glavitsch, Network topology optimization with security constraints, *IEEE Transactions on Power Systems* 1 (4) (1986) 103–111.
- [14] S. Frank, I. Steponavice, S. Rebennack, Optimal power flow: A bibliographic survey i: Formulations and deterministic methods, *Energy systems* 3 (2012) 221–258.
- [15] F. Capitanescu, J. M. Ramos, P. Panciatici, D. Kirschen, A. M. Marcolini, L. Platbrood, L. Wehenkel, State-of-the-art, challenges, and future trends in security constrained optimal power flow, *Electric power systems research* 81 (8) (2011) 1731–1741.
- [16] F. Capitanescu, Critical review of recent advances and further developments needed in ac optimal power flow, *Electric Power Systems Research* 136 (2016) 57–68.
- [17] E. B. Fisher, R. P. O’Neill, M. C. Ferris, Optimal transmission switching, *IEEE transactions on power systems* 23 (3) (2008) 1346–1355.
- [18] K. W. Hedman, R. P. O’Neill, E. B. Fisher, S. S. Oren, Optimal transmission switching with contingency analysis, *IEEE Transactions on Power Systems* 24 (3) (2009) 1577–1586.
- [19] M. Khanabadi, H. Ghasemi, M. Doostizadeh, Optimal transmission switching considering voltage security and n-1 contingency analysis, *IEEE Transactions on Power Systems* 28 (1) (2012) 542–550.
- [20] J. D. Fuller, R. Ramasra, A. Cha, Fast heuristics for transmission-line switching, *IEEE Transactions on Power Systems* 27 (3) (2012) 1377–1386.
- [21] K. W. Hedman, S. S. Oren, R. P. O’Neill, A review of transmission switching and network topology optimization, in: *2011 IEEE power and energy society general meeting*, IEEE, 2011, pp. 1–7.
- [22] M. Numan, M. F. Abbas, M. Yousif, S. S. Ghoneim, A. Mohammad, A. Noorwali, The role of optimal transmission switching in enhancing grid flexibility: A review, *IEEE Access* 11 (2023) 32437–32463.
- [23] Y. Zhou, A. S. Zamzam, A. Bernstein, H. Zhu, Substation-level grid topology optimization using bus splitting, in: *2021 American Control Conference (ACC)*, IEEE, 2021, pp. 1–7.

- [24] M. Heidarifar, H. Ghasemi, A network topology optimization model based on substation and node-breaker modeling, *IEEE Transactions on Power Systems* 31 (1) (2015) 247–255.
- [25] B. Park, C. L. Demarco, Optimal network topology for node-breaker representations with ac power flow constraints, *IEEE Access* 8 (2020) 64347–64355.
- [26] S. Babaeinejadsarookolae, B. Park, B. Lesieutre, C. L. DeMarco, Transmission congestion management via node-breaker topology control, *IEEE Systems Journal* 17 (3) (2023) 3413–3424.
- [27] J. Viebahn, M. Naglic, A. Marot, B. Donnot, S. H. Tindemans, Potential and challenges of ai-powered decision support for short-term system operations, in: *CIGRE Session*, Vol. 2022, 2022.
- [28] R. S. Sutton, *Reinforcement learning: an introduction*, A Bradford Book (2018).
- [29] R. Bellman, *Dynamic programming*, *science* 153 (3731) (1966) 34–37.
- [30] P.-T. De Boer, D. P. Kroese, S. Mannor, R. Y. Rubinstein, A tutorial on the cross-entropy method, *Annals of operations research* 134 (2005) 19–67.
- [31] B. Donnot, *Grid2op-a testbed platform to model sequential decision making in power systems*, GitHub repository (2020).
- [32] M. Lerousseau, Design and implementation of an environment for learning to run a power network (l2rpn), *arXiv preprint arXiv:2104.04080* (2021).
- [33] G. Serré, E. Boguslawski, B. Donnot, A. Pavão, I. Guyon, A. Marot, Reinforcement learning for energies of the future and carbon neutrality: a challenge design, *arXiv preprint arXiv:2207.10330* (2022).
- [34] A. Marot, I. Guyon, B. Donnot, G. Dulac-Arnold, P. Panciatici, M. Awad, A. O’Sullivan, A. Kelly, Z. Hampel-Arias, L2rpn: Learning to run a power network in a sustainable world neurips2020 challenge design, Réseau de Transport d’Électricité, Paris, France, White Paper (2020).
- [35] A. Marot, B. Donnot, K. Chaouache, A. Kelly, Q. Huang, R.-R. Hossain, J. L. Cremer, Learning to run a power network with trust, *Electr. Power Syst. Res.* 212 (2022) 108487.
- [36] D. A. E. Lab, Learning to run a power network - delft 2023 competition, <https://codalab.lisn.upsaclay.fr/competitions/12420>, codaLab; Accessed 08-08-2024 (2023).
- [37] RTE, EPRI, Chalearn, Tailor, E. Union, L2rpn challenge, <https://l2rpn.chalearn.org/>, chalearn; Accessed 20-08-2024 (2024).
- [38] A. R. R. Matavalam, K. P. Guddanti, Y. Weng, V. Ajjarapu, Curriculum based reinforcement learning of grid topology controllers to prevent thermal cascading, *IEEE Transactions on Power Systems* 38 (5) (2022) 4206–4220.
- [39] T. Lan, J. Duan, B. Zhang, D. Shi, Z. Wang, R. Diao, X. Zhang, Ai-based autonomous line flow control via topology adjustment for maximizing time-series atcs, in: *2020 IEEE Power & Energy Society General Meeting (PESGM)*, IEEE, 2020, pp. 1–5.
- [40] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, K. Kavukcuoglu, Asynchronous methods for deep reinforcement learning, in: *International conference on machine learning*, PMLR, 2016, pp. 1928–1937.
- [41] Y. Bengio, J. Louradour, R. Collobert, J. Weston, Curriculum learning, in: *Proceedings of the 26th annual international conference on machine learning*, 2009, pp. 41–48.
- [42] J. Kober, J. Peters, Imitation and reinforcement learning, *IEEE Robotics & Automation Magazine* 17 (2) (2010) 55–62.
- [43] T. Schaul, J. Quan, I. Antonoglou, D. Silver, Prioritized experience replay, *arXiv preprint arXiv:1511.05952* (2015).
- [44] D. Yoon, S. Hong, B.-J. Lee, K.-E. Kim, Winning the l2rpn challenge: Power grid management via semi-Markov afterstate actor-critic, in: *International Conference on Learning Representations*, 2021.
- [45] W. B. Powell, *Approximate Dynamic Programming: Solving the curses of dimensionality*, Vol. 703, John Wiley & Sons, 2007.
- [46] M. Hassouna, C. Holzhüter, P. Lytaev, J. Thomas, B. Sick, C. Scholz, Graph reinforcement learning for power grids: A comprehensive survey, *arXiv preprint arXiv:2407.04522* (2024).
- [47] E. van der Sar, A. Zocca, S. Bhulai, Multi-agent reinforcement learning for power grid topology optimization, *arXiv preprint arXiv:2310.02605* (2023).

- [48] M. de Jong, J. Viebahn, Y. Shapovalova, Imitation learning for intra-day power grid operation through topology actions, arXiv preprint arXiv:2407.19865 (2024).
- [49] M. de Jong, J. Viebahn, Y. Shapovalova, Generalizable graph neural networks for robust power grid topology control, arXiv preprint arXiv:2501.07186 (2025).
- [50] Z. Yan, Y. Xu, Learning to run a power network wcci 2020 competition - one possible solution, https://github.com/ZM-Learn/L2RPN_WCCI_a_Solution, GitHub; Accessed 13-08-2024 (2020).
- [51] A. Marot, B. Donnot, G. Dulac-Arnold, A. Kelly, A. O’Sullivan, J. Viebahn, M. Awad, I. Guyon, P. Panciatici, C. Romero, Learning to run a power network challenge: a retrospective analysis, in: Proceedings of the NeurIPS 2020, Vol. 133 of PMLR, 2021, pp. 112–132.
- [52] L. Omnes, A. Marot, B. Donnot, Adversarial training for a continuous robustness control problem in power systems, in: 2021 IEEE Madrid PowerTech, IEEE, 2021, pp. 1–6.
- [53] B. Zhou, H. Zeng, Y. Liu, K. Li, F. Wang, H. Tian, Action set based policy optimization for safe power grid management, in: Machine Learning and Knowledge Discovery in Databases. Applied Data Science Track, Springer International Publishing, 2021, pp. 168–181.
- [54] T. Salimans, J. Ho, X. Chen, S. Sidor, I. Sutskever, Evolution strategies as a scalable alternative to reinforcement learning, arXiv preprint arXiv:1703.03864 (2017).
- [55] H. T. El Innovation Lab, Huawei Cloud, Neurips competition 2020: Learning to run a power network (l2rpn) - robustness track, https://github.com/AsprinChina/L2RPN_NIPS_2020_a_PP0_Solution, GitHub; accessed 01-08-2024 (2020).
- [56] Y. Zhihong, X. Chunlei, L. Jixiang, X. Hongsheng, X. Kang, C. Tianyu, X. Zhilin, L. Junjun, L. Jinjun, C. Tianhua, A winning approach of neurips_2020_l2rpn_comp, https://github.com/lujasone/NeurIPS_2020_L2RPN_Comp_An_Approach, GitHub; Accessed 13-08-2024 (2020).
- [57] A. Kelly, A. Marot, RTE and eprri present: Learning to run a power network (l2rpn) 2021 – towards a machine learning based control center digital assistant, <https://www.epri.com/events/E4B4A785-98DB-4258-935E-AC1DC26D0299>, ePRI; Accessed 20-08-2024 (2021).
- [58] C. Wang, X. WenToa, X. Zhang, R. Qin, Y. Yu, Winner of l2rpn icaps 2021, https://github.com/polixir/L2RPN_2021, GitHub; Accessed 26-08-2024 (2021).
- [59] H. Martinez, L2rpn, <https://github.com/horacioMartinez/L2RPN>, GitHub; Accessed 26-08-2024 (2021).
- [60] E. A. Maze-RL, Maze-rl l2rpn - icaps 2021 submission, <https://github.com/enlite-ai/maze-l2rpn-2021-submission>, GitHub; Accessed 23-08-2024 (2021).
- [61] M. Dorfer, A. R. Fuxjäger, K. Kozak, P. M. Blies, M. Wasserer, Power grid congestion management via topology optimization with alphazero, arXiv preprint arXiv:2211.05612 (2022).
- [62] A. Pavão, Learning to run a power network with renewable energies: a challenge design and analysis, <https://adrienpavao.com/blog/l2rpn-report/l2rpn-report.html>, accessed 20-08-2024 (2022).
- [63] A. R. Fuxjäger, K. Kozak, M. Dorfer, P. M. Blies, M. Wasserer, Reinforcement learning based power grid day-ahead planning and ai-assisted control, arXiv preprint arXiv:2302.07654 (2023).
- [64] R. G. Alibaba, L2rpn wcci 2022 competition, <https://github.com/AlibabaResearch/l2rpn-wcci-2022/tree/main>, GitHub; Accessed 15-08-2024 (2022).
- [65] RTE France, Welcome to l2rpn-baselines’s documentation!, <https://l2rpn-baselines.readthedocs.io/en/latest/index.html>, read the Docs; Accessed 26-08-2024 (2024).
- [66] E. Honda Research Institute, L2rpn trial and error agent, https://github.com/HRI-EU/l2rpn_tae_agent, GitHub; Accessed 15-08-2024 (2022).
- [67] J. Sintes, V. T. Dang, How we built the winning real time autonomous agent for power grid management in the l2rpn challenge 2023, <https://lajavaness.medium.com/how-we-built-the-winning-real-time-autonomous-agent-for-power-grid-management-in-the-l2rpn-41ab3c>, medium; accessed: 01-08-2024 (2023).
- [68] M. Lehna, J. Viebahn, A. Marot, S. Tomforde, C. Scholz, Managing power grids through topology actions: A comparative study between advanced rule-based and reinforcement learning agents, Energy and AI 14 (2023) 100276.
- [69] N. Lair, A. Bossavy, P. Champion, V. Renault, Artificial agents designed to run a power network - white paper, https://www.artelys.com/app/uploads/2024/04/White_paper_L2RPN_2023_.pdf, accessed: 05-08-2024 (2023).

- [70] M. Subramanian, J. Viebahn, S. H. Tindemans, B. Donnot, A. Marot, Exploring grid topology reconfiguration using a simple deep reinforcement learning approach, 2021 IEEE Madrid PowerTech (2021) 1–6.
- [71] I. Damjanović, I. Pavić, M. Puljiz, M. Brcic, Deep reinforcement learning-based approach for autonomous power flow control using only topology changes, *Energies* 15 (19) (2022) 6920.
- [72] N. Henka, Q. Francois, S. Tazi, M. Ruiz, P. Panciatici, Power grid segmentation for local topological controllers, *Electric Power Systems Research* 213 (2022) 108302.
- [73] A. Chauhan, M. Baranwal, A. Basumatary, Powrl: A reinforcement learning framework for robust management of power networks, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37, 2023, pp. 14757–14764.
- [74] B. Manczak, J. Viebahn, H. van Hoof, Hierarchical reinforcement learning for power network topology control (2023). [arXiv:2311.02129](https://arxiv.org/abs/2311.02129).
- [75] X. Hu, Y. Zhang, H. Xia, W. Wei, Q. Dai, J. Li, Towards fair power grid control: A hierarchical multi-objective reinforcement learning approach, *IEEE Internet of Things Journal* 11 (2024) 6582 – 6595.
- [76] M. Lehna, C. Holzhüter, S. Tomforde, C. Scholz, Hugo–highlighting unseen grid options: Combining deep reinforcement learning with a heuristic target topology approach, *arXiv preprint arXiv:2405.00629* (2024).
- [77] X. Wang, N. Lu, Alleviating imbalanced problems of reinforcement learning when applying in real-time power network dispatching and control, *Expert Systems with Applications* 255 (2024) 124730.
- [78] D. Horgan, J. Quan, D. Budden, G. Barth-Maron, M. Hessel, H. Van Hasselt, D. Silver, Distributed prioritized experience replay, *arXiv preprint arXiv:1803.00933* (2018).
- [79] R. S. Sutton, D. Precup, S. Singh, Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning, *Artificial intelligence* 112 (1-2) (1999) 181–211.
- [80] E. Boguslawski, A. Leite, M. Dussartre, B. Donnot, M. Schoenauer, Emulation of zonal controllers for the power system transport problem, *RJCIA* (2024) 41.
- [81] G. Losapio, D. Beretta, M. Mussi, A. M. Metelli, M. Restelli, State and action factorization in power grids, *arXiv preprint arXiv:2409.04467* (2024).
- [82] B. de Mol, D. Barbieri, J. Viebahn, D. Grossi, Centrally coordinated multi-agent reinforcement learning for power grid topology control, *arXiv preprint arXiv:2502.08681* (2025).
- [83] M. Hassouna, C. Holzhüter, M. Lehna, M. de Jong, J. Viebahn, B. Sick, C. Scholz, Learning topology actions for power grid control: A graph-based soft-label imitation learning approach, *arXiv preprint arXiv:2503.15190* (2025).
- [84] RTE France, Welcome to grid2op’s documentation, <https://grid2op.readthedocs.io/en/latest/>, read the Docs; Accessed 26-08-2024 (2024).
- [85] S. Ioffe, Batch normalization: Accelerating deep network training by reducing internal covariate shift, *arXiv preprint arXiv:1502.03167* (2015).
- [86] H. P. Van Hasselt, A. Guez, M. Hessel, V. Mnih, D. Silver, Learning values across many orders of magnitude, *Advances in neural information processing systems* 29 (2016).
- [87] G. Klambauer, T. Unterthiner, A. Mayr, S. Hochreiter, Self-normalizing neural networks, *Advances in neural information processing systems* 30 (2017).
- [88] A. Bhatt, M. Argus, A. Amiranashvili, T. Brox, Crossnorm: Normalization for off-policy td reinforcement learning, *arXiv preprint arXiv:1902.05605* 10 (2019).
- [89] N. Bjorck, C. P. Gomes, K. Q. Weinberger, Towards deeper deep reinforcement learning with spectral normalization, *Advances in neural information processing systems* 34 (2021) 8242–8255.
- [90] J. Eschmann, Reward function design in reinforcement learning, *Reinforcement Learning Algorithms: Analysis and Applications* (2021) 25–33.
- [91] S. Schaal, Is imitation learning the route to humanoid robots?, *Trends in cognitive sciences* 3 (6) (1999) 233–242.
- [92] T. Osa, J. Pajarinen, G. Neumann, J. A. Bagnell, P. Abbeel, J. Peters, et al., An algorithmic perspective on imitation learning, *Foundations and Trends® in Robotics* 7 (1-2) (2018) 1–179.
- [93] D. A. Pomerleau, Alvin: An autonomous land vehicle in a neural network, *Advances in neural information processing systems* 1 (1988).
- [94] M. Bain, C. Sammut, A framework for behavioural cloning., in: *Machine Intelligence* 15, 1995, pp. 103–129.

- [95] S. Russell, Learning agents for uncertain environments, in: Proceedings of the eleventh annual conference on Computational learning theory, 1998, pp. 101–103.
- [96] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, et al., Mastering the game of go with deep neural networks and tree search, *nature* 529 (7587) (2016) 484–489.
- [97] S. Ross, D. Bagnell, Efficient reductions for imitation learning, in: Proceedings of the thirteenth international conference on artificial intelligence and statistics, JMLR Workshop and Conference Proceedings, 2010, pp. 661–668.
- [98] S. Ross, G. Gordon, D. Bagnell, A reduction of imitation learning and structured prediction to no-regret online learning, in: Proceedings of the fourteenth international conference on artificial intelligence and statistics, JMLR Workshop and Conference Proceedings, 2011, pp. 627–635.
- [99] P. D. Hines, I. Dobson, P. Rezaei, Cascading power outages propagate locally in an influence graph that is not the actual grid topology, *IEEE Transactions on Power Systems* 32 (2) (2016) 958–967.
- [100] A. Marot, S. Tazi, B. Donnot, P. Panciatici, Guided machine learning for power grid segmentation, in: 2018 IEEE PES Innovative Smart Grid Technologies Conference Europe (ISGT-Europe), IEEE, 2018, pp. 1–6.
- [101] M. Hessel, J. Modayil, H. Van Hasselt, T. Schaul, G. Ostrovski, W. Dabney, D. Horgan, B. Piot, M. Azar, D. Silver, Rainbow: Combining improvements in deep reinforcement learning, in: Proceedings of the AAAI conference on artificial intelligence, Vol. 32, 2018.
- [102] E. Liang, R. Liaw, R. Nishihara, P. Moritz, R. Fox, K. Goldberg, J. Gonzalez, M. Jordan, I. Stoica, Rllib: Abstractions for distributed reinforcement learning, in: International conference on machine learning, PMLR, 2018, pp. 3053–3062.
- [103] A. Taylor, I. Dusparic, M. Guériau, S. Clarke, Parallel transfer learning in multi-agent systems: What, when and how to transfer?, in: 2019 International Joint Conference on Neural Networks (IJCNN), IEEE, 2019, pp. 1–8.
- [104] F. L. Da Silva, A. H. R. Costa, A survey on transfer learning for multiagent reinforcement learning systems, *Journal of Artificial Intelligence Research* 64 (2019) 645–703.
- [105] W. van Heeswijk, Five ways to handle large action spaces in reinforcement learning, <https://towardsdatascience.com/five-ways-to-handle-large-action-spaces-in-reinforcement-learning-8ba6b6ca> (2023).

A Acronyms

Throughout this paper, we use acronyms to refer to the solutions discussed. An overview of these acronyms, along with their corresponding reference papers and code if available, is provided in [Table 33](#).

In addition, [Table 34](#) provides an overview of the other acronyms used throughout this paper.

³⁰Their preceding version can be found at <https://github.com/enlite-ai/maze-l2rpn-2021-submission>.

Solution acronym	Reference paper	Reference code
A3C-2019	[38]	https://github.com/amar-iastate/L2RPN-using-A3C
DDQN-2019	[39]	https://github.com/shidi1985/L2RPN
SMAAC-2021	[44]	https://github.com/sunghoonhong/SMAAC
A3C-2020	[50]	https://github.com/ZM-Learn/L2RPN_WCCI_a_Solution
SAS-2021	[53]	https://github.com/PaddlePaddle/PARL/tree/develop/examples/NeurIPS2020-Learning-to-Run-a-Power-Network-Challenge
Binbinchen-2020	[55]	https://github.com/AsprinChina/L2RPN_NIPS_2020_a_PPO_Solution
D3QN-2020	[56]	https://github.com/lujasone/NeurIPS_2020_L2RPN_Comp_An_Approach
CEM-2021	[70]	N.A.
D3QN-2022	[71]	N.A.
PowRL-2022	[73]	N.A.
AlphaZero-2022	[61]	https://github.com/enlite-ai/maze-l2rpn-2022-submission ⁸⁰
BruteForce-2022	[64]	https://github.com/AlibabaResearch/l2rpn-wcci-2022
HRI-EU-2022	[66]	https://github.com/HRI-EU/l2rpn_tae_agent
Curriculum-2023	[68]	https://github.com/FraunhoferIEE/CurriculumAgent
HRL-2023	[74]	https://github.com/bmanczak/runPowerNetworks
MARL-2023	[47]	https://gitlab.com/ericavandersar/marl4powergridtopo
LJNAgent-2024	[67]	N.A.
Artelys-2024	[69]	N.A.
HUGO-2024	[76]	https://github.com/FraunhoferIEE/CurriculumAgent
IL-2024	[48]	https://github.com/MatthijsdeJ/Imitation_Learning_Topology_Control
BDQN-2024	[77]	N.A.
Zonal-2024	[80]	N.A.
GNNIL-2025	[49]	https://github.com/MatthijsdeJ/GNN_PN_Imitation_Learning
CCMA-2025	[82]	N.A.
SoftIL-2025	[83]	N.A.

Table 33: Overview of the solution acronyms used in this paper and references to the corresponding papers.

Acronym	Meaning
A3C	Asynchronous-Advantage-Actor-Critic
AT	Activation Threshold
BC	Behavioral Cloning
CE	Cross-Entropy
D3QN	Double Dueling DQN
DN-action	Do-Nothing action
DQN	Deep Q-Learning (or Deep Q-Network)
DRL	Deep-RL
FCNN	Fully Connected Neural Network
GNN	Graph Neural Network
HRL	Hierarchical Reinforcement Learning
IL	Imitation Learning
IRL	Inverse RL
L2RPN	Learning to Run a Power Network
LODF	Line Outage Distribution Factor
MARL	Multi-Agent Reinforcement Learning
MCTS	Monte Carlo Tree Search
MDP	Markov Decision Process
PER	Prioritized Experience Replay
PNC	Power Network Control
PPO	Proximal Policy Optimization
RL	Reinforcement Learning
RT	Revert Threshold
SAC	Soft Actor-Critic
TD	Temporal Difference
TSO	Transmission System Operator
TT	Target Topology

Table 34: Overview of acronyms used in this paper.

B Observation spaces

Table 35 shows a quick overview of the observation spaces considered in the experiments and their size.

Observation Space	Total size	Grid2Op features included	Custom features included
Baseline	154	$load_p, gen_p, p_or, p_ex, \rho, timestep_overflow, topo_vect$	
Danger	174	$load_p, gen_p, p_or, p_ex, \rho, timestep_overflow, topo_vect$	$danger$
History	1044	$load_p, gen_p, p_or, p_ex, \rho, timestep_overflow, topo_vect$	$danger$ and 6 time steps of observed values
Complete	387	$load_p, gen_p, p_or, p_ex, load_q, gen_q, q_or, q_ex, load_v, gen_v, v_or, v_ex, load_\theta, gen_theta, \theta_or, \theta_ex, a_or, a_exp, timestep_overflow, topo_vect$	$time_of_day, day_of_year$
D3QN-2022	140	$v_or, v_ex, a_or, a_ex, \rho, topo_vect$	

Table 35: Overview of the size of the observation space that is used as input for the PPO agent in the experiments.

C Results PPO with different hyperparameters

This appendix contains the validation results of a PPO agent trained without an opponent using the hyperparameters that are presented in the column “Value with opponent (PPO*)” in Table 11. The values in this table show how different hyperparameters can influence the performance of the agent.

		Completed episodes	Steps survived	Steps overloaded
Baseline		70.8%	84.45%	0.010%
Action Spaces				
$\mathcal{A}_{sym}$		89.0%	95.14%	0.009%
$\mathcal{A}_{n-1}$		86.6%	93.99%	0.015%
$\mathcal{A}_{d3qn}$		19.0%	47.37%	0.125%
Observation Spaces				
Complete		69.2%	82.77%	0.013%
D3QN_2022		72.8%	86.59%	0.010%
Danger		70.6%	84.40%	0.009%
History		65.2%	81.56%	0.013%
Line loads		66.2%	82.05%	0.013%
Reward functions				
AlphaZero		71.8%	85.85%	0.011%
Constant		71.0%	85.07%	0.011%
SMAAC		70.4%	84.00%	0.014%
Binbinchen		71.4%	85.63%	0.011%
Base + Bonus/Penalty		69.9%	84.52%	0.010%
Curriculum Training				
Level 1: 20000, level 2: 46667		59.8%	80.03%	0.016%
Activation Threshold				
AT	Train AT			
0.8	0.8	0.0%	6.83%	0.497%
	0.9	0.0%	8.46%	0.411%
	0.95	0.0%	6.48%	0.459%
	0.99	0.0%	5.80%	0.499%
	0.8	2.8%	26.44%	0.135%

Continued on next page...

0.9

		Completed episodes	Steps survived	Steps overloaded
	0.9	29.6%	59.07%	0.033%
	0.95	12.6%	50.47%	0.052%
	0.99	3.4%	32.91%	0.091%
0.95	0.8	24.0%	51.50%	0.056%
	0.9	72.4%	84.94%	0.013%
	0.99	54.4%	77.49%	0.019%
0.99	0.8	30.4%	52.57%	0.063%
	0.9	85.0%	90.56%	0.014%
	0.95	91.0%	95.09%	0.008%
	0.99	86.2%	94.92%	0.010%
Line switching				
Final	Train			
No rules	No rules	70.0%	83.87%	0.013%
	Reconnect	71.0%	85.19%	0.009%
	Reconnect + Disconnect	64.6%	82.21%	0.013%
Reconnect	No rules	70.0%	83.65%	0.013%
	Reconnect + Disconnect	64.4%	81.13%	0.012%
Reconnect + Disconnect	No rules	67.8%	83.62%	0.010%
	Reconnect	70.2%	84.58%	0.009%
	Reconnect + Disconnect	65.8%	82.45%	0.012%
Revert Topology				
RT	Train RT			
0.0	0.8	55.6%	78.61%	0.022%
	0.9	66.4%	82.12%	0.014%
	0.95	65.6%	80.47%	0.015%
0.8	0.0	72.8%	85.45%	0.009%
	0.8	60.2%	78.41%	0.014%
	0.9	66.2%	81.19%	0.014%
	0.95	63.4%	79.45%	0.019%
0.9	0.0	72.4%	85.18%	0.010%
	0.8	63.2%	80.07%	0.013%
	0.9	67.4%	80.73%	0.014%
	0.95	66.4%	80.84%	0.018%
0.95	0	72.2%	85.29%	0.011%
	0.8	59.8%	78.62%	0.016%
	0.9	66.2%	81.46%	0.012%
	0.95	63.6%	77.83%	0.016%

Table 36: Validation results of PPO agents with different design choices, trained on an environment without an opponent, using the hyperparameters from PPO* in Table 11.